TECHNISCHE UNIVERSITÄT
CHEMNITZ

Privacy-Preserving Source Code Vulnerability Detection and Repair using Retrieval-Augmented LLMs for Visual Studio Code

Master Thesis

Submitted in Fulfilment of the
Requirements for the Academic Degree
M.Sc. Web Engineering

Dept. of Computer Science
Chair of Computer Engineering

Submitted by: Md Hafizur Rahman
Student ID: 810641
Date: 14.05.2026

Internal Supervisor: Abubaker Gaber M.Sc.
Internal Examiner: Dr.-Ing. Sebastian Heil

Abstract

Static analysis tools miss issues that need contextual reasoning, while cloud Large Language Models (LLMs) introduce privacy risks because source code is sent off-device. This thesis combines locally running LLMs with Retrieval-Augmented Generation (RAG) in a VS Code extension to detect and repair vulnerabilities entirely on the developer's machine, with locally stored CWE, CVE, and OWASP knowledge.

The system is evaluated on 101 JavaScript/TypeScript cases (71 vulnerable, 30 secure) with three runs per sample. Detection is measured by precision, recall, F1, and sample-level secure-FPR; usability by end-to-end latency; repair quality by both manual review and an automated auto-applicable rate. The best configuration, `qwen3:8b` with RAG, reaches F1 71.94%, precision 73.53%, recall 70.42%, FPR 10.00%, and 1 573 ms median latency under canonical-taxonomy matching. An inter-run confidence gate at $\geq 2/3$ agreement lifts precision to 77.78% at the cost of about one recall point. Auto-applicable repair rate is 0% across all 297 generated repairs: the model fills the suggested-fix field with prose rather than executable code, operationalising a known field-wide weakness in automated repair-quality measurement. RAG also degrades some models severely (`codellama+RAG` loses about 30 F1 points), confirming that retrieval augmentation must be calibrated per model.

Overall, the study shows that local privacy-preserving LLM workflows can support useful vulnerability analysis at interactive latency on consumer hardware, while a stricter structured-output contract for repairs remains open for future work.

Keywords: source code security, vulnerability detection, secure code repair, LLMs, RAG, privacy-preserving systems, VS Code

Acknowledgment

I am grateful to everyone who supported me during this Master's thesis.

My sincere thanks go to my internal supervisor, *Abubaker Gaber, M.Sc.*, for his continuous guidance, constructive feedback, and thoughtful discussions throughout the project. His support was essential in shaping both the technical direction and the academic quality of this work.

I also thank the academic staff of the Master's program for providing a strong foundation and a stimulating learning environment.

I am equally thankful to my colleagues and peers for their encouragement and for creating a supportive atmosphere during the study period.

Most of all, I thank my family for their patience, trust, and constant encouragement. Their support made it possible for me to complete this thesis.

Task Description

Traditional static application security testing (SAST) tools, such as Semgrep and CodeQL, are effective for detecting predefined vulnerability patterns but cannot reason about cross-file or context-dependent weaknesses. Cloud-based language model assistants provide stronger semantic analysis yet compromise data confidentiality and reproducibility because proprietary source code must leave the local environment. This thesis aims to design and evaluate a fully local, privacy-preserving secure coding assistant for Visual Studio Code that integrates large language models (LLMs) with Retrieval-Augmented Generation (RAG). All processing occurs on the developer's machine under a strict zero-egress policy to ensure that source code and intermediate data remain confidential.

The proposed assistant combines two complementary modes: real-time inline diagnostics that deliver immediate vulnerability alerts, and asynchronous audit and question–answering modes that use offline retrieval of vulnerability information such as CWE, CVE, and OWASP entries. A modular architecture separates the LLM inference layer, the local retrieval pipeline, and the Visual Studio Code extension interface. The system enforces privacy through local embeddings, signed corpora, and network isolation, while reproducibility is achieved through deterministic decoding, version-pinned dependencies, and containerized execution. The threat model assumes local adversaries capable of attempting code exfiltration, retrieval poisoning, or prompt injection, mitigated through corpus signing, network isolation, and provenance verification.

Evaluation will be conducted on benchmark and real-world JavaScript/TypeScript code. The comparison will be conducted across benchmark datasets (Juliet and OWASP Benchmark, approximately 40–50 test cases mapped to CWE identifiers) and one actively maintained real-world JavaScript/TypeScript project with documented vulnerabilities and verified patches. If the optional SAST baseline is executed, Semgrep and a scoped CodeQL subset will be included for contextual comparison. Evaluation metrics include precision, recall, F1-score, latency, resource usage, privacy validation, and reproducibility, all measured under deterministic local execution.

Contents

Abstract . . . . . **ii**

Acknowledgment . . . . . **iii**

Task Description . . . . . **iv**

Contents . . . . . **v**

List of Figures . . . . . **xi**

List of Tables . . . . . **xiii**

List of Abbreviations . . . . . **xv**

1. Introduction **1**

1.1. Problem Description 1

1.2. Motivating Scenario 1

1.3. Scope of the Thesis 2

1.4. Objectives 4

1.5. Thesis Outline 5

2. Analysis **6**

2.1. Requirements Analysis 6

2.1.1. R1: Accuracy 7

2.1.2. R2: Consistency 8

2.1.3. R3: Repair Quality 10

2.1.4. R4: Usability 11

2.1.5. R5: Privacy . . . 12
2.2. Related Work . . . 13
2.2.1. Traditional Static Application Security Testing . . . 13
2.2.2. LLM-Based Vulnerability Detection and Repair . . . 15
2.2.3. Retrieval-Augmented Generation for Secure Coding . . . 17
2.2.4. Privacy-Preserving and IDE-Integrated Approaches . . . 19
2.2.5. Positioning of This Thesis . . . 21
2.2.6. Critical Comparison Across Paradigms . . . 23
2.3. Summary . . . 23

3. Concept . . . 25
3.1. Concept Derivation from Analysis Results . . . 25
3.1.1. From Cloud-Based to Privacy-Preserving Local Deployment . . . 25
3.1.2. Retrieval-Augmented Generation for Security Knowledge Grounding . . . 26
3.1.3. IDE Integration, Modular Components, and Two-Stage Pipeline 26
3.1.4. Context-Aware Analysis and Structured Explainability . . . 27
3.1.5. Deterministic Inference and Consensus Filtering . . . 27
3.2. System Components . . . 27
3.2.1. Context Extraction . . . 28
3.2.2. Knowledge Retrieval (RAG) . . . 28
3.2.3. Vulnerability Detection . . . 28
3.2.4. Repair Generation and IDE Integration . . . 29
3.2.5. Supporting Subsystems . . . 29
3.3. Detection Workflows . . . 30
3.3.1. Real-Time Function-Level Diagnostics . . . 30
3.3.2. On-Demand File Diagnostics . . . 30
3.3.3. Interactive Analysis and Follow-up Q&A . . . 31
3.3.4. Workspace Scan and Security Dashboard . . . 31
3.3.5. Repair Suggestion Workflow . . . 32

3.3.6. Confidence Abstention and Hybrid Gating 32
3.3.7. Workflow Selection in Practice 33
3.4. System Architecture and Design 33
3.4.1. Context View: System Boundary and External Interactions . . 33
3.4.2. Container View: Internal Component Structure 34
3.4.3. Process View: End-to-End Workflows 37
3.4.4. Sequence Diagrams: Concrete Detection Flows 39
3.5. Summary 43

4. Implementation 44
4.1. Technology Stack and Rationale 44
4.2. System Workflow 47
4.3. Core Component Implementation 49
4.3.1. Context Extraction and Scoping 49
4.3.2. LLM Analyser (Local JSON-Only Output) 49
4.3.3. Diagnostics Mapping 50
4.3.4. RAG Manager and Knowledge Updates 50
4.3.5. Analysis Cache 50
4.3.6. Workspace Scanner and Dashboard 51
4.3.7. Privacy Boundary and Threat Model 51
4.4. User Interface 51
4.4.1. Inline Diagnostics and Hover Explanations 51
4.4.2. Quick Fixes for Repair Suggestions 52
4.4.3. Interactive Analysis View and Contextual Q&A 52
4.4.4. Workspace Security Dashboard 53
4.4.5. Model and RAG Controls 53
4.4.6. Human Factors and Safe Adoption 54
4.5. Repair Safety and Limitations 54
4.5.1. Why Automated Repair Verification Was Deprioritised 54
4.5.2. Repair Safety Mechanisms 55

4.5.3. Observed Repair Patterns 55
4.6. Privacy and Threat-Model Enforcement 56
4.6.1. Network Policy and Zero-Egress Gate 56
4.6.2. Signed Corpus and Provenance 56
4.6.3. Containerised Harness 57
4.7. Implementation Summary 57

5. Evaluation 58
5.1. Datasets 58
5.2. Evaluation Metrics 60
5.3. Experimental Setup 62
5.3.1. Reproducibility Snapshot and Run Configuration 64
5.4. R1: Accuracy 65
5.4.1. Detection Quality Across Configurations 65
5.4.2. RAG Ablation Impact on Accuracy 66
5.4.3. Per-Category Detection Breakdown 67
5.4.4. Qualitative Error Analysis on Zero-Recall Categories 67
5.4.5. SAST Baseline Comparison 68
5.4.6. Canonical Matching 69
5.4.7. Inter-Run Confidence Gate 70
5.4.8. External-Corpus Stress Test 70
5.4.9. Real-World Whole-Project Run on OWASP NodeGoat 71
5.4.10. Level Mapping 72
5.5. R2: Consistency 73
5.5.1. Structured Output Stability 73
5.5.2. Inter-Run Detection Agreement 74
5.5.3. Label and Severity Stability 74
5.5.4. Level Mapping 75
5.6. R3: Repair Quality 76
5.6.1. Repair Generation Rate and Correctness 76

5.6.2. Repair Quality by Vulnerability Type 77
5.6.3. Representative Examples 78
5.6.4. Level Mapping 78
5.6.5. Auto-Applicable Rate 79
5.7. R4: Usability 80
5.7.1. Latency Across Configurations 80
5.7.2. Latency Component Breakdown 81
5.7.3. Resource Usage 81
5.7.4. Latency Feasibility for IDE Workflows 82
5.7.5. Level Mapping 82
5.8. R5: Privacy 83
5.8.1. Verification Results 83
5.8.2. Evidence 83
5.8.3. Level Mapping 85
5.9. Reproducibility 86
5.9.1. Method 86
5.9.2. Results 86
5.10. Summary 87

6. Discussion 90
6.1. Result Analysis 90
6.2. Comparison with Related Work 91
6.3. Deviations from the Approved Task 92
6.4. Limitations 93
6.5. Summary 94

7. Conclusion 95
7.1. Summary of Contributions 95
7.2. Key Findings and Answers to Research Questions 95
7.3. Implications for Practice 96

7.4. Future Work 98
7.5. Conclusion 99

Bibliography **100**

A. Implementation Details **106**
A.1. Prompt Contract (JSON-Only Output) 106
A.2. Runtime Guardrails 106
A.3. Key Configuration Options 107
A.4. VS Code Commands (Prototype) 107
A.5. Evaluation Harness Location 108

B. Evaluation Details **109**
B.1. Curated Test Suite Overview 109

C. Detailed Case Studies **112**
C.1. Case 1: True Positive with Effective Repair 112
C.2. Case 2: False Positive (Over-Sensitivity) 112
C.3. Case 3: False Negative 113
C.4. Case 4: Detection with Partial Repair 113
C.5. Case 5: Multi-CWE Detection 113

D. Privacy Verification Evidence **115**
D.1. Network Traffic Analysis 115
D.2. Privacy Boundary Architecture 117

Declaration of Originality **118**

List of Figures

3.1. C4 Context diagram showing Code Guardian within its operational environment. The system communicates with VS Code (IDE platform), Ollama (local LLM on localhost:11434), and a local vector database, all within the privacy boundary (green dashed). CVE/NVD APIs (gold, dashed) are the only optional external connection and do not receive source code. 34

3.2. C4 Container diagram showing internal components. The VS Code Extension handles triggers, caching, and UI rendering. The Local Analysis Backend performs LLM inference with JSON-mode and multi-pass consensus filtering, RAG orchestration, and Stage 2 repair generation. Vector Database and Result Cache are local data stores. All source code stays within the system boundary. 36

3.3. Process view showing two primary workflows with swim lanes. Workflow 1: Real-time function analysis with debouncing (800 ms) and caching, where cache hits bypass LLM inference and cache misses invoke local analysis. Workflow 2: On-demand analysis with optional RAG; when enabled, the backend generates embeddings, performs vector search (runtime default k=3; evaluation uses k=5), and augments the prompt before LLM invocation. When RAG is disabled, analysis proceeds directly to LLM inference without retrieval. Color coding: user events (yellow), extension (orange), backend (purple), LLM (green). Privacy boundary (green dashed): all processes execute on localhost. 38

3.4. Inline mode with cache hit: no LLM invocation when a previously analysed function reappears. 40

3.5. Audit mode with RAG: vector search, two-pass consensus detection, then Stage-2 repair generation. 41

3.6. Real-time detection with debouncing: an 800 ms timer prevents redundant LLM invocations during continuous typing. 42

5.1. Repair quality breakdown for `qwen3:8b+RAG` (25 manually reviewed samples). Most repairs are semantically correct and strategy-aligned, but less than half contain directly executable code. 77

D.1. Local privacy boundary for code analysis workflows. 117

List of Tables

1.1. Threat model summary for Code Guardian. 4

2.1. Requirements overview for Code Guardian. 6

2.2. Traceability from requirements to harness measures. 7

2.3. Evaluation scale for R1: Detection Accuracy (example thresholds). . . 8

2.4. Evaluation scale for R2: Detection Consistency (example thresholds). 9

2.5. Evaluation scale for R3: Repair Quality (example thresholds). 11

2.6. Evaluation scale for R4: Usability (example thresholds). 12

2.7. Verification checklist for R5: Privacy. 13

2.8. Evaluation of traditional SAST tools against requirements R1–R5. . . 14

2.9. Evaluation of LLM-based vulnerability detection approaches against requirements R1–R5. 17

2.10. Evaluation of RAG-based secure coding approaches against requirements R1–R5. 19

2.11. Evaluation of privacy-preserving and IDE-integrated approaches against requirements R1–R5. 20

2.12. Positioning comparison with representative security assistance approaches. 21

2.13. Unified requirement comparison of all reviewed approaches and Code Guardian. 22

2.14. Critical comparison of major security-assistance paradigms. 23

5.1. Run configuration for the headline evaluation. 65

5.2. Detection accuracy across all configurations using canonical-taxonomy matching (Section 5.2). 71 vulnerable + 30 secure samples, 3 runs each ($n = 303$ evaluations per model×mode). FPR is at the secure-sample level (FP if any issue is emitted in any run). 66

5.3. Residual zero-recall categories after canonical matching (`qwen3:8b+RAG`, 3-run pooled). 68

5.4. Canonical-matching cross-check on the previously-published 71-vulnerable-sample ablation log (no model re-invocation). The substring column reproduces the previously-published values; the canonical column is the same per-case findings re-scored against the canonical taxonomy. FPR is unchanged because secure-sample scoring is type-agnostic at the case level. 69

5.5. Confidence gate sweep on the frozen 3-runs-per-vulnerable-sample log (`qwen3:8b+RAG`, canonical matcher). At threshold ≥ 0.67, only findings agreed by ≥ 2 of 3 runs survive. 70

5.6. `qwen3:8b` on the external corpus, LLM-only / 1 run per sample. Comparison row reproduces the curated 101-case canonical-match headline from Section 5.4.6. 70

5.7. NodeGoat whole-project scan summary. Documented-vulnerability list is the curated 10-entry OWASP-Top-10 mapping under `evaluation/realworld/nodegoat`

5.8. Inter-run detection agreement (3 runs per sample, 71 vulnerable samples). 74

5.9. Label and severity stability among consistently detected samples (3/3 runs). 75

5.10. Repair generation rate by vulnerability type (`qwen3:8b+RAG`). 77

5.11. End-to-end latency per analysis request (milliseconds), pooled across the 303 evaluations per model×mode. 80

5.12. Per-inference harness-side resource footprint for `qwen3:8b` on a 30-case sample (LLM-only, single-run per sample; the headline configuration's wall-clock numbers are reproduced from Section 5.7). CPU columns are harness-process CPU only and exclude the Ollama-server cost, which is recorded separately by operating-system process accounting. 81

5.13. Latency feasibility for IDE workflow modes. 82

5.14. Privacy verification results for R5. 83

5.15. Requirement compliance summary for Code Guardian (`qwen3:8b+RAG`). 88

7.1. Recommended deployment profiles from thesis results. 97

D.1. Privacy-relevant configuration parameters in the evaluated setup. . . 116

List of Abbreviations

AST	Abstract Syntax Tree
CVE	Common Vulnerabilities and Exposures
CWE	Common Weakness Enumeration
F1	F1 Score
FPR	False Positive Rate
HNSW	Hierarchical Navigable Small World
IDE	Integrated Development Environment
LLM	Large Language Model
LRU	Least Recently Used
OWASP	Open Worldwide Application Security Project
RAG	Retrieval-Augmented Generation
SAST	Static Application Security Testing
VS Code	Visual Studio Code

1. Introduction

Injection flaws, cross-site scripting, and insecure data handling remain among the most frequently reported weakness classes in industry surveys [1, 2]. Two families of tools currently attempt to catch them before production. Static Application Security Testing (SAST) tools such as Semgrep and CodeQL scan source code for known patterns and integrate naturally into CI/CD pipelines [3, 4]; Large Language Models (LLMs) have more recently been applied to detection and repair suggestion [5, 6]. Neither family delivers all three properties developers need at once.

1.1. Problem Description

SAST tools are deterministic and privacy-preserving but shallow: they match syntactic patterns without understanding context, cannot explain *why* a finding matters, and rarely suggest how to fix it. The false-positive rate is high enough that developers routinely ignore the output under delivery pressure [7, 8]. LLM-based assistants close both gaps — contextual reasoning and concrete repair — but the dominant offerings transmit source code to cloud servers, which is unacceptable for organisations bound by confidentiality requirements, regulations such as GDPR, or air-gapped environments [9, 10]. The result is a three-way trade-off no current tool resolves: SAST gives determinism and privacy but not reasoning or repair; cloud LLM assistants give reasoning and repair but not privacy. A developer who needs all three has no off-the-shelf option. This thesis explores whether a local, modular design — keeping inference on the developer's machine, grounding model reasoning in curated security knowledge, and integrating into the editor with developer-controlled remediation — can close that gap.

1.2. Motivating Scenario

To illustrate these challenges in a realistic development setting, consider a typical workflow at a fintech company building a Node.js backend.

A developer is implementing an Express.js endpoint that retrieves user account details. The handler reads a user ID from the request query string and constructs a

SQL query to look up the account. The code is straightforward, the tests pass, and the pull request is approved by a colleague:

Listing 1.1: Vulnerable Express.js endpoint with SQL injection

```
app.get('/api/account', (req, res) => {
  const userId = req.query.id;
  const query = "SELECT * FROM accounts WHERE id = '" + userId + "'";
  db.query(query, (err, results) => {
    res.json(results);
  });
});
```

The query is built by string concatenation. An attacker who sends `id=' OR '1'='1` retrieves every account in the database. No one notices the vulnerability until a penetration test three months later.

The scenario surfaces every arm of the three-way trade-off in a single endpoint: a linter (ESLint) stays silent because the concatenation is syntactically valid; a SAST scanner (Semgrep) at best produces a generic "potential SQL injection" warning without grounding in the specific `req.query` → `db.query` data flow; a cloud LLM assistant (GitHub Copilot) explains and repairs the issue but requires transmitting the source to an external server. No tool provides context-aware detection, actionable repair, and privacy together. A local AI assistant integrated into VS Code closes this gap. As the developer writes the endpoint, the assistant analyses the enclosing function, identifies the unsanitised `req.query.id` flowing into a SQL string, and produces:

> **SQL Injection (CWE-89):** User-controlled input `req.query.id` is concatenated into a SQL query without sanitization. Replace with a parameterized query: `db.query("SELECT * FROM accounts WHERE id = ?", [userId])`.

The finding appears as an inline diagnostic in the editor. The developer hovers to read the explanation, clicks the quick-fix action to preview the parameterized query, and applies the patch — all without the code leaving the machine. This is what Code Guardian provides: contextual detection, grounded explanation, and developer-controlled repair, running entirely on localhost.

1.3. Scope of the Thesis

This thesis focuses on **JavaScript/TypeScript** projects in **Visual Studio Code**. The primary privacy goal is **no source-code exfiltration**: all code and IDE context are processed exclusively by local services running on the developer's machine. The

system is designed as a VS Code extension that connects to a local Ollama instance for LLM inference, with an optional retrieval-augmented generation (RAG) module backed by a local HNSW vector store for security knowledge grounding.

The technical scope includes local LLM inference via Ollama, function-level code scoping, structured JSON output enforcement, and retrieval-augmented prompting with curated security knowledge bases (CWE descriptions, OWASP guidance). No model fine-tuning or training is performed; the focus is on system-level orchestration — prompt design, output parsing, caching, and IDE integration — using off-the-shelf local models served through Ollama [11].

The evaluation is limited to the accuracy, consistency, and latency of the structured output produced from code snippets in a controlled test harness. This thesis does not cover areas such as multi-language support beyond JavaScript/TypeScript, real-time collaborative editing, endpoint hardening, hardware-level attacks, or enterprise identity and network controls. Optional knowledge-base refresh may fetch *public* vulnerability metadata (for example CVE/CWE descriptions) and can be disabled for fully offline operation.

Research Questions

The thesis addresses three research questions:

- **RQ1 (Feasibility):** To what extent can a locally deployed LLM integrated into VS Code provide useful vulnerability detection and repair suggestions without transmitting source code to external services?
- **RQ2 (Grounding):** How does retrieval-augmented prompting influence detection quality and the qualitative grounding of explanations compared to an LLM-only configuration?
- **RQ3 (Practicality):** What performance mechanisms (scoping, caching, debouncing) are necessary to make LLM-based security feedback usable within interactive IDE workflows?

Threat Model and Trust Assumptions

The main threat addressed is unintended disclosure of proprietary code through external analysis services. Local inference reduces this risk surface but does not remove all security risks.

Table 1.1.: Threat model summary for Code Guardian.

Threat class	Assumption / attack path	Design response in this thesis
Source-code exfiltration	External API calls could expose proprietary code or derived context.	Local inference only; prompts and code remain on localhost in evaluated workflows.
Prompt injection	Attacker-controlled code/-comments attempt to override analysis instructions [12].	Strict JSON contract, defensive parsing, and user approval before applying any repair.
Retrieval poisoning	Low-quality or malicious knowledge entries reduce grounding quality.	Curated local knowledge sources and explicit retrieval scope; stronger provenance controls left for future work.
Local host compromise	Malware or untrusted local users access local code, prompts, or caches.	Out of scope; assumes trusted host and OS-level hardening.

1.4. Objectives

The main objective of this thesis is to design and evaluate an IDE-integrated assistant — called Code Guardian — that provides contextual vulnerability detection and repair suggestions while preserving source-code confidentiality. The system aims to overcome the limitations of existing approaches by combining local LLM inference with retrieval-augmented generation in a privacy-preserving architecture that runs entirely on the developer's machine.

To achieve this goal, the following three specific objectives are defined:

1. **Design a privacy-preserving VS Code extension for local vulnerability analysis** that separates the process into a two-stage pipeline: detection followed by developer-controlled repair suggestion. The extension enforces structured JSON output for machine-checkable diagnostics, integrates an optional RAG module backed by curated security knowledge (CWE descriptions, OWASP guidance), and implements performance mechanisms — function-level scoping, LRU caching, and debounced triggering — for practical IDE interaction latency.
2. **Evaluate system performance across local model configurations** using a documented evaluation harness with 101 test cases spanning 14 CWE categories. Five local models are tested in both LLM-only and LLM+RAG modes, reporting precision, recall, F1, false-positive rate, JSON parse success, and end-to-end latency. Results from all configurations are compared to identify the most effective trade-off between detection quality, output robustness, and response time.

3. **Assess the practical impact of retrieval-augmented prompting** by comparing LLM-only and LLM+RAG configurations to determine whether grounding model reasoning in external security knowledge improves detection quality and explanation grounding, or whether the added retrieval context introduces noise that degrades specific model configurations.

These objectives guide the development and validation of a system that is not only technically functional but also practical for interactive developer workflows. Chapter 5 identifies the strongest configuration among those tested and reports its trade-offs across accuracy, consistency, latency, and privacy.

1.5. Thesis Outline

Following this introduction, the thesis begins with a review of related work on static analysis tools, LLM-based vulnerability detection, and retrieval-augmented generation for secure coding. The literature is analyzed to identify current limitations and to derive requirements for a system that combines contextual reasoning, repair generation, and source-code privacy. This provides the conceptual foundation for understanding the need for a tool like Code Guardian.

The next chapter introduces the overall system concept. It presents the architecture of Code Guardian together with key design decisions such as the two-stage detection-then-repair pipeline, structured JSON output enforcement, and the optional RAG module. After outlining the concept, the implementation chapter describes the VS Code extension, technology choices, and how core components — such as the analyzer, RAG manager, analysis cache, and diagnostic rendering — were developed, with an emphasis on modularity and local execution.

The evaluation chapter examines system performance across five local models in both LLM-only and LLM+RAG configurations, using metrics such as precision, recall, F1, false-positive rate, JSON parse success, and latency. The thesis concludes by summarizing key findings, discussing limitations, and outlining opportunities for future work. Appendices provide supplementary material including prompt templates, detailed evaluation data, implementation details, case studies, and privacy verification evidence.

2. Analysis

This chapter defines the requirements for automated vulnerability detection and repair assistance in an IDE and positions the thesis approach against related work. Following the style of the reference theses, the chapter first derives the requirements baseline, then reviews existing approaches, and finally summarizes the implications for the proposed concept.

2.1. Requirements Analysis

The thesis targets local IDE assistance for JavaScript/TypeScript development: detect vulnerabilities, explain them, and propose repairs without code exfiltration. Compared with rule-based SAST, the approach adds LLM reasoning and optional retrieval grounding, but must still satisfy strict operational constraints.

Based on the problem statement and related work, five requirements define the evaluation baseline. Table 2.1 summarizes them before they are discussed in detail.

Table 2.1.: Requirements overview for Code Guardian.

ID	Requirement	Operational intent
R1	Accuracy	Find real vulnerabilities with useful precision, recall, and controlled false positives, using context-aware reasoning.
R2	Consistency	Produce stable findings and structured output under fixed settings.
R3	Repair quality	Suggest concrete, security-improving fixes while keeping the developer in control.
R4	Usability	Deliver feedback at latencies that fit normal IDE workflows.
R5	Privacy	Keep all analysis local with no source-code transmission.

The following subsections define each requirement in detail and map it to measurable criteria. Table 2.2 traces the concrete harness measures for each requirement; threshold scales and per-configuration values are in the corresponding subsections and in Chapter 5.

Table 2.2.: Traceability from requirements to harness measures.

ID	Requirement	Harness measures
R1	Accuracy	Precision, recall, F1, sample-level secure FPR (Section 5.4); canonical-taxonomy type matching with per-canonical-category P / R / F1; inter-run confidence and confidence-gated metrics (Sections 5.4.6, 5.4.7).
R2	Consistency	JSON parse success rate; inter-run detection agreement (%); label and severity agreement (%) over 3-run subsamples (Section 5.5).
R3	Repair quality	Manual review of repair fix rate, semantic correctness, strategy alignment, executable-code share on a 25-sample subset (Section 5.6); auto-applicable rate (`@babel/parser` syntax check on every `suggestedFix`) plus `byReason` breakdown over the full set (Section 5.6.5).
R4	Usability	End-to-end latency (mean and median, ms) per configuration (Section 5.7); stage-split components `inferenceTimeMs` and `parseTimeMs` recorded per call.
R5	Privacy	Five-item architectural verification checklist (Section 5.8).

Every measure is computed by the harness from per-case logs; only the R3 manual-review portion is explicitly hand-aggregated.

2.1.1. R1: Accuracy

R1 addresses accuracy. Code Guardian should find real vulnerabilities while keeping false alarms low. In an IDE workflow, accuracy means not just finding issues, but producing results that developers can trust and act on. This includes context-aware reasoning: the system should tell apart code that is actually vulnerable from code that only looks suspicious, by considering surrounding logic like input validation and sanitization.

This requirement comes from a key trade-off in security tools. Traditional tools often produce too many false warnings because they match surface patterns without understanding context [7, 4]. LLM-based systems can both over-warn and miss real issues, depending on the model and prompt [13, 14]. Without enough context, they may give confident but wrong explanations. A useful tool must be judged by how correct its findings are, not just by whether it can explain them.

For this thesis, R1 means detection quality must be good enough to deserve developer attention under local deployment. This includes recognizing when mitigation is already present and avoiding false conclusions when context is missing. The design uses scoped code extraction, structured prompts, optional retrieval grounding, comparison

with rule-based tools, and clear reporting of precision, recall, F1, and false-positive rate (FPR).

Accuracy has four key parts. Precision measures how many reported issues are real, since too many false alarms cause developers to ignore the tool. Recall measures how many real vulnerabilities the system finds, including those that depend on data flow and surrounding context. F1 balances both into a single score. The secure-sample FPR tracks how often the system flags safe code as vulnerable, which is the best predictor of whether developers will accept the tool [7].

To allow comparison with other tools and across Code Guardian's own settings, the thesis defines a four-level scoring scale. The thresholds follow standard conventions from information retrieval and software engineering [7, 13]. Table 2.3 shows this scale.

Table 2.3.: Evaluation scale for R1: Detection Accuracy (example thresholds).

Visual Score	**Interpretation (with thresholds)**
	**High accuracy.** F1 ≥ 0.75, precision ≥ 0.70, recall ≥ 0.70, and FPR ≤ 0.20. The system finds vulnerabilities well with few false alarms. Suitable for always-on IDE use.
	**Medium accuracy.** F1 in $[0.60, 0.75)$, precision ≥ 0.55, recall ≥ 0.55, and FPR in $(0.20, 0.40]$. Results are worth reviewing with some manual checking needed. Suitable for on-demand or audit-mode use.
	**Low accuracy.** F1 in $[0.45, 0.60)$, or precision/recall below 0.55, or FPR in $(0.40, 0.70]$. Findings need heavy manual checking. Only useful alongside rule-based tools.
	**Insufficient accuracy.** F1 < 0.45 or FPR > 0.70. Detection is too weak or false alarms too frequent. The system is not useful in practice.

2.1.2. R2: Consistency

R2 addresses detection consistency. Code Guardian should produce stable results when the same code is analyzed multiple times with the same settings. In security analysis, unstable output quickly reduces trust and makes it harder for developers to act on findings.

This requirement matters because LLMs are nondeterministic. They can produce different outputs, schema errors, or changed labels across runs [13, 6]. Even if detection quality is good, inconsistent output makes a tool unsuitable for IDE use. Developers need to trust that warnings will not change randomly between runs [7].

Consistency has four key parts. First, structured output: the system must return valid JSON that follows the expected schema, since broken output is useless for IDE automation. Second, detection agreement: running the same code twice should produce the same findings. Third, labeling: vulnerability types and severity ratings should stay the same across runs. Fourth, localization: reported line ranges should not shift between runs.

For this thesis, R2 means stable JSON output, stable findings across repeated runs, and repeatable labels and locations. The design uses a strict JSON contract, defensive parsing, scoped prompts, caching, and optional retrieval grounding to reduce random variation.

A four-level scale is defined for R2. Output stability is measured by JSON parse success rate. Detection agreement is measured by inter-run agreement rate (the fraction of vulnerable samples for which all N repeated runs agree on the binary detection outcome). Label and severity stability are measured as raw agreement percentages over samples detected in all runs. Cohen's κ is not computed because it requires a baseline-rate adjustment that varies across the unbalanced per-category sub-populations in the dataset; the raw agreement percentages reported here are interpretable directly and align with how the harness computes them. Table 2.4 shows this scale.

Table 2.4.: Evaluation scale for R2: Detection Consistency (example thresholds).

Visual Score	**Interpretation (with thresholds)**
	**High consistency.** JSON parse rate $\geq 99\%$, detection agreement $\geq 90\%$, label / severity agreement $\geq 90\%$. Output is stable and reliable. Suitable for always-on IDE use.
	**Medium consistency.** JSON parse rate in $[95\%, 99\%)$, detection agreement in $[75\%, 90\%)$, or label / severity agreement in $[75\%, 90\%)$. Mostly stable with occasional changes. Suitable for on-demand or audit-mode use.
	**Low consistency.** JSON parse rate in $[85\%, 95\%)$, detection agreement in $[60\%, 75\%)$, or label / severity agreement in $[60\%, 75\%)$. Frequent variation. Findings need repeated confirmation before acting.
	**No consistency.** JSON parse rate $< 85\%$, detection agreement $< 60\%$, or label / severity agreement $< 60\%$. Output is unreliable. The system cannot produce repeatable findings.

2.1.3. R3: Repair Quality

R3 addresses repair quality. Code Guardian should not only detect vulnerabilities but also suggest concrete fixes. A good repair is specific to the affected code, follows recognized secure-coding practices, and is presented as a suggestion that the developer can review and accept or reject.

This matters because developers often receive warnings without usable guidance on how to fix them, which increases effort and reduces follow-through [7, 8]. At the same time, research on automated program repair shows that generated patches are only useful when they fix the root cause without introducing new problems [15].

For Code Guardian, R3 means repairs should target the affected code region, use established mitigations like parameterization, input validation, and safer API choices, and be delivered as optional diffs rather than automatic edits. The developer always keeps control over whether to apply a fix.

Repair quality is measured along two complementary axes. The first is a qualitative manual review on a sampled subset, scoring each repair on whether it addresses the reported vulnerability type, follows recognized secure-coding patterns, and is presented as syntactically applicable code rather than prose guidance (Section 5.6). The second is the deterministic, fleet-wide *auto-applicable rate*: the fraction of generated `suggestedFix` strings that syntactically parse as JavaScript or TypeScript without errors (defined in Section 5.2). The manual review captures *decision-support quality* (would a developer benefit from reading this?); the auto-applicable rate captures *automation quality* (could the IDE quick-fix surface insert this without further editing?). Both are reported.

The threshold scale for R3 below combines them: the rate qualifier targets the manual-review proportion, with the auto-applicable rate reported as a secondary signal in the corresponding evaluation section. Unlike R1 and R2, which use four levels, a coarser three-level scale is appropriate here because repair quality is partly qualitative and fine-grained distinctions are not reliable on small reviewer samples. Table 2.5 shows this scale.

Table 2.5.: Evaluation scale for R3: Repair Quality (example thresholds).

Visual Score	Interpretation (with thresholds)
●	**High.** $\geq 75\%$ of repairs are syntactically valid, address the correct vulnerability type, and follow established secure-coding patterns. Fixes are specific and ready for developer review.
◑	**Medium.** $[50\%, 75\%)$ of repairs are valid and relevant. Most fixes target the right issue but may use generic patterns or need minor edits before applying.
○	**Low.** $< 50\%$ of repairs are valid and relevant. Fixes are mostly unusable, too generic, or miss the root cause.

2.1.4. R4: Usability

R4 addresses usability. Security feedback is only useful if it arrives fast enough to fit normal editing workflows. A tool that takes too long will be ignored, no matter how accurate it is.

This is especially important for local LLM-based systems because model inference, retrieval, and prompt processing can make analysis noticeably slower than traditional diagnostics [16, 17]. Growing delay breaks the developer's flow and reduces willingness to use the tool.

For Code Guardian, R4 means that both inline and on-demand analysis must stay practical on real hardware. The design uses function-level scoping, debouncing, caching, and separate modes for lightweight inline checks versus heavier full-file scans.

Usability is measured on a three-level scale based on end-to-end latency. End-to-end latency is the wall-clock time from request submission to parsed result. The harness additionally records this latency in two components — `inferenceTimeMs` (the duration of the local model call) and `parseTimeMs` (the duration of the response parser) — so that the source of any observed latency change can be attributed unambiguously when comparing configurations. The threshold scale itself uses end-to-end median latency, since that is what the developer actually experiences. The thresholds reflect established response-time guidelines from HCI research [16, 17]. Table 2.6 shows this scale.

Table 2.6.: Evaluation scale for R4: Usability (example thresholds).

Visual Score	Interpretation (with thresholds)
●	**High.** Median latency $\leq$ 5s for inline analysis, $\leq$ 15s for full-file scans. Feedback feels responsive and fits normal editing flow.
◑	**Medium.** Median latency in $(5\text{s}, 15\text{s}]$ for inline, $(15\text{s}, 30\text{s}]$ for full-file. Noticeable delay but still practical for on-demand use.
○	**Low.** Median latency $>$ 15s for inline or $>$ 30s for full-file. Too slow for regular IDE use. Only practical for planned audit scans.

2.1.5. R5: Privacy

R5 addresses privacy. Code Guardian must detect and repair vulnerabilities without sending source code to external services. All prompts, findings, retrieved context, and generated fixes should stay on the developer's machine.

This matters because source code often contains proprietary logic or regulated information. Many organizations cannot use cloud-based analysis tools for this reason. Partial redaction does not solve the problem, since security-relevant context is spread across the code and may be lost when stripped.

For Code Guardian, R5 means local model inference, local retrieval, and clear separation between code-bearing workflows and optional public-metadata refresh paths. The design uses Ollama for local inference, local vector storage for RAG, and disabled-by-default background data updates.

Unlike the other requirements, privacy is a design constraint rather than a spectrum. Table 2.7 defines the verification checklist.

Table 2.7.: Verification checklist for R5: Privacy.

	Privacy criterion
☐	LLM inference runs locally via Ollama. No source code is sent to cloud APIs.
☐	RAG vector store and embeddings are stored locally. No code-derived data leaves the machine.
☐	Analysis prompts and results stay on localhost. No telemetry or external transmission.
☐	Optional vulnerability data refresh uses only public metadata (NVD, OWASP, CWE) and can be disabled for fully offline operation.
☐	No source code or code fragments are included in any outbound network request.

2.2. Related Work

This section reviews prior work on SAST, LLM-based security assistance, retrieval-augmented prompting, and privacy-preserving IDE integration, then positions the thesis approach against these paradigms.

2.2.1. Traditional Static Application Security Testing

Static Application Security Testing (SAST) tools remain the default workhorse for vulnerability detection in many development workflows. Rule-based analyzers and taint-analysis systems such as Semgrep and CodeQL statically inspect source code for predefined patterns and data-flow queries [18, 19]. Their main advantage is operational: results are deterministic, reproducible, and easy to integrate into CI/CD pipelines and compliance-driven environments.

From a research perspective, modern static analyzers build on foundational ideas such as abstract interpretation [20] and scalable bug-finding techniques [21]. In practice, large-scale deployments demonstrate that static analysis can find real defects in industrial codebases at massive scale [3]. Security-oriented static analysis has also been studied explicitly, including work that frames static analysis as an effective security engineering control when combined with secure development processes [4, 22].

A second advantage of SAST is *auditability*. Findings are tied to explicit rules or queries, which enables traceable reasoning, stable regression behavior, and predictable security gates. This is particularly relevant in regulated settings, where organizations

need evidence of controls rather than only aggregate accuracy claims. Determinism also simplifies governance: when a rule is updated, teams can review the diff in findings directly instead of inferring changes from opaque model behavior.

These strengths come with well-known limitations. SAST tools can struggle with contextual reasoning and generalization, producing false positives when a risky-looking pattern is guarded elsewhere (e.g., validation in a different function, framework-enforced invariants) and false negatives when vulnerabilities depend on semantic relationships or framework-specific behavior. Language and framework abstractions further complicate exhaustive rule coverage and often push teams toward conservative, noisy rulesets.

There is also a structural precision/recall tension in static analysis deployments. Aggressive rule sets can improve vulnerable-case coverage, but often increase triage burden through noisy alerts; stricter rules reduce noise but risk under-detection. The trade-off is not only technical but organizational: teams with limited security-review capacity may prefer conservative rule sets, while high-assurance environments may tolerate larger alert queues to avoid misses.

Finally, many SAST tools provide limited remediation guidance, requiring developers to interpret findings and design fixes manually. Empirical studies show that warning overload and poor actionability reduce adoption and remediation rates [7, 8]. This gap between *detection* and *remediation support* is one reason why SAST outputs are often strongest as gatekeeping signals, but weaker as day-to-day developer coaching tools.

Table 2.8 evaluates the main SAST tools discussed above against the requirements defined in Section 2.1.

Table 2.8.: Evaluation of traditional SAST tools against requirements R1–R5.

Tool	**R1**	**R2**	**R3**	**R4**	**R5**	**Key Gap**
Semgrep [18]	◑	●	○	●	●	No contextual reasoning or repair generation
CodeQL [19]	◕	●	○	●	●	No repair suggestions; rigid query language
ESLint security plugins	◔	●	○	●	●	Surface-level patterns only; no semantic analysis

All three tools achieve high consistency (R2) because their results are deterministic and reproducible. They also score high on usability (R4) due to fast execution and CI/CD integration, and high on privacy (R5) since they run fully locally. However, accuracy (R1) varies: CodeQL's taint analysis provides better context than Semgrep's

pattern matching, while ESLint security plugins cover only surface-level patterns. The main gap is repair quality (R3) — none of these tools suggest concrete fixes, leaving developers to interpret findings and write repairs manually.

These limitations motivate approaches that keep deterministic checks as guardrails while adding more flexible reasoning and repair generation closer to the developer workflow.

2.2.2. LLM-Based Vulnerability Detection and Repair

Recent advances in Large Language Models (LLMs) have renewed interest in using learned models for code understanding, vulnerability detection, and repair suggestions. Contemporary systems build on transformer architectures [23] and large-scale pretraining objectives in the style of BERT/T5 [24, 25]. Both general-purpose LLMs and code-specialized models can generate syntactically plausible code and perform transformations such as refactoring and patch drafting [5, 6]. For IDE-integrated security assistance, these capabilities are attractive because they enable explanations and candidate fixes at the point of code.

However, empirical evaluations show that model-generated code can be *functionally correct yet insecure*, and that probabilistic generation can introduce hallucinated assumptions or omit critical security checks [14, 13]. This matters disproportionately in security, where small omissions (e.g., missing validation, unsafe sinks) can create exploitable weaknesses.

Recent survey work shows both the breadth and the fragmentation of this area. Reviews by Zhang et al. and Gholami cover broad cybersecurity use cases, but both report recurring problems in reproducibility, evaluation quality, false positives, and operational trust [26, 27]. Focusing specifically on code security, Basic and Giaretta show that the literature spans not only detection and repair but also security risks introduced by LLM-generated code itself [28].

A key difference from classical static analysis is that LLM behavior is strongly prompt- and format-dependent. Small changes in instructions, output schema, or contextual examples can materially change both finding rates and false-positive behavior. As a result, measured model quality is not only a property of model weights, but also of engineering choices such as scope selection, schema strictness, retrieval context, and failure handling.

For IDE integration, this dependence has an additional implication: free-form natural language answers are often insufficient for automation. Practical tools typically require machine-checkable outputs (e.g., JSON findings with line spans) and conservative handling of malformed responses. In this thesis, structured-output robustness is therefore treated as a deployment gate, not merely a presentation detail.

Before LLMs, machine-learning-based vulnerability detection explored learning semantic representations of code for classification. Early systems used deep learning over code fragments and patterns [29], while representation-learning work explored embeddings for code structure and semantics [30]. More recent approaches leveraged graph representations to capture richer program semantics [31, 32]. These methods improved over purely lexical baselines, but often required task-specific training data and did not naturally provide developer-facing explanations or repair suggestions.

Finally, security-related failure modes of LLMs extend beyond simple false positives/negatives. The broader LLM risk literature emphasizes both ethical/operational risks [33] and adversarial behaviors such as jailbreaks and prompt-based attacks [34]. For IDE-integrated security tools, these risks motivate strict output contracts, careful prompt design, and conservative integration of model suggestions into developer workflows.

Another practical challenge is *calibration*. Many LLM-based assistants can explain a vulnerability convincingly even when the underlying label is wrong or weakly grounded. Without explicit confidence controls and abstention behavior, this can increase developer over-trust. For security tooling, persuasive explanations are not sufficient; the system must also provide stable structure, traceable evidence, and clear boundaries between high-confidence findings and uncertain hypotheses.

Recent empirical studies sharpen these concerns. Li et al. show that context-poor evaluation can underestimate LLM capability and distort rationale assessment [35]. IRIS shows that LLMs are stronger when coupled with static analysis than when used alone [36]. CWEVAL shows that static-rule scoring can miss or misjudge functionally correct yet insecure outputs, motivating outcome-driven security oracles [37]. SecRepair similarly frames secure repair as a full-pipeline problem spanning data, model adaptation, and evaluation [38].

In response, contemporary approaches increasingly combine learned models with auxiliary signals such as retrieval, tools, or deterministic checks. Retrieval-augmented generation provides a mechanism to ground model reasoning in external security knowledge without retraining [39, 40]. Tool-oriented prompting (e.g., using dedicated retrieval or analysis tools) further motivates modular designs where different components provide complementary evidence [41]. In Code Guardian, these ideas are reflected in the optional RAG module and the strict JSON-output contract used for IDE diagnostics and evaluation.

Repair suggestion quality is also connected to the broader literature on automated program repair. Classic systems such as GenProg and subsequent patch-learning approaches show both the promise of automation and the difficulty of evaluating patch correctness beyond passing tests [42, 15]. For IDE-integrated security assistance, these findings motivate presenting repairs as *optional* quick fixes, requiring explicit developer review and preserving responsibility for functional correctness.

Table 2.9 evaluates representative LLM-based approaches against the requirements defined in Section 2.1.

Table 2.9.: Evaluation of LLM-based vulnerability detection approaches against requirements R1–R5.

Approach	**R1**	**R2**	**R3**	**R4**	**R5**	**Key Gap**
GitHub Copilot [5]	◕	◔	◑	●	○	Cloud-only; source code leaves the machine
IRIS [36]	◕	◑	◑	○	○	Batch-only; no IDE integration or privacy
SecRepair [38]	◑	◑	◑	○	○	No IDE workflow; cloud model assumed
CWEVAL [37]	◕	◑	○	○	—	Evaluation framework only; no repair or IDE

LLM-based approaches show stronger accuracy (R1) than SAST tools due to contextual reasoning, but consistency (R2) is weaker because of nondeterministic generation. GitHub Copilot offers the best usability (R4) through native IDE integration, but fails on privacy (R5) since it requires cloud inference. IRIS and SecRepair provide medium repair quality (R3) by combining static analysis with LLM-generated patches, but neither is integrated into IDE workflows (low R4). CWEVAL is an evaluation framework, not a repair tool, so R3 does not apply. The main gap across all approaches is the lack of local, privacy-preserving deployment.

2.2.3. Retrieval-Augmented Generation for Secure Coding

Retrieval-Augmented Generation (RAG) is a general paradigm for grounding language model outputs in external knowledge without retraining. In RAG, a retriever selects relevant documents for a given query and the generator conditions on those documents when producing an answer [39]. Retrieval can be implemented with lexical scoring functions such as BM25 [43] or with dense retrieval based on semantic embeddings. Dense retrieval approaches such as DPR and retrieval-augmented pretraining (REALM) motivate this design by showing that semantic retrieval can provide effective context for generation [40, 44]. Survey work further highlights RAG as a practical way to reduce hallucination and improve factuality [45, 13].

In secure coding settings, the retrieved context typically corresponds to vulnerability taxonomies (e.g., CWE), secure coding guidelines (e.g., OWASP cheat sheets), and historical vulnerability examples. This fits vulnerability detection and repair well

because many issues are best explained by mapping code patterns to known weakness classes and established mitigations [2, 1]. By grounding the model in such sources, a system can improve detection consistency (R2) and explanation quality (R4), while reducing unsupported guesses.

Recent secure-code generation research suggests that *what* is retrieved matters as much as retrieval itself. RESCUE argues that raw security documents are often too noisy; it instead distills vulnerability-fix data into hierarchical guidance and concise code examples, then retrieves across security facets such as API pattern, vulnerability cause, and code structure [46]. The same lesson is relevant for vulnerability detection: curated security-specific retrieval units are likely to be more useful than generic prose.

Nevertheless, RAG introduces its own failure modes. If retrieval returns weakly related snippets, the generator can become distracted and produce confident but misaligned explanations. If retrieval returns overly generic security text, outputs may drift toward checklist-style warnings that increase false positives. In security tooling, retrieval quality is therefore as important as model quality: grounding helps only when retrieved context is both relevant and specific to the analyzed code pattern.

Implementation-wise, RAG depends on embedding models and efficient nearest-neighbor search. Sentence-level embedding methods such as Sentence-BERT are commonly used to map text into vector space [47]. Approximate nearest-neighbor indices such as HNSW provide high recall with practical latency [48], and libraries such as FAISS popularized large-scale similarity search in practice [49]. In Code Guardian, these ideas are realized through local embeddings and a persistent HNSW vector store to keep retrieval on-device and compatible with IDE latency constraints.

From an engineering perspective, RAG design involves a three-way trade-off among latency, grounding depth, and noise:

- increasing top-k retrieved chunks can improve recall of relevant guidance but expands prompt size and latency;
- larger chunks preserve context but may dilute precision in similarity search;
- aggressive knowledge refresh increases topicality but can introduce quality variance and reduce reproducibility if source curation is weak.

These trade-offs motivate model-specific calibration rather than one global RAG configuration. The same retrieval setup can improve one model while degrading another, depending on how each model integrates external context under strict output constraints. This observation is central to the ablation design in this thesis.

Table 2.10 evaluates representative RAG-based approaches against the requirements defined in Section 2.1.

Table 2.10.: Evaluation of RAG-based secure coding approaches against requirements R1–R5.

Approach	R1	R2	R3	R4	R5	Key Gap
Generic RAG [39]	◕	◑	◑	◑	—	No IDE integration; privacy depends on deployment
RESCUE [46]	◕	◕	●	○	—	No IDE or usability focus; cloud evaluation only

RAG improves accuracy (R1) by grounding LLM reasoning in security knowledge, though the degree of improvement is model-dependent. RESCUE achieves higher repair quality (R3) than generic RAG because it retrieves structured vulnerability-fix pairs rather than raw security text. Consistency (R2) benefits from retrieval grounding but remains medium since the underlying LLM is still nondeterministic. Neither approach reports usability metrics for IDE integration (R4), and privacy (R5) depends on deployment — generic RAG can run locally but is often evaluated with cloud models. The key insight is that RAG is a useful augmentation, not a standalone solution.

2.2.4. Privacy-Preserving and IDE-Integrated Approaches

Privacy concerns pose a significant barrier to adopting LLM-based security tools in real-world settings. Cloud-hosted assistants require transmitting proprietary source code to external servers, which is unacceptable in many regulated or security-sensitive environments [10]. Beyond policy constraints, research has demonstrated concrete privacy risks in large language models, including memorization and extraction of training data [9] and inference attacks that raise questions about model confidentiality and data exposure [50]. For security tooling, these risks motivate designs that keep source code and analysis artifacts local by default.

The IDE context introduces additional security considerations. Because the assistant processes attacker-controlled inputs (e.g., comments, strings, dependency code), prompt-injection and tool-manipulation attacks become relevant; OWASP explicitly catalogs such risks for LLM applications [12]. These concerns motivate designs that treat code as data, constrain outputs to machine-checkable schemas, and isolate retrieval sources to curated security knowledge.

IDE-integrated security tools can improve developer engagement and remediation rates by providing feedback during active development rather than post hoc analysis [7, 8]. However, many IDE assistants prioritize productivity features such as code completion and refactoring, with limited focus on security guarantees, reproducibility,

or privacy boundaries. In practice, local deployment and deterministic interaction contracts become critical: users must understand what data is processed, where it is processed, and how results may vary across models and runs.

Local deployment, however, is not a complete security solution. Moving inference on-device shifts the trust boundary from cloud providers to local runtime integrity, workstation hardening, and extension-level data handling. Attackers who gain local access can still exfiltrate prompts, caches, or generated repairs unless operational controls (OS hardening, access control, and secure storage defaults) are in place.

A second practical boundary concerns *mixed connectivity*. Many systems operate with local inference but optional online knowledge refresh. This hybrid pattern can preserve source-code privacy while still introducing supply-chain and provenance risks if external knowledge sources are not validated. Privacy-preserving design should therefore separate code-bearing data paths from public-metadata update paths and make this separation visible to users.

Table 2.11 evaluates representative privacy-aware and IDE-integrated approaches against the requirements defined in Section 2.1.

Table 2.11.: Evaluation of privacy-preserving and IDE-integrated approaches against requirements R1–R5.

Approach	**R1**	**R2**	**R3**	**R4**	**R5**	**Key Gap**
GitHub Copilot [5]	◕	◔	◑	●	○	No privacy; all code sent to cloud
Snyk Code (local) [51]	◕	●	○	●	◕	No repair suggestions; partial cloud dependency
Semgrep (local) [18]	◑	●	○	●	●	No contextual reasoning or repair generation

GitHub Copilot offers the best IDE experience (R4) and medium repair quality (R3), but fails on privacy (R5) because all inference happens in the cloud. Snyk Code's local mode provides high consistency (R2) and good usability (R4), but its proprietary engine limits transparency and it offers no repair suggestions (low R3). Its privacy score is medium-high because some features still require cloud connectivity. Semgrep runs fully locally (high R5) with deterministic results (high R2), but lacks contextual reasoning and repair generation. The gap across all approaches is the combination of local privacy, contextual LLM reasoning, and repair suggestions in a single IDE-integrated tool.

2.2.5. Positioning of This Thesis

This thesis focuses on privacy-preserving vulnerability detection and repair using locally deployed LLMs integrated into Visual Studio Code, combining retrieval-augmented reasoning with IDE-level context extraction for both real-time and on-demand JS/TS analysis. Unlike fine-tuned or cloud-dependent approaches, the system runs on-device by default, operates fully offline when knowledge refresh is disabled, decouples security knowledge from model parameters, and is evaluated through documented local ablation experiments with quantitative metrics.

Cloud assistants vs. local SAST vs. research prototypes. GitHub Copilot, Amazon CodeWhisperer, and GitHub Advanced Security [5, 52, 53] dominate AI-assisted coding by leveraging cloud infrastructure for completions and increasingly for vulnerability scanning, but they transmit source code to external servers — unacceptable for regulated industries, government projects, or strict-IP environments. Hybrid SAST vendors such as Snyk Code (with a local mode) and Semgrep [51, 18] run on-device and are fast and deterministic but emit only rule identifiers, not natural-language explanations or repair suggestions. Research prototypes such as IRIS and SecRepair [36, 38] combine static analysis with LLM reasoning but evaluate in batch mode on benchmarks, assume cloud-hosted models, and do not report IDE integration latency. Code Guardian makes a different bet: accept weaker local models in exchange for privacy, interactive response times, structured output contracts, and no external dependency. Table 2.12 summarises the differences across the dimensions most relevant to this thesis.

Table 2.12.: Positioning comparison with representative security assistance approaches.

Approach	Privacy	Contextual reasoning	Repair suggestions	IDE integration	Output contract
GitHub Copilot	Cloud-dependent	Strong (LLM)	Yes	Native	Unstructured
Snyk Code (local)	Local analysis	Rule-based	Limited	Plugin	Structured
Semgrep	Fully local	Rule-based	No	CLI/Plugin	Structured
IRIS / SecRepair	Varies	Strong (hybrid)	Yes	Batch-only	Research-grade
Code Guardian	Fully local	LLM + RAG	Yes	Native VS Code	Strict JSON

Table 2.13 provides a unified requirement-level comparison across all reviewed approaches using the evaluation scales from Section 2.1. This table consolidates the per-subsection evaluations into a single view, making it possible to identify the specific requirement gaps that Code Guardian addresses.

Table 2.13.: Unified requirement comparison of all reviewed approaches and Code Guardian.

Approach	**R1**	**R2**	**R3**	**R4**	**R5**
Semgrep	◑	●	○	●	●
CodeQL	◕	●	○	●	●
GitHub Copilot	◕	◔	◑	●	○
IRIS	◕	◑	◑	○	○
SecRepair	◑	◑	◑	○	○
RESCUE	◕	◕	●	○	—*
Snyk Code	◕	●	○	●	◔
Code Guardian	◕	◕	◑	●	●

Legend: ● High ◕ Medium ◑ Medium–Low ◔ Low ○ Insufficient/None

*Property not evaluated or reported in the original work.

The table shows that no existing approach satisfies all five requirements at medium or above. SAST tools excel at R2, R4, and R5 but lack contextual reasoning (R1) and repair generation (R3). LLM-based research prototypes offer strong R1 and R3 but fail on R4 (no IDE integration) and R5 (cloud dependency). Cloud assistants like Copilot trade privacy for usability. Code Guardian is the only approach that achieves at least medium on all five requirements simultaneously, addressing the combined gap that motivates this thesis.

The positioning claim is not that local LLMs outperform all alternatives across all metrics, but that they enable a *different deployment envelope*: privacy-preserving IDE assistance with explicit control over model choice, retrieval strategy, and output contracts. This makes the approach operationally distinct from cloud assistants and complementary to (not a replacement for) pure SAST pipelines.

Concrete contributions relative to prior work. Relative to existing work, this thesis contributes five mechanisms not jointly applied by any reviewed approach: structured-output robustness as a first-class requirement (strict JSON contract with format-level enforcement and defensive parsing), multi-pass consensus filtering with two different seeds to suppress sampling-variance false positives, deterministic inference controls (temperature 0, fixed seed, JSON-mode decoding) to address

output instability, RAG treated as a model-dependent intervention measured per-model rather than assumed universal, and a unified local evaluation harness that compares against Semgrep / CodeQL / ESLint under the same dataset and scoring logic. Together these address accuracy, consistency, repair quality, usability, and privacy within one framework, bridging academic LLM advances and the practical requirements of real-world development workflows.

2.2.6. Critical Comparison Across Paradigms

The reviewed literature indicates that security assistants should be compared as *design paradigms*, not as isolated tools. Table 2.14 summarizes the trade-offs that motivate this thesis architecture.

Table 2.14.: Critical comparison of major security-assistance paradigms.

Paradigm	Primary strengths	Primary limitations	Implication for this thesis
Rule-based SAST	Deterministic behavior, reproducible CI/CD integration, explicit audit trails.	Limited semantic adaptation and weak repair guidance.	Keep deterministic baseline signals and add contextual reasoning.
LLM-only IDE assistant	Flexible reasoning, explanation, and repair suggestion in context.	Probabilistic behavior, hallucination risk, and variable false-positive control.	Enforce strict output contracts and developer-controlled patching.
RAG-assisted LLM	Better grounding and explanation alignment in favorable setups.	Model-dependent gains and additional latency/-complexity.	Treat RAG as tunable per model/workflow, not universal.
Local-first deployment	Strong source-code privacy boundary and reproducibility.	Dependence on local hardware/host security assumptions.	Prioritize no-exfiltration while documenting residual risks.

Two conclusions follow. First, no single paradigm satisfies R1–R5 alone; practical systems combine complementary mechanisms. Second, evaluation must jointly report quality and operational metrics because deployment value depends on recall, false-positive control, repair quality, latency, and privacy guarantees together.

2.3. Summary

The chapter established R1–R5 and compared the main design paradigms in the literature. A practical solution must combine deterministic baselines, contextual

reasoning, explicit privacy boundaries, and workflow-aware responsiveness — the motivation for the concept in the next chapter.

3. Concept

This chapter presents the conceptual design of Code Guardian, a VS Code extension for local vulnerability detection and repair in JavaScript/TypeScript projects. The design derives from two observations in the preceding analysis: first, that no existing tool simultaneously offers contextual reasoning, repair guidance, and source-code privacy; and second, that the requirements from Chapter 2 impose concrete constraints on how such a tool must be built — particularly around output stability (R2), response latency (R4), and the guarantee that code never leaves the machine (R5).

Section 3.1 traces how each architectural decision maps to analysis results and requirement gaps. Section 3.2 describes the core components. Section 3.3 presents the detection workflows. Section 3.4 provides C4 architecture views and sequence diagrams that make the privacy boundary explicit.

3.1. Concept Derivation from Analysis Results

The previous chapter identified five requirements that no existing tool satisfies simultaneously. This section walks through the main architectural decisions of Code Guardian and explains why each one was chosen, starting from the privacy boundary and moving inward through retrieval, detection strategy, and IDE integration.

3.1.1. From Cloud-Based to Privacy-Preserving Local Deployment

Local deployment is the starting design constraint, not an optimisation. Cloud LLM assistants transmit source code to external services, which conflicts with GDPR [10], the OWASP LLM Top 10 sensitive-information-disclosure risk [12], and documented training-data leakage from code-generation models [9, 54]. Membership-inference attacks can also reveal whether specific fragments were in a model's training set [50]. Running the LLM, retrieval pipeline, and analysis components entirely on local hardware keeps source code, intermediate representations, and analysis results on-device, supports R5, and enables fully offline operation when knowledge refresh is disabled. The privacy boundary is defined around *source code*: analysed code and

IDE-derived context are sent only to a local LLM backend; the optional knowledge-base refresh fetches public CVE/CWE/OWASP metadata only and can be disabled. The trade-off is increased latency relative to cloud accelerators, addressed by careful model selection, retrieval design, and prompt scoping (R4).

3.1.2. Retrieval-Augmented Generation for Security Knowledge Grounding

A standalone LLM can hallucinate vulnerability types, misclassify severity, and produce inconsistent results across repeated analyses [13]. Retrieval-augmented generation grounds reasoning in external knowledge that evolves independently of model parameters [39, 40]. The local knowledge base aggregates CWE-aligned vulnerability patterns and mitigations [2], OWASP guidance [1, 55], public CVE/NVD summaries [56, 57], and curated JS/TS notes (prototype pollution, unsafe deserialisation, etc.). During analysis, semantically-similar entries are retrieved and provided as context. Retrieval is not universally beneficial: the evaluation (Chapter 5) shows it improves quality on some model families and degrades it on others, so it is treated as a configurable option rather than a default.

3.1.3. IDE Integration, Modular Components, and Two-Stage Pipeline

Traditional SAST tools sit downstream in CI/CD; their delayed feedback loop increases cognitive load and reduces remediation rates [7, 8]. Code Guardian integrates directly into Visual Studio Code, providing inline debounced detection during active development and on-demand audit commands for selections, files, and workspaces. IDE integration addresses R4 (interactive presentation of findings, explanations, and quick-fix repairs) and R3 (preview-and-apply remediation with minimal friction).

Internally the system decomposes into four components — context extraction (scope selection plus localisation metadata), knowledge retrieval (vector search over the local corpus), vulnerability detection (LLM analysis grounded in retrieved context), and repair generation (developer-controlled patches as quick fixes). Each is independently testable and replaceable; intermediate outputs remain inspectable for debugging and reproducibility. Detection and repair are deliberately split into two stages: Stage 1 asks only "identify the vulnerabilities and return structured findings" (type, severity, location, description); Stage 2 takes each confirmed finding and runs a dedicated repair prompt with the full snippet. Coupling both tasks into a single prompt biases the model toward generating a plausible fix rather than accurately classifying the issue; separating them improves R1 detection precision and R3 repair quality, and lets repairs run in parallel.

3.1.4. Context-Aware Analysis and Structured Explainability

R1 demands semantic and structural reasoning rather than syntactic pattern matching. The system addresses this through reliable scope selection (the enclosing function via TypeScript AST) plus prompt grounding (retrieved security guidance when RAG is enabled), and expects the model to reason about data flow, sources, sinks, and guards *within the provided scope*. Evidence outside the snippet (validation in another file, framework-level middleware) is the principal limitation and is named as future work in Chapter 7. R3 and R4 additionally require explainable output: the diagnostics pipeline emits structured reports with code localisation, a short risk-explanation message, and an optional quick-fix repair; interactive WebView modes can render longer Markdown explanations and incorporate retrieved CWE/OWASP knowledge to ground the reasoning [2, 55]. The harness used in Chapter 5 additionally requests `type` and `severity` fields to support quantitative scoring.

3.1.5. Deterministic Inference and Consensus Filtering

Output instability — the same code yielding different findings across runs — undermines developer trust and complicates evaluation (R2). Two mechanisms address this. *Deterministic inference*: the harness fixes `temperature=0` and a seed, and enables JSON-mode decoding (`format='json'`) at the Ollama API level so the sampler can only emit valid JSON tokens; this eliminates parse failures from malformed output and makes the response machine-readable without post-hoc repair. *Multi-pass consensus*: the detection prompt is run twice with different seeds (42 and 137); only findings confirmed by both passes are retained, filtering out sampling-variance noise without sacrificing recall on consistently-detected weaknesses. If consensus removes everything, the system falls back to the first pass to avoid silent suppression.

The next section describes the components that realise this design.

3.2. System Components

The Code Guardian prototype is organised into a small set of components that map directly to the requirements from Chapter 2. The system is implemented as a VS Code extension that (i) extracts an appropriate analysis scope from the editor, (ii) invokes a local LLM for vulnerability analysis, (iii) optionally augments prompts with locally retrieved security knowledge (RAG), and (iv) renders findings and fix suggestions through IDE-native UI elements.

3.2.1. Context Extraction

Context extraction determines which code is analysed for a given interaction mode and provides the analyser with enough metadata to localise findings in the editor. Code Guardian supports four scopes with different latency/completeness trade-offs: *function scope* (real-time, the innermost `FunctionLikeDeclaration` at the cursor) is the default for continuous feedback; *file scope* (on-demand) analyses the full document and emits diagnostics; *selection scope* (interactive) sends a region into a WebView analysis view that supports follow-up Q&A; *workspace scope* (batch) scans all JS/TS files and aggregates results into a security dashboard.

Function-scope extraction uses the TypeScript compiler API to find the smallest enclosing function-like block (function declarations, arrow functions, methods, constructors) around the cursor and returns both the snippet and its 0-based start-line offset, which lets model-reported line numbers map back to VS Code diagnostic ranges. Function scoping keeps prompt size predictable for real-time use (R4); deeper program context (imports, cross-file flows) is not extracted explicitly and is named as future work.

3.2.2. Knowledge Retrieval (RAG)

The retrieval component implements RAG [39, 40]: a dedicated `RAGManager` maintains a local knowledge base populated from CWE-aligned weakness patterns and mitigations [2], OWASP Top 10 guidance [1], CVE/NVD summaries from the NVD API [57, 56], and JS-ecosystem advisories. Knowledge artefacts are cached on disk and reused across sessions; if network access is unavailable the system falls back to a small baseline bundle so retrieval remains functional with reduced coverage. Entries are chunked recursively (chunk size 1000, overlap 200), embedded locally through an Ollama embedding model, and stored in an HNSW vector index [48] via LangChain [58]. At query time the top-k chunks are retrieved (runtime default $k = 3$; the evaluation harness in Chapter 5 uses $k = 5$) and injected into the prompt as a "relevant security knowledge" section. Retrieval and embeddings execute locally, and the optional knowledge refresh fetches only public metadata — user source code is never transmitted (R5).

3.2.3. Vulnerability Detection

For inline diagnostics, the detection component enforces structured output through two mechanisms: a strict JSON-only output contract in the system prompt and JSON-mode decoding at the Ollama API level (`format: 'json'`) which constrains token sampling to valid JSON. Deterministic inference (`temperature=0`, fixed seed) further improves consistency (R2). Each finding carries a short `message` and 1-based

`startLine/endLine` indices relative to the analysed snippet. Repair suggestions are not requested during detection — they are generated separately in Stage 2 (Section 3.2.4) and attached to confirmed findings before rendering. The analyser performs defensive parsing by stripping code fences and extracting the first JSON array substring if the model emits surrounding text. To reduce false positives (R1) the detection prompt runs twice with different seeds; only findings confirmed by both passes (matched by type and line range) are retained, filtering sampling-variance noise without additional model capacity. Transient failures are handled through retry with exponential backoff, and an LRU cache keyed by a hash of (code, model, prompt mode) avoids re-inference on unchanged code. A WebView-based analysis view supports the interactive selection mode with Markdown-formatted responses and follow-up Q&A, optionally enriched by the RAG manager.

3.2.4. Repair Generation and IDE Integration

Repair generation follows a **two-stage pipeline**. Stage 1 identifies vulnerabilities without fixes; Stage 2 takes each confirmed vulnerability after consensus filtering and runs a dedicated repair prompt with the vulnerability type, affected lines, and full code context. Separating detection and repair lets each prompt focus on a single task. The repair prompt returns a JSON object with the suggested fix; the IDE integration layer exposes each repair as a Quick Fix action. Application is always user-initiated and reversible via the editor undo stack (R3). Functional correctness of repairs is not automatically validated and is named as future work.

The IDE integration layer wires the analysis pipeline into the VS Code APIs [59]. Triggers include a debounced real-time analyser on document changes (800 ms default), a manual file-analysis command with a size guard, interactive selection / contextual Q&A WebViews, and a workspace scanner that aggregates results into a dashboard. Findings render as VS Code diagnostics (squiggles, Problems panel, hover tooltips) and repairs surface as Quick Fix actions.

3.2.5. Supporting Subsystems

Model management queries available local Ollama models, filters suitable code models, and supports runtime model switching. The analysis cache is a bounded LRU (100 entries, 30-minute TTL) with user-visible hit/miss statistics. The workspace dashboard computes a coarse security score using issue density and keyword-based severity heuristics. The vulnerability-data manager caches public CVE/CWE/OWASP metadata with a time-based expiry to support offline reuse after refresh. The next section explains how these components are orchestrated into workflows for real-time feedback, on-demand inspection, and batch scans.

3.3. Detection Workflows

While the component architecture defines what responsibilities exist in the system, IDE usability depends on how these components are orchestrated in concrete workflows. Code Guardian supports several workflows that trade off latency, context breadth, and output structure. This section describes the main workflows implemented in the prototype and relates them to requirements R1–R5.

3.3.1. Real-Time Function-Level Diagnostics

The real-time workflow provides continuous feedback while a developer edits JavaScript/TypeScript code. The key goal is to deliver timely, IDE-native warnings without disrupting flow (R4).

Trigger. The extension listens to document change events for JavaScript and TypeScript documents. Analysis is *debounced* (default: 800 ms, following common VS Code extension practice for balancing responsiveness against unnecessary re-analysis) so the model is not invoked on every keystroke.

Scope and guards. When the debounce fires, Code Guardian extracts the innermost enclosing function at the cursor position and analyzes only that snippet. A size guard skips unusually large functions (default: 2 000 characters, chosen empirically to keep prompt size within the range where local models produce reliable structured output) to bound worst-case latency and avoid overloading the local model.

Structured output for diagnostics. The analyzer is prompted to return a strict JSON array of issues. Each issue contains a message and a line range. Repair suggestions are generated separately in Stage 2 and attached before rendering. The diagnostic adapter maps the snippet-relative, 1-based line numbers to VS Code ranges using the extractor's line offset and clamps ranges to valid document bounds. This enables stable rendering in the Problems panel, editor squiggles, and hover tooltips.

Trade-offs. Function-level analysis improves responsiveness, but it can miss vulnerabilities whose evidence lies outside the current scope (e.g., validation in a different module). This limitation motivates the on-demand and workspace workflows and is addressed further in the future-work chapter.

3.3.2. On-Demand File Diagnostics

The file workflow provides broader coverage when a developer explicitly requests a deeper scan.

Trigger. A command palette action runs analysis over the full active document.

Scope guard. To avoid generating excessively large prompts, the prototype skips very large files (default: 20,000 characters) and warns the user instead.

Output. Findings are surfaced through the same structured diagnostics pipeline as the real-time workflow. This keeps the UI consistent, and it ensures that file scans can be reviewed in the Problems panel and navigated using standard IDE tooling.

3.3.3. Interactive Analysis and Follow-up Q&A

Some security questions require richer explanations than a single diagnostic message. Code Guardian therefore includes webview-based interaction modes.

Selection analysis view. The user can analyze a selected region (or the current line) and inspect the response in a dedicated analysis panel. This mode supports follow-up questions and streams Markdown-formatted responses for readability. Because it is user-initiated and not continuously triggered, it can tolerate higher latency than real-time diagnostics.

Contextual Q&A view. The contextual Q&A panel allows the user to select files and folders as context and ask security questions about that subset of the workspace. The extension reads the selected files locally and sends their contents only to the local LLM backend, preserving the no-exfiltration objective for source code.

RAG usage. When enabled, the RAG manager can enrich prompts for these interactive workflows by injecting retrieved security knowledge snippets (CWE/OWASP/CVE-derived guidance). Retrieval and embeddings are performed locally; optional knowledge refresh operations fetch only public metadata.

3.3.4. Workspace Scan and Security Dashboard

The workspace workflow supports periodic audits and prioritization across a repository.

File discovery and bounds. The scanner enumerates `*.js`, `*.jsx`, `*.ts`, and `*.tsx` files in the workspace while excluding dependency folders such as `node_modules`. Very large files are skipped (default: >500 KB) to keep runtime bounded.

Aggregation and scoring. Scan results are aggregated into a dashboard WebView. In addition to listing per-file issues, the dashboard computes a coarse security score based on issue density (issues per KLOC) and a keyword-based severity heuristic. This score is intended for prioritization rather than as a formal risk metric.

Developer interaction. The dashboard supports opening affected files directly from the report and rerunning scans. This workflow complements real-time diagnostics

by helping developers understand which parts of the codebase concentrate the most findings.

3.3.5. Repair Suggestion Workflow

After consensus-filtered detection (Stage 1), the system generates targeted repairs for each confirmed vulnerability using a dedicated repair prompt (Stage 2). Repairs are exposed as VS Code quick fixes. Applying repairs is always user-initiated and integrates with the undo stack. This preserves developer control (R3) and reduces the risk of unintended behavioral changes from automatically applied patches.

3.3.6. Confidence Abstention and Hybrid Gating

Two design decisions complement consensus filtering to address the false-positive problem identified in the analysis chapter (R1) without re-introducing the deterministic-rule rigidity that motivated using LLMs in the first place.

Confidence as a first-class signal. Multi-pass consensus is itself an empirical signal: if two seeded passes report the same finding, the model agrees with itself; if only one pass reports it, the finding is noisier. The refined design promotes that signal from a binary keep-or-drop decision to an explicit per-finding confidence value, computable as the fraction of passes that produced a matching finding. A configurable threshold then decides which findings reach the diagnostic stream. The default behaviour is unchanged (no gating), but operators who care more about precision than recall can lift the threshold to suppress single-pass noise. This makes the precision/recall trade-off a configuration choice rather than a design constant.

Hybrid SAST agreement as a confirmation signal. A second source of corroboration is the established SAST tooling. When a deterministic baseline (Semgrep) flags a finding on overlapping lines and a matching canonical category, the LLM finding is promoted to maximum confidence and labelled as hybrid-source. The hybrid label is reported back to the IDE so the developer can see which findings have independent corroboration. This re-uses Semgrep's strengths (low false-positive rate on known patterns) without adopting its weaknesses (very narrow coverage), and it does so transparently — the LLM's own findings remain visible at their original confidence even when Semgrep is silent.

Both mechanisms are conservative: they extend the existing pipeline with optional gates rather than replacing the un-gated path. The chapter's evaluation reports both with-gate and without-gate metrics so the contribution of each gate is attributable.

3.3.7. Workflow Selection in Practice

In typical use, workflows complement each other:

1. Developers receive continuous feedback via debounced, function-level diagnostics while writing code.
2. Before committing or during review, developers run file scans for broader coverage.
3. For ambiguous findings or design-level questions, developers use the interactive analysis and contextual Q&A views to obtain richer explanations and mitigation guidance.
4. Periodically, workspace scans provide an aggregate view that supports prioritization and remediation planning.

Together, these workflows operationalize the core idea of Code Guardian: privacy-preserving local analysis combined with IDE-native feedback and developer-controlled remediation.

3.4. System Architecture and Design

This section presents the high-level architecture of Code Guardian using C4-style views (context, containers, and process). The goal is to make the privacy boundary and the main data flows explicit: code stays on-device, local inference is performed through a localhost LLM backend, and retrieval (when enabled) is backed by a local knowledge base and vector index.

3.4.1. Context View: System Boundary and External Interactions

Figure 3.1 positions Code Guardian within its operational environment. The primary actor is the developer working inside Visual Studio Code. The system executes in the VS Code extension host and communicates with a local LLM backend (Ollama) running on the same machine [11].

Local assets. The core local assets are:

- **Workspace source code** (JavaScript/TypeScript) being edited.
- **Local LLM runtime** (Ollama) used for analysis and (optionally) embeddings.
- **Local security knowledge base** and **vector index** used for retrieval when RAG is enabled [58, 48].
- **Local caches** for analysis results and vulnerability metadata.

Optional external data. Code Guardian may optionally fetch *public vulnerability metadata* to refresh the knowledge base (e.g., CVE records from the NVD API). This traffic does not include user source code and can be disabled by running in offline mode. After a refresh, cached knowledge can be reused without network access. This separation preserves the core privacy goal (no source-code exfiltration, R5) while still allowing the knowledge base to evolve over time.

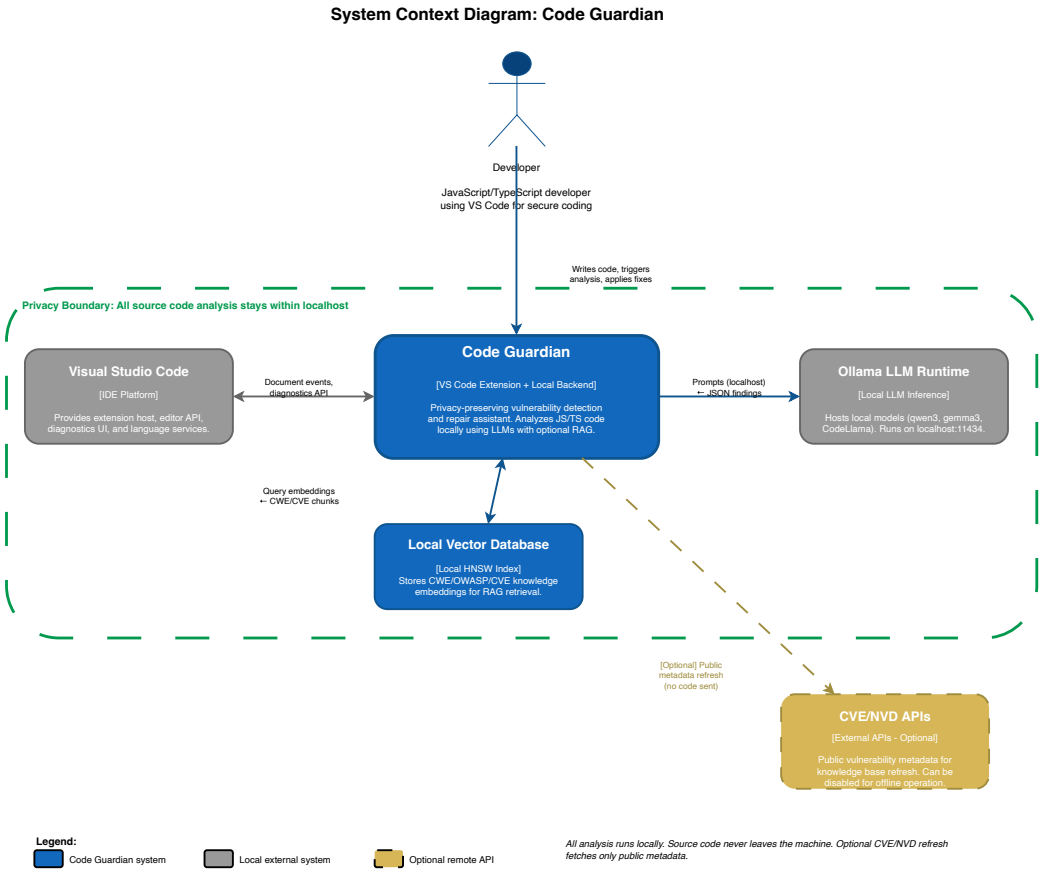

Figure 3.1.: C4 Context diagram showing Code Guardian within its operational environment. The system communicates with VS Code (IDE platform), Ollama (local LLM on localhost:11434), and a local vector database, all within the privacy boundary (green dashed). CVE/NVD APIs (gold, dashed) are the only optional external connection and do not receive source code.

3.4.2. Container View: Internal Component Structure

Figure 3.2 summarizes the main internal containers and their responsibilities.

VS Code extension host. The extension is responsible for registering editor triggers, extracting analysis scopes, and rendering results using VS Code diagnostics and WebViews [59]. It provides commands for file analysis, selection analysis, contextual Q&A, model selection, cache inspection, and workspace scanning.

Structured diagnostics pipeline. For real-time and file-level diagnostics, the extension invokes a JSON-only analyzer that returns a list of issues with line ranges. Repair suggestions are generated in a separate stage and attached before rendering. Results are mapped into VS Code diagnostics and quick fixes. A bounded analysis cache reduces redundant LLM calls for unchanged snippets.

Interactive analysis pipeline. For selection analysis and contextual Q&A, the extension opens a WebView panel and streams Markdown-formatted responses from the local model. This mode supports conversational follow-up and can incorporate retrieved knowledge when RAG is enabled.

RAG manager and data manager. When enabled, the RAG manager maintains a local knowledge base (serialized entries) and a persistent vector store. A vulnerability data manager refreshes public metadata (CWE-/OWASP-aligned curated entries and optional CVE/advisory sources) and caches results on disk. The vector store is rebuilt or updated based on the knowledge base content.

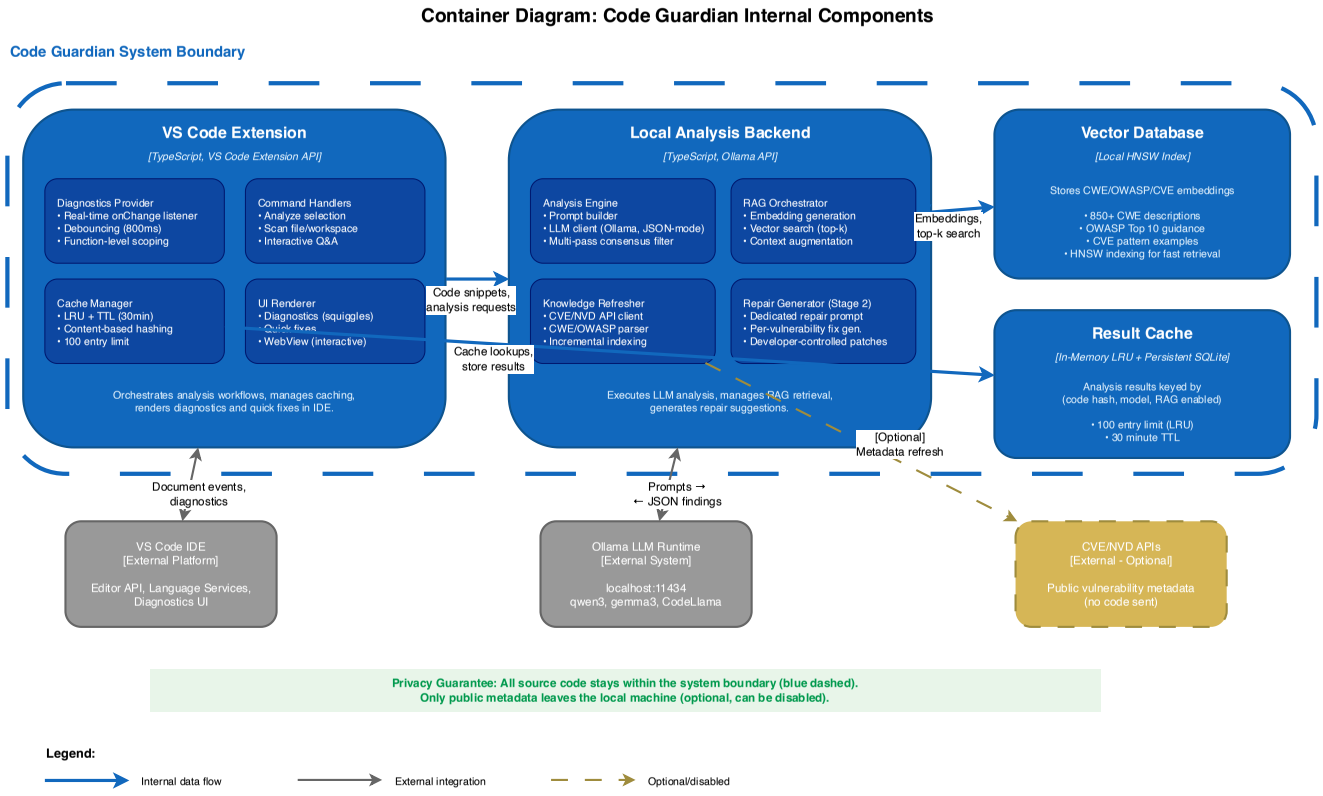

Figure 3.2.: C4 Container diagram showing internal components. The VS Code Extension handles triggers, caching, and UI rendering. The Local Analysis Backend performs LLM inference with JSON-mode and multi-pass consensus filtering, RAG orchestration, and Stage 2 repair generation. Vector Database and Result Cache are local data stores. All source code stays within the system boundary.

3.4.3. Process View: End-to-End Workflows

Figure 3.3 illustrates the dynamic execution flow for the four main workflows. *Real-time diagnostics* debounce document-change events for 800 ms, extract the innermost enclosing function at the cursor, invoke the analyser within size limits, parse and cache the JSON findings, and surface repair suggestions as quick fixes. *On-demand file diagnostics* run the analyser over the full document subject to a size guard. *Interactive analysis and contextual Q&A* open a WebView, stream Markdown responses from the local model, and optionally inject retrieved knowledge to ground explanations. *Workspace scan* enumerates JS/TS files (excluding dependencies, skipping very large files), analyses each locally, aggregates by severity heuristic, and renders the results in a dashboard. Across all workflows source code is sent only to localhost; the optional metadata refresh fetches public CVE/CWE/OWASP records without source content and can be disabled for fully offline operation.

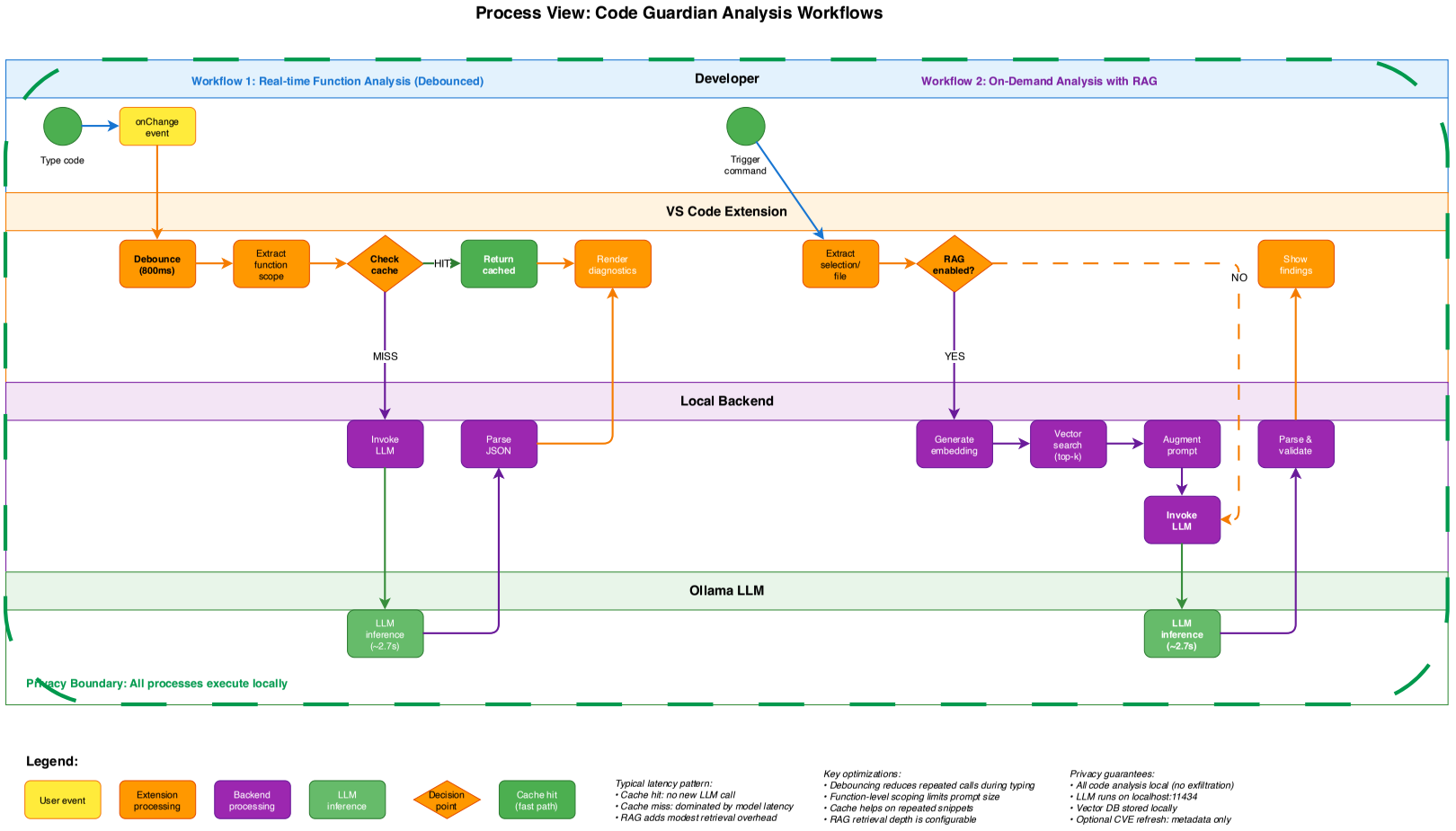

Figure 3.3.: Process view showing two primary workflows with swim lanes. Workflow 1: Real-time function analysis with debouncing (800 ms) and caching, where cache hits bypass LLM inference and cache misses invoke local analysis. Workflow 2: On-demand analysis with optional RAG; when enabled, the backend generates embeddings, performs vector search (runtime default k=3; evaluation uses k=5), and augments the prompt before LLM invocation. When RAG is disabled, analysis proceeds directly to LLM inference without retrieval. Color coding: user events (yellow), extension (orange), backend (purple), LLM (green). Privacy boundary (green dashed): all processes execute on localhost.

3.4.4. Sequence Diagrams: Concrete Detection Flows

Three sequence diagrams capture the dominant interaction patterns. Inline mode with a cache hit (Figure 3.4) returns previously-stored findings without invoking the model, which is the cheap path that absorbs most repeated keystrokes. Audit mode with RAG (Figure 3.5) embeds the snippet, performs vector search (top-$k = 3$ at runtime, $k = 5$ in the evaluated setting), injects retrieved CWE/OWASP chunks, runs two detection passes with seeds 42 and 137 plus consensus filtering, and finally generates repairs in a separate Stage-2 pass. Real-time detection (Figure 3.6) layers an 800 ms debounce timer over the inline path so continuous typing collapses into a single analysis.

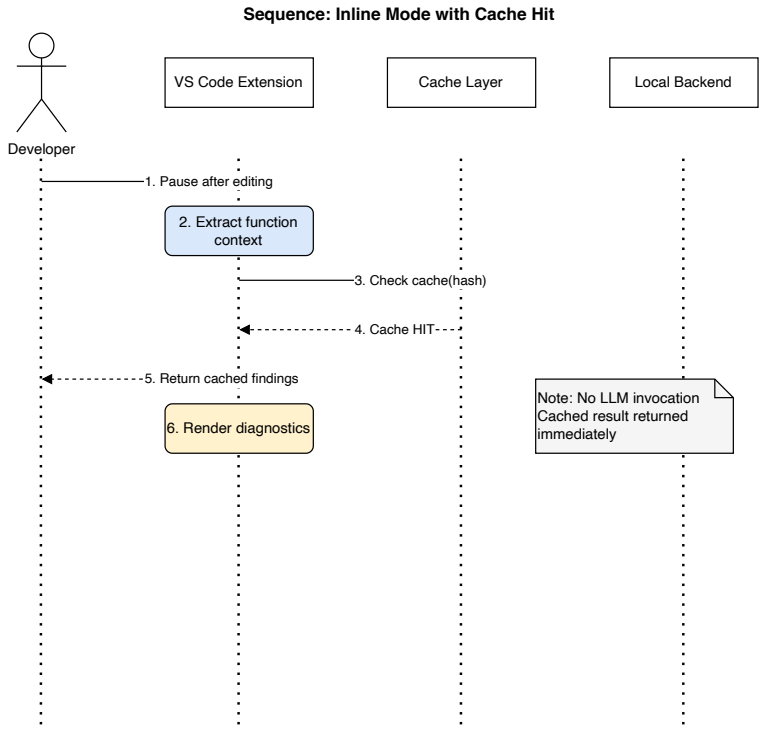

Figure 3.4.: Inline mode with cache hit: no LLM invocation when a previously analysed function reappears.

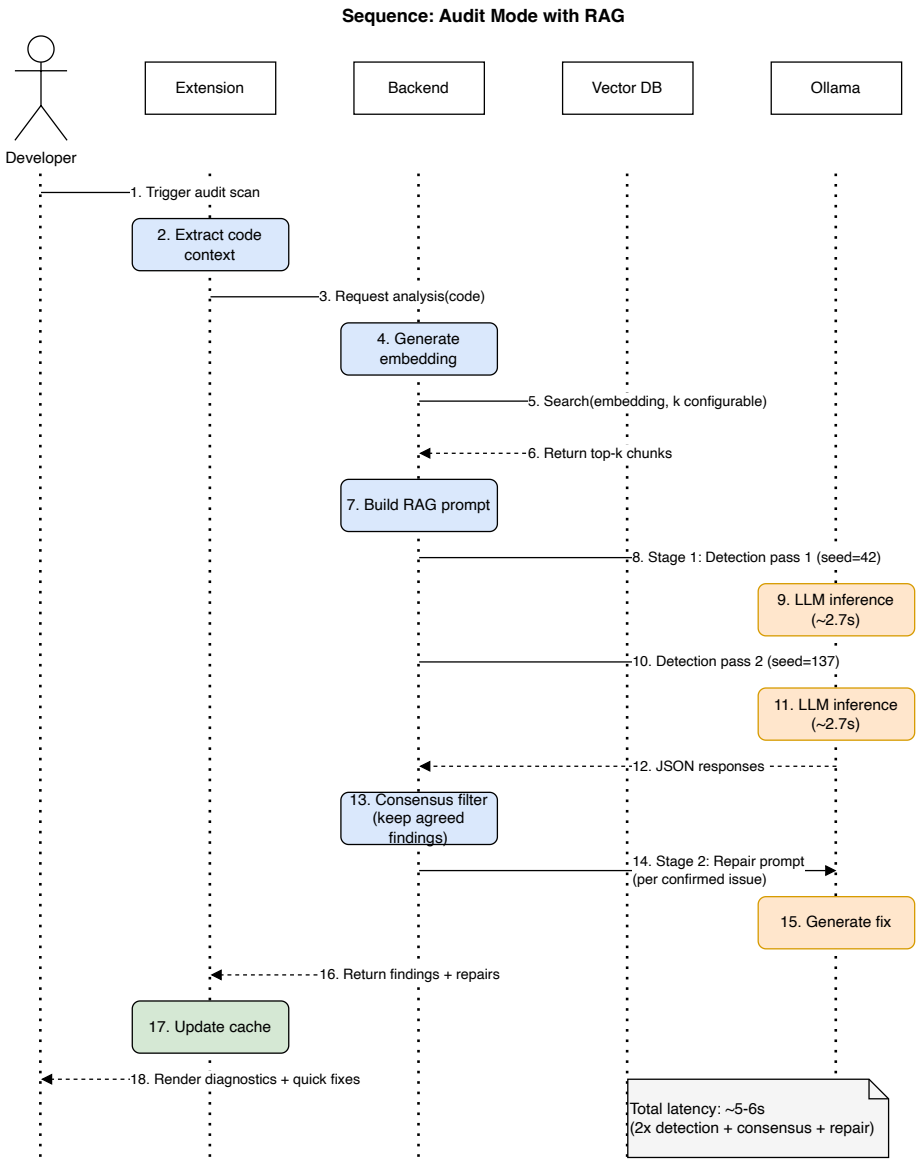

Figure 3.5.: Audit mode with RAG: vector search, two-pass consensus detection, then Stage-2 repair generation.

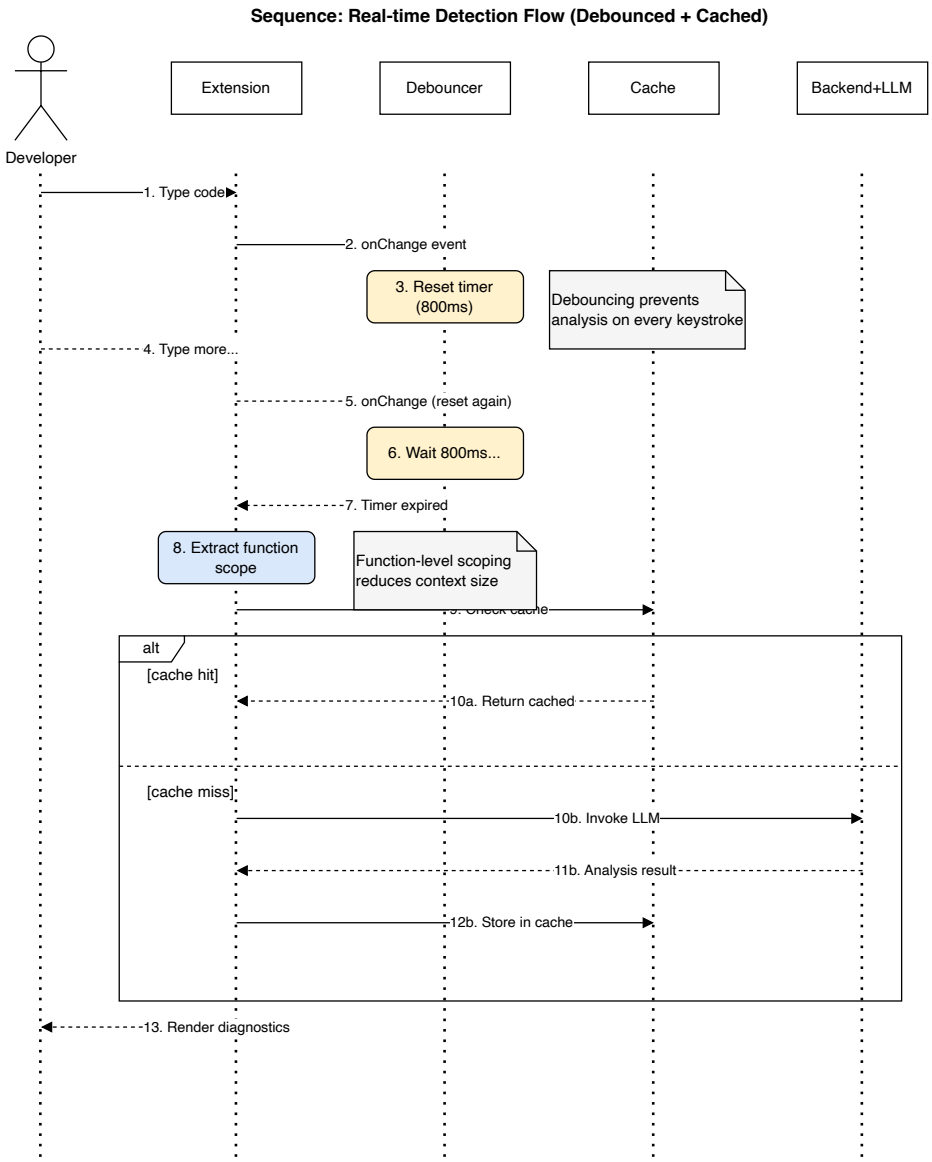

Figure 3.6.: Real-time detection with debouncing: an 800 ms timer prevents redundant LLM invocations during continuous typing.

3.5. Summary

The concept chapter derived Code Guardian's architecture from the requirements and gaps identified in Chapter 2. The central design choices — local-only inference, consensus-based false-positive filtering, strict JSON output contracts, and a two-stage detection-then-repair pipeline — each address specific weaknesses in existing tools while respecting the privacy and latency constraints of an IDE-integrated workflow. The C4 views and sequence diagrams make one boundary explicit above all others: source code stays on the developer's machine. The next chapter describes how this design was implemented.

4. Implementation

This chapter describes how Code Guardian was implemented as a VS Code extension, translating the conceptual design from Chapter 3 into a deployable system. The focus is on practical engineering decisions that affect reproducibility, latency, and privacy preservation on developer hardware: local inference boundaries, deterministic output handling, latency guardrails, and developer-controlled remediation.

Section 4.1 introduces the technology stack and runtime boundary. Section 4.2 explains the end-to-end detection pipeline. Section 4.3 describes the core components. Section 4.4 presents the user-facing integration. Section 4.5 discusses repair safety.

4.1. Technology Stack and Rationale

Code Guardian is implemented as a Visual Studio Code extension that performs security analysis and repair suggestion *locally* on the developer machine. The design goal is to integrate vulnerability detection into the IDE workflow while maintaining a strict *no source-code exfiltration* property (R5) and practical responsiveness for interactive use (R4).

VS Code Extension (TypeScript)

The extension is implemented in **TypeScript** using the VS Code Extension API [59]. Core responsibilities include registering editor events (on-change and on-command triggers), mapping analysis findings to VS Code diagnostics, and exposing quick fixes through code actions. The extension also provides two WebView-based interfaces: an interactive analysis view for selected code and a workspace security dashboard that aggregates findings across the project. Where IDE integration benefits from standardized editor tooling, the Language Server Protocol provides a relevant reference point for diagnostics-style interactions in modern IDEs [60].

Local LLM Inference via Ollama

All model inference is performed through **Ollama** running on `localhost:11434` [11]. Code Guardian supports multiple local model families (e.g., `qwen3`, `gemma3`, `CodeLlama`) and allows switching models at runtime through settings. Requests are configured for low-temperature decoding to reduce randomness and improve schema adherence, and retry logic with exponential backoff is applied to handle transient failures (e.g., model warm-up, timeouts).

Retrieval-Augmented Generation (RAG)

Optional RAG is implemented with **LangChain** and a local vector index [58]. Security knowledge entries (CWE patterns, OWASP Top 10 guidance, selected CVE summaries, and JavaScript/TypeScript secure coding notes) are embedded using a lightweight local embedding model (`nomic-embed-text` via Ollama) and stored in an **HNSW** vector store. At analysis time, relevant knowledge snippets are retrieved and injected into the prompt to improve grounding of the model's reasoning and support R1–R4.

The `nomic-embed-text` model was selected for three reasons: (1) native availability via Ollama, ensuring the same local deployment model as the LLM runtime; (2) strong performance on code and technical text retrieval tasks relative to its size (768-dimensional embeddings at 137M parameters); and (3) permissive licensing for local deployment. Alternative embedding models such as `all-MiniLM-L6-v2` or `bge-small-en` were considered but required separate runtime infrastructure or offered inferior code-domain performance in preliminary testing.

Dynamic Vulnerability Knowledge Updates

The knowledge base is designed to be refreshable. Code Guardian supports updating OWASP and CVE information and persists cached data on disk. Importantly, only public vulnerability metadata is fetched; source code and extracted context remain local.

Engineering Selection Criteria

Technology choices were guided by four practical criteria:

- **Local deployability:** all core components must run on a developer workstation without managed cloud dependencies.

- **Operational transparency:** outputs and failure modes must be inspectable for debugging and thesis reproducibility.
- **Incremental integration:** the solution should align with VS Code-native diagnostics and command workflows instead of requiring a separate toolchain.
- **Replaceability:** model backends and retrieval components should be swappable without redesigning the full extension.

These criteria explain the concrete stack decisions: Ollama for local model serving, TypeScript for extension maintainability and VS Code API fit, and local vector retrieval for optional grounding. Together they form a modular but operationally coherent baseline for privacy-preserving IDE security assistance.

Shared TS / JS Modules

The system uses four small modules that are kept in sync between the extension runtime (TypeScript) and the evaluation harness (Node.js) by sharing a single source-of-truth JSON catalogue. Each module is covered by a deterministic snapshot test that asserts twin agreement on a fixed fixture set.

- **Canonical category taxonomy** (`src/categoryTaxonomy.json`, `src/categoryTaxonomy.t` `evaluation/category-taxonomy.js`). A versioned list of 23 canonical vulnerability classes plus an ordered set of regex patterns mapping raw model-emitted labels to canonical IDs. Used by both the extension and the harness for consistent type-level scoring.
- **SAST fusion logic** (`src/sastBridge.ts`, `evaluation/sast-fusion.js`). Pure function `fuseSastWithLlm` that promotes LLM findings agreed by Semgrep on line range and canonical category to confidence 1.0 and source `hybrid`. Spawning Semgrep from the extension is deferred; the harness already drives Semgrep for the SAST baseline so the fusion path reuses the same workspace.
- **Import / sink resolver** (`src/importResolver.ts`, `evaluation/import-resolver.js`). Lightweight regex-based scanner that surfaces imports, validator-package presence (`sanitize-html`, `joi`, `zod`, etc.), and category-tagged sink occurrences (e.g., `eval`, `child_process.exec`, `crypto.createHash('md5')`, `jsonwebtoken.verify`). Output is appended to the user prompt behind an opt-in flag.
- **Repair syntax validator** (`src/repairValidator.ts`, `evaluation/repair-validator.js` Uses `@babel/parser` with TypeScript, JSX, and async plugins to check whether a generated repair string is syntactically applicable JavaScript or TypeScript. Markdown fences are stripped before parsing; obvious prose openings are rejected before the parser is invoked.

These additions keep the privacy boundary intact: every module operates on data that is already on the developer's machine. The validator and resolver depend only on `@babel/parser`, which itself runs locally and ships with the extension bundle.

4.2. System Workflow

This section explains how Code Guardian turns JavaScript/TypeScript code into vulnerability findings and repair suggestions, implementing the design from Chapter 3 against the requirements in Chapter 2 (R3, R4, R5).

End-to-End Flow

Two execution paths matter in the prototype: a *structured diagnostics* path that emits JSON findings for editor diagnostics, and an *interactive analysis* path that emits Markdown explanations in a WebView with follow-up Q&A. A run is triggered automatically while editing (debounced) or explicitly via a command (analyse selection, analyse file, scan workspace), and the chosen scope determines how much context is included. The extension extracts the relevant code fragment and its location: for real-time diagnostics, the default is the enclosing function found by depth-first AST traversal via the TypeScript language service, with a 2 000-character size guard; for selection analysis, the unit is the selected region; for file analysis, the entire document. If RAG is enabled and initialised, the system retrieves security-knowledge snippets through a local embedding model and HNSW index and appends the guidance to the prompt; the real-time diagnostics path is kept lightweight to preserve responsiveness, while RAG is primarily used in interactive WebView and batch-scan workflows. The local Ollama model is invoked with output constraints matching the workflow: structured diagnostics request a JSON array of issues with defensive parsing (strip Markdown fences, extract the first JSON array substring, validate fields) and degrade safely to empty findings on parse failure; interactive analysis streams Markdown to the WebView. Findings render as VS Code diagnostics (squiggles, Problems panel, hover tooltips) with optional Quick Fix actions; results are cached by (snippet, model, mode) under LRU + TTL so repeated edits do not re-invoke the model.

Although inference is probabilistic, the orchestration is deterministic: for a fixed input snapshot, active model, and prompt mode, scope extraction, prompt-assembly structure, parsing, localisation mapping, and diagnostic emission follow a fixed sequence with explicit guards. This isolates model effects during debugging and evaluation. On each trigger the extension extracts and validates scope, computes a content-based cache key, returns immediately on hit, and on miss assembles the prompt (optionally with RAG), calls the model, validates the JSON response, maps snippet-relative line numbers to document positions, caches the result, and renders.

Debouncing and Workflow Modes

A debouncer suppresses analysis on every keystroke: each document-change event resets a timer, and analysis runs only after an 800 ms pause. The real-time workflow uses function-level scope plus the 2 000-character guard (∼400–500 tokens) to keep inline feedback within acceptable bounds on consumer hardware. On-demand selection or file analysis applies a 20 000-character file guard (∼4 000–5 000 tokens) so prompts fit the effective context window of smaller local models (8K–16K) with headroom for RAG context and response generation. The workspace scanner iterates JS/TS files (excluding `node_modules`) and aggregates findings into a dashboard; files above 500 KB are skipped so memory use stays bounded across large monorepos.

Pipeline Gates

Four configurable gates sit between the consensus filter and diagnostic emission, each toggleable through extension settings. *Confidence assignment* stamps every stage-1 finding with a confidence score and a `detectionSource` provenance tag — both-pass agreement is 1.0, and the single-pass fallback (which would otherwise drop findings silently) keeps them at 0.3 so they remain visible to downstream gates. The opt-in *confidence gate* applies a threshold in $[0, 1]$ before stage 2 (default 0, no gating); at ≥ 0.67 on three passes only findings agreed by at least two survive. The opt-in *hybrid SAST fusion* runs Semgrep once over the snippet, and any LLM finding overlapping a Semgrep finding on line range plus canonical category is promoted to `detectionSource: 'hybrid'` with confidence 1.0; LLM-only findings keep their inter-run confidence. *Repair syntax validation* parses every `suggestedFix` after stage 2 and tags findings with an `autoApplicable` boolean (and a short rejection reason such as `prose_not_code`); the IDE quick-fix surface uses this to decide whether to offer one-click apply or a preview-first action.

System Prompt and Latency Telemetry

The stage-1 system prompt is a tight ∼80-token instruction; JSON-mode decoding plus the response schema enforce output shape, so a verbose response-format example block is unnecessary. Per-call latency is reported as `inferenceTimeMs` plus `parseTimeMs` so any latency change can be attributed unambiguously. Inference dominates the total by more than 99% in measured micro-evaluation; the prompt size is therefore the dominant prefill-token cost lever.

4.3. Core Component Implementation

The subsections below walk through the extension's components in the order data flows through them: context extraction, LLM analysis, diagnostic rendering, retrieval, caching, and workspace-level scanning.

4.3.1. Context Extraction and Scoping

Code Guardian extracts the smallest useful unit of code for interactive analysis: the enclosing function at the cursor. This reduces prompt size and improves responsiveness without requiring full-program analysis. The extractor parses the active document, locates the innermost function-like node covering the cursor, and returns both the snippet and its 0-based start-line offset within the original document; the offset is later used to map model-reported line numbers back to the full file so diagnostics are placed correctly. If parsing fails or no suitable node is found, the implementation falls back to a conservative line/selection scope, favouring availability over strict completeness in real-time mode. For explicit commands the scope expands to a selected region (or current line) or the full file — broader scopes used for deeper inspection or pre-commit review.

4.3.2. LLM Analyser (Local JSON-Only Output)

The analyser performs local inference through Ollama and requires the model to return only a JSON array of findings against the schema below.

Listing 4.1: Security issue output schema used by Code Guardian

```
interface SecurityIssue {
  message: string;
  startLine: number;    // 1-based
  endLine: number;      // 1-based
  suggestedFix?: string;
}
```

Reliability rests on three mechanisms. The system prompt enforces strict JSON-only output. Defensive parsing applies staged normalisation — trim wrapper text, strip Markdown fences, extract the first array-like substring, then parse and validate field types — and discards invalid entries rather than partially accepting them; hard parse failures produce an empty issue list with a categorised parse error so malformed outputs cannot propagate into editor state. Transient errors (timeouts, temporary unavailability) are handled by bounded retries with exponential backoff that absorbs short-lived contention from model warm-up or concurrent local inference; retries are capped to prevent cascading delays in interactive workflows. Non-recoverable failures

(repeated timeouts, model-not-found) surface a user-facing message and return an empty list so the editor stays responsive while preserving explicit control over model configuration.

4.3.3. Diagnostics Mapping

The diagnostic adapter converts the model's 1-based line numbering into VS Code's 0-based ranges and clamps indices to valid document bounds. When a function snippet is analysed, the previously-computed start-line offset is added to each finding's range. If a suggested fix is attached, it is exposed as a Quick Fix action.

4.3.4. RAG Manager and Knowledge Updates

When enabled, the RAG manager maintains a local security knowledge base and a persistent HNSW vector index, embedding entries via a local Ollama model (`nomic-embed-text`). Entries are chunked using a recursive text splitter to balance semantic coherence and retrieval recall, and the index is persisted in an extension-managed directory so it can be reused across sessions without rebuilding; initialisation is lazy to avoid slowing extension activation when retrieval is not used. For each analysis request, the manager retrieves the top-k snippets and augments the prompt with short vulnerability definitions and mitigation guidance. The vulnerability-data manager can refresh public sources on demand (OWASP Top 10, a curated CWE set, a bounded number of recent CVEs); retrieved metadata is cached on disk, only public information is fetched, and no source code or extracted context leaves the device (R5). A minimal baseline knowledge bundle is used when updates fail so RAG-enabled prompting remains functional offline with reduced coverage.

4.3.5. Analysis Cache

To avoid repeated inference on unchanged snippets, results are cached in a bounded LRU with time-to-live expiration. The cache key is a SHA-256 hash of the trimmed snippet plus the active model name and prompt mode (RAG on/off), so LLM-only and RAG results are stored separately. Entries expire after 30 minutes and the cache holds at most 500 entries; least-recently-used eviction handles overflow. In repetitive editing patterns (undo/redo, small refactors) this removes a substantial share of otherwise-repeated LLM calls.

4.3.6. Workspace Scanner and Dashboard

For project-level visibility, the workspace scanner analyses JavaScript and TypeScript files in batch and aggregates results into a dashboard WebView. The scanner enumerates by extension, excludes dependency folders (e.g. `node_modules`), and skips very large files to bound runtime; processing is bounded and sequential to avoid saturating local inference capacity and degrading the interactive experience, optimising for reproducible periodic audits rather than throughput. Because the JSON-only schema does not mandate a severity field, the scanner applies a conservative keyword-based severity heuristic for coarse dashboard prioritisation, treated as a presentation aid rather than a ground-truth classifier.

4.3.7. Privacy Boundary and Threat Model

The privacy boundary is at inference: analysed code and extracted context go only to the local Ollama server, never to external services. Knowledge updates fetch only public vulnerability metadata and cache it locally before any prompt grounding. The threat-model arms named in the task description — code exfiltration, retrieval poisoning, and prompt injection — are addressed by the network policy, signed corpus, and structured-output schema described in Section 4.6.

4.4. User Interface

Code Guardian is designed to integrate security feedback into the existing Visual Studio Code workflow rather than introducing a separate application. The interface emphasizes (i) low-friction detection during coding, (ii) actionable remediation via quick fixes, and (iii) workspace-level prioritization.

4.4.1. Inline Diagnostics and Hover Explanations

Detected issues are rendered as standard VS Code diagnostics. This provides familiar affordances: squiggles in the editor, a centralized list in the Problems panel, and hover tooltips that summarize the finding. This presentation supports R4 by making explanations available at the point of code, without requiring context switching.

Real-time behavior. For JavaScript and TypeScript documents, diagnostics can be produced during normal editing via a debounced trigger. To preserve responsiveness, the default real-time scope is the enclosing function at the cursor, and unusually large functions are skipped. Developers can therefore treat Code Guardian feedback similarly to linting warnings: it appears close to the code being written and is automatically updated as the local scope changes.

4.4.2. Quick Fixes for Repair Suggestions

When a finding includes a suggested fix, Code Guardian offers a *Quick Fix* action. Fix application is always user-initiated and confirmed, which preserves developer control and reduces the risk of unintended behavior changes (R5). Fixes integrate with the editor undo stack so developers can revert and iterate.

Developer control. The prototype does not apply patches automatically. Instead, suggested fixes are attached to diagnostics and surfaced through the standard lightbulb UI. This design reflects the practical risk that an LLM-generated patch may be security-improving but behavior-changing; keeping the developer in the loop is therefore essential for safe adoption.

Interaction contract for fixes. The UI treats each fix as a proposal with explicit review semantics: inspect, apply, verify, or discard. This interaction contract reduces automation surprise and makes remediation decisions auditable in normal editor history. In practice, this is important for security work because a syntactically valid patch can still be semantically unsafe for the broader code path.

4.4.3. Interactive Analysis View and Contextual Q&A

For deeper inspection, the extension provides WebView-based views. A selection analysis view presents the model output in a dedicated panel for longer explanations and follow-up questions (e.g., “why is this vulnerable?”, “what is the safer alternative?”). In addition, Code Guardian provides a contextual Q&A view in which the developer can select files or folders as context and ask security-related questions about a codebase segment. Both views support switching the local model at runtime and (when enabled) benefit from retrieval-augmented grounding.

Model controls inside WebViews. The WebView views include a model selector that lists locally available Ollama models and supports refreshing the list at runtime. This enables quick comparison of smaller models (fast feedback) versus larger models (more detailed explanations) without restarting the extension.

4.4.4. Workspace Security Dashboard

The workspace dashboard aggregates findings across the codebase. It provides a severity breakdown and highlights the most vulnerable files, enabling developers to prioritize remediation work. This mode is complementary to inline diagnostics: it supports periodic security reviews and progress tracking after fixes.

Score and prioritization. To support triage, the dashboard computes a coarse security score based on issue density and severity heuristics, and surfaces the files with the highest concentration of findings. The score is intended as a prioritization aid rather than a formal risk metric; the underlying findings should still be inspected and validated by the developer.

Triage workflow support. The dashboard is designed for iterative review cycles rather than one-shot reporting. Developers can jump from aggregate views to file-level findings, apply fixes, and rescan to observe trend changes. This feedback loop helps convert raw detection output into actionable remediation planning at repository scale.

4.4.5. Model and RAG Controls

Because Code Guardian is intended to run on developer hardware with varying resource constraints, model selection is exposed as a first-class UI feature. The extension provides commands to list available local Ollama models and switch the active model without restarting the extension. RAG can be toggled through settings and is also surfaced as a lightweight status indicator to make the current analysis mode explicit during development sessions. This transparency supports detection consistency (R2) and reduces user confusion about why results differ across runs.

RAG and cache management. The prototype also exposes maintenance commands for (i) updating vulnerability metadata used for the local knowledge base, (ii) rebuilding or searching the knowledge base, and (iii) inspecting and clearing the analysis cache. These controls help developers validate performance improvements (cache hit rates) and keep the retrieval corpus current without sacrificing source-code privacy.

4.4.6. Human Factors and Safe Adoption

Beyond visual presentation, safe adoption depends on how the interface shapes trust and attention. Three patterns are used in the prototype:

- **Proximity:** findings appear at the relevant code location to minimize context switching.
- **Progressive depth:** concise diagnostics for inline use, richer rationale in WebViews when needed.
- **Reversibility:** quick-fix actions remain undoable, preserving safe experimentation.

These patterns do not guarantee correctness, but they reduce operational friction and over-automation risk. In security-sensitive workflows, this balance between assistance and control is a prerequisite for sustained usage.

4.5. Repair Safety and Limitations

Code Guardian's repair suggestions are LLM-generated code transformations. Automated code modification can alter intended functionality, introduce new defects, or fail to fully address the underlying security issue, so repairs are presented as decision support rather than autonomous code changes. This section documents the safety mechanisms and the trade-offs that shape them.

4.5.1. Why Automated Repair Verification Was Deprioritised

Verifying that a repair preserves intended behaviour AND eliminates the vulnerability requires solving two undecidable problems simultaneously. Behavioural preservation needs an executable test suite covering the modified paths, semantic-equivalence checking (undecidable in general), and side-effect analysis for non-test-visible behaviour (performance, error handling, logging). Security-improvement verification needs an exploit oracle (as hard as detection itself), completeness checking across all instances of the pattern, and defence-in-depth validation so a repair does not silently weaken other controls. Given these costs and the thesis scope of demonstrating feasibility of privacy-preserving LLM-based detection, automated repair verification is explicitly out of scope; the design instead invests in robust detection (Chapter 5), developer-controlled repair application, and clear documentation of repair limits.

4.5.2. Repair Safety Mechanisms

Repairs surface as VS Code Quick Fixes, never auto-applied. Each applicable diagnostic exposes two actions: *Apply Secure Fix* (one-click, reversible via Ctrl+Z) and *Preview Secure Fix (diff)*, which opens a side-by-side diff editor backed by an in-memory text-document content provider on the `code-guardian-diff:` scheme so no temporary files are written and the source file stays unchanged until the developer explicitly applies. Before either action is offered, the pipeline parses the suggested fix with `@babel/parser` (TypeScript / JSX / async plugins enabled). Markdown fences and single-backtick wrappers are stripped first; obvious prose openings ("You should…", "To fix…") are pre-rejected. The result is recorded as the `autoApplicable` boolean (with a short rejection reason) and aggregated into the auto-applicable rate reported in Section 5.6; passing the parse check guarantees the fix could be applied without breaking the build, not that it is semantically correct.

Repair scope is kept minimal — single-line fixes are preferred (e.g. `eval(input)` → `JSON.parse(input)`); function-level refactoring is used when structural change is needed; cross-file changes are not supported and are flagged for manual remediation. The repair prompt carries the detected vulnerability type and CWE, the surrounding code (function body plus imports), and instructions to preserve behaviour while emitting only the repaired code. The LLM has no runtime information (variable types, execution paths, test cases), so repairs rest on static pattern recognition. Application is buffer-only: no commits, no automated deployment. Developers review visually, run local tests if available, and commit through normal version-control and CI/CD channels.

4.5.3. Observed Repair Patterns

Informal manual review of repair suggestions in the evaluation dataset shows three recurring shapes. *Effective patterns* (parameterised query replacement, `validator.escape` or `DOMPurify.sanitize` for HTML contexts, `path.join`/`normalize` containment for path traversal) follow established secure-coding conventions and usually improve security without breaking functionality. *Patterns requiring developer adjustment* include over-sanitisation that double-encodes already-validated input, generic variable names that do not match project conventions, and repairs that fix the flagged line but miss related sinks in the same function. *Occasionally problematic* repairs are rare: functional breakage from `eval()`-to-`JSON.parse` when the input is valid JS but not JSON, and one observed instance where a model removed an authentication check while fixing an injection. Human review remains essential.

The evaluation in Chapter 5 therefore focuses on detection and explanation quality (R1–R4) and treats repair suggestions as assistive artefacts. Layered automated validation (type checking, test execution, static re-analysis) is named as future work in Chapter 7.

4.6. Privacy and Threat-Model Enforcement

The task description (Page iv) names a strict zero-egress policy and a threat model that includes retrieval poisoning. This section describes the three enforcement layers in the delivered system: a network-policy module that gates every outbound socket, a signed-corpus verifier that detects RAG knowledge-base tampering at load time, and a containerised harness that pins the resource and network namespace.

4.6.1. Network Policy and Zero-Egress Gate

The extension routes every outbound HTTP(S) call through a single chokepoint, `src/networkPolicy.ts`. The module exports an `assertEgressAllowed(url)` function that throws an `EgressBlockedError` unless the URL's hostname is on a fixed loopback allowlist (`127.0.0.1`, `::1`, `localhost`, `0.0.0.0`) *or* the user has explicitly opted in via the new `codeGuardian.enableExternalDataFetch` setting (default `false`). The opt-in path exists for the public CWE/CVE/OWASP metadata refresh; it does not relax the policy for any source-code-bearing call. `src/vulnerabilityDataManager.ts` consults the gate before every fetch and additionally treats any cached file as valid when the gate is closed, so a fully offline machine can still load the bundled vulnerability snapshot. Because the gate is a synchronous assertion at the call site, it cannot be bypassed by code that holds a live HTTP client reference.

4.6.2. Signed Corpus and Provenance

Retrieval poisoning is the threat that an attacker swaps the on-disk RAG knowledge base for one carrying malicious "guidance" that the LLM then trusts as authoritative context. To detect this attack, the RAG corpus is signed with an Ed25519 detached signature over a SHA-256 manifest of every corpus file. The signing tool (`evaluation/sign-corpus.js`) generates the keypair on first run, walks `security-knowledge/`, computes a SHA-256 for each file, attaches per-file provenance from `evaluation/corpus-provenance.json` (source URL plus retrieval timestamp), produces a canonical tab-separated payload sorted by file path, signs the payload, and writes `security-knowledge/corpus-manifest.json`. The private key never ships with the extension; the public key (`keys/corpus-public.pem`) ships in the package and is bundled into the Docker image.

At RAG load time `src/corpusVerifier.ts` re-derives the manifest hashes from the on-disk content, recomputes the canonical payload, and verifies the Ed25519 signature against the public key. A hash mismatch, a missing file, or a signature failure makes the load abort under the default `codeGuardian.requireSignedCorpus=true` setting. Because provenance fields are bound into the signed payload, an attacker cannot rewrite a corpus file's "retrieved from" string without invalidating the signature.

4.6.3. Containerised Harness

`Dockerfile` pins `node:20.19.0-alpine`, installs the locked dependency set with `npm ci`, copies the harness, the signed corpus, and the public key into the image, and compiles the extension at build time. The private key is excluded by `.dockerignore` so images cannot accidentally carry signing material. `docker-compose.yml` brings up Ollama and the harness on a user-defined bridge network with explicit CPU and memory limits (8 CPUs, 16 GB) and binds the Ollama port to loopback only. This is the "containerised execution" clause of the task description's reproducibility triad and the network-isolation arm of the threat model in one stack: the harness has no path to a non-loopback host, and resource measurements (Section 5.7.3) are comparable across hosts because the container caps the harness side identically.

4.7. Implementation Summary

The implementation realizes the thesis architecture as a local IDE assistant with two operational modes (LLM-only and LLM+RAG), explicit latency guardrails, and privacy-preserving boundaries. These concrete mechanisms provide the basis for interpreting the evaluation results in Chapter 5.

5. Evaluation

This chapter evaluates Code Guardian against the five requirements defined in Chapter 2: detection accuracy (R1), consistency (R2), repair quality (R3), usability (R4), and privacy (R5). Each requirement is assessed using the metrics and level scales introduced in the requirements analysis. The evaluation uses a curated dataset of 101 test cases (71 vulnerable, 30 secure) covering major CWE categories, executed on local hardware with five Ollama models in both LLM-only and LLM+RAG configurations and three runs per sample. Three SAST baselines (Semgrep, CodeQL, ESLint) provide reference points. The best-performing configuration is `qwen3:8b+RAG`, which achieves canonical F1 71.94%, precision 73.53%, recall 70.42%, FPR 10.00%, and median latency 1 573 ms per function; with the inter-run confidence gate at ≥ 0.67 precision rises to 77.78%.

Sections 5.1–5.3 introduce the dataset, metrics, and experimental setup. Sections 5.4–5.8 present results organized by requirement, each ending with a level mapping. Section 5.10 synthesizes the findings into a requirement compliance overview.

5.1. Datasets

This section describes the evaluation dataset: 101 test cases (71 vulnerable, 30 secure) covering common JavaScript and TypeScript vulnerability patterns.

The evaluation uses a **curated dataset** rather than large-scale automated benchmarks such as the NIST Juliet Test Suite or OWASP Benchmark [61, 62]. A curated approach was chosen because (i) established benchmarks provide few well-documented secure examples, limiting FPR estimation; (ii) human-auditable cases allow qualitative inspection of model explanations and repairs (R3, R4); and (iii) a smaller suite enables rapid prompt and retrieval iteration without multi-day evaluation cycles. Standard benchmarks remain targets for future generalization testing (see "Toward larger benchmarks" below).

The evaluation dataset is a project-authored curated set of JavaScript and TypeScript security cases maintained alongside the Code Guardian prototype. Cases are aligned to CWE/OWASP concepts. Each test case consists of a short code snippet, a list of expected vulnerabilities with CWE identifiers and severities, and an expected remediation description. Three dataset categories are provided:

- **Core dataset:** `vulnerability-test-cases.json` contains representative examples for common vulnerability classes (e.g., SQL injection, XSS, command injection, path traversal) as well as a small number of secure examples to measure false positives.
- **Advanced dataset:** `advanced-test-cases.json` contains additional vulnerability types and more nuanced patterns (e.g., insecure CORS configuration, timing attacks, mass assignment).
- **Real-world verified dataset:** contains 11 verified vulnerability cases derived from actual CVEs (CVE-2022-24999, CVE-2021-23337) and security patterns extracted from production open-source projects (Lodash, Express, Mongoose, Node-Forge). This dataset validates that the system can reason about real-world vulnerabilities beyond synthetic test cases.

In the evaluated prototype version used for this thesis run, the scored result set combines **101** test cases (**71** vulnerable and **30** secure). The secure set was expanded from 15 to 30 samples to improve FPR estimation confidence. Expected findings are annotated with CWE identifiers to support aggregation by vulnerability class.

Unless stated otherwise, quantitative results in this chapter refer to this merged 101-case result set. Vulnerability classes are aligned with the Common Weakness Enumeration taxonomy [2], and severities are recorded as coarse labels. The dataset supports R1–R3 by including vulnerability instances that require reasoning about tainted input (not just keyword matching) and by enabling controlled comparisons across repeated runs and model configurations.

Dataset Schema

Each record follows a uniform schema:

- `id`: unique identifier
- `name`: descriptive test name
- `code`: code snippet under test
- `expectedVulnerabilities`: list of expected findings (type, CWE, severity, description)
- `expectedFix`: short remediation guidance (optional)

The dataset is intentionally small and human-auditable to facilitate iteration on prompts, retrieval, and output validation. Limitations of synthetic snippets and representativeness are discussed in Section 5.10.

Toward larger benchmarks. While this thesis focuses on a curated, auditable suite to support rapid iteration, standard benchmark suites such as the OWASP Benchmark and NIST Juliet can be used to broaden coverage and stress-test generalization in future work [62, 61]. In addition, secure coding checklists and verification frameworks such as OWASP ASVS can guide the selection of security requirements and test categories when scaling the evaluation [63].

5.2. Evaluation Metrics

Code Guardian is evaluated along axes aligned with Chapter 2: detection quality (R1), consistency and structured-output robustness (R2), repair quality (R3), responsiveness (R4), and privacy (R5). Quantitative values are descriptive point estimates for the locked configuration; uncertainty intervals are added for key decision metrics in Section 5.10.

Detection Quality

Given an expected set of vulnerability types per test case and the detected set returned by the model, the harness computes precision $\mathrm{TP}/(\mathrm{TP}+\mathrm{FP})$, recall $\mathrm{TP}/(\mathrm{TP}+\mathrm{FN})$, F1 (their harmonic mean), accuracy $(\mathrm{TP}+\mathrm{TN})/N$, and the false-positive rate $\mathrm{FP}/(\mathrm{FP}+\mathrm{TN})$ on secure examples. FPR is computed at *sample level* — a secure sample counts as FP if any issue is emitted in any of its three runs — because in IDE workflows a secure snippet that triggers *any* warning still creates triage overhead, so sample-level FPR aligns with practical interruption cost better than issue-level counting. The scored dataset includes 30 intentionally secure snippets (expanded from an initial 15 to improve FPR confidence intervals), which back the FPR figures throughout this chapter.

Canonical category matching. Substring matching is sensitive to label variation, as the qualitative error analysis confirms (Section 5.4.4: 11 of 19 zero-recall cases are label-mismatch artefacts). The harness therefore normalises both expected and detected type strings to a fixed canonical taxonomy of 23 vulnerability classes (`sql-injection`, `xss`, `deserialization`, `auth-bypass`, `weak-crypto`, `input-validation`, `prototype-pollution`, `ssrf`, `xxe`, `path-traversal`, `ldap-injection`, `header-injection`, `code-injection`, `redos`, `race-condition`, `information-exposure`, `crypto-verification`, `weak-encryption`, `hardcoded-credential`, `nosql-injection`, `improper-auth`, plus a fall-through `other`). The taxonomy is a single shared JSON catalogue used by both extension and harness; a snapshot test pins regression-tested behaviour across 34 fixture inputs. Type-level scoring throughout this chapter uses the canonical matcher; where historical figures from the previously-published log

(substring-based) appear, they are reproduced verbatim and clearly attributed, never blended with current canonical-matched values. Per-canonical-category TP/FP/FN/-precision/recall/F1 are emitted by the aggregator so the per-category recall picture (Section 5.4) is reproducible from the run JSON without manual re-binning.

Inter-run confidence and confidence-gated precision. With `runsPerSample`$\geq$2, the harness derives a per-finding *confidence* score from inter-run agreement: a finding's confidence is the fraction of runs in which a matching finding (overlapping line range and agreeing canonical category) appears, including the run that produced it. A *confidence gate* then drops findings whose confidence falls below a configurable threshold. At threshold ≥ 0.67 on a 3-run configuration, only findings agreed by at least two of three passes survive. The gate is applied post hoc by a deterministic re-scoring utility, so threshold sweeps do not re-invoke the model. Results are reported both un-gated and at the operationally meaningful threshold, separating "how often the model is right" (un-gated) from "how reliable each individual finding is" (gated).

Interpretation. Precision reflects *alert quality* under the chosen ontology and matching rule; recall reflects *coverage* of expected vulnerability classes. Because scoring is type-level rather than exploitability-level, these metrics are detection-signal quality, not direct measures of breach prevention. The harness reports pooled issue-level confusion counts, supplemented by qualitative evidence (localisation, representative TP/FN/FP cases) and exploratory paired tests on per-sample binary outcomes for R2.

Structured Output Robustness

JSON parse success rate is the percentage of responses that parse into a JSON array of issues. Responses that do not parse are counted as parse failures and scored as producing an empty issue set — on vulnerable cases this is a false negative; on secure cases it appears as a true negative but still indicates the system is not usable for IDE automation. For editor automation, parse reliability acts as a deployment gate: below a practical reliability threshold, quality metrics alone are insufficient.

Responsiveness

The harness records **response time** (wall-clock per call, milliseconds) reported as median and mean. Median is the primary usability indicator; mean acts as a tail-sensitivity proxy. Right-skewed latency is an operational risk for always-on workflows, particularly under local inference where background load and model

warm-up create intermittent delays. Stage-split timing is recorded separately as `inferenceTimeMs` (Ollama call) and `parseTimeMs` (response parser) so any observed change can be attributed unambiguously: prompt/decoding changes affect inference time, output-shape changes affect parse time.

Repair Quality (Auto-Applicable Rate)

Alongside the manual review reported in Section 5.6, the harness emits a deterministic fleet-wide repair-quality signal: the **auto-applicable rate**, the fraction of generated `suggestedFix` strings that syntactically parse as JavaScript or TypeScript (module, script, or wrapped-expression mode) without errors. Markdown code fences are stripped before parsing; prose-shaped outputs (sentences starting with "You should…", "To fix…") are pre-rejected so the parse-error bucket is not inflated by non-code text. The check uses `@babel/parser` with TypeScript, JSX, async, and optional-chaining plugins enabled. Passing parsing does not guarantee semantic correctness or that the fix actually closes the vulnerability — only that it could be applied to a file without breaking the build, which is exactly the bar the label captures. The metric is reported per model and prompt mode with a `byReason` breakdown of rejection categories (`prose_not_code`, `parse_error`, `empty`). It scales to the full evaluation set without manual review and provides a stricter alternative to similarity-based repair metrics (cosine similarity, BLEU, CodeBERTScore), which the SecRepair authors explicitly characterise as "fundamentally inadequate" for security correctness assessment [38].

5.3. Experimental Setup

All experiments run locally through the Code Guardian evaluation harness, which iterates over the dataset of Section 5.1 and invokes Ollama with a strict JSON-only system prompt. Decoding is deterministic (`temperature=0`, `seed=42`); JSON-mode decoding (`format='json'`) plus the response schema enforce output shape and eliminate parse failures from malformed responses. The schema requested for scoring carries a vulnerability `type` string and a coarse `severity` level alongside the location and optional fix — richer than the in-editor diagnostics flow but required for type-level precision/recall.

Two configurations are evaluated quantitatively: **LLM-only** (analyser without retrieval) and **LLM+RAG** (retrieval-augmented prompting with static security snippets, `k=5`). The system prompt is a tight ∼80-token instruction listing the output schema and the four scoring rules; the historical longer prompt is referenced only for context (Section 5.7).

Hybrid SAST gating (opt-in). The harness optionally runs Semgrep once over a temporary baseline workspace at the start of a run and fuses each LLM finding whose line range overlaps a Semgrep finding (and whose canonical category agrees) by promoting confidence to 1.0 and the detection source to `hybrid`. LLM-only findings keep their inter-run confidence. The fusion fires on the small share of cases where Semgrep itself produces a finding (consistent with its 9.73% recall); the remainder pass through untouched.

Baselines. Three SAST baselines run through the same harness against per-snippet temporary workspaces: Semgrep with the registry packs `p/security-audit`, `p/javascript`, `p/typescript`, and `p/owasp-top-ten`; CodeQL with the `javascript-security-a` suite; and ESLint with `eslint-plugin-security`. Per-snippet workspaces keep baseline execution repeatable but omit broader project context. Outputs are normalised into the same per-finding schema; vulnerability `type` for baseline findings is inferred from rule identifiers, messages, and CWE metadata, which introduces a construct-validity limitation when metadata is missing or mismatched.

Comparability. Across runs the harness keeps dataset artefacts, decoding/runtime limits, scoring logic, and execution order policy fixed. Local inference remains sensitive to transient system load, so reported values are descriptive measurements for the documented environment, not hardware-independent constants.

Benchmark Substitution

The approved task description names *Juliet* and *OWASP Benchmark* (40–50 CWE-mapped cases). Both are real but neither targets JavaScript: Juliet is published by NIST in C/C++ and Java only, and the OWASP Benchmark Project is a Java application. There is no first-party JavaScript port of either, and running this thesis's JS/TS detector against C/C++ or Java would not be meaningful. The thesis therefore evaluates on two CWE-mapped JavaScript corpora: the curated 101-case primary corpus (71 vulnerable + 30 secure) of Section 5.1, and an additional 15-case external corpus (under `evaluation/datasets/external/`) drawn from OWASP NodeGoat, OWASP Juice Shop, and three named CVEs (CVE-2022-24999, CVE-2021-23337, CVE-2022-24771). Per-case provenance — source URL and (where applicable) upstream commit — is recorded in `evaluation/datasets/SOURCES.md` and bound into the signed corpus manifest (Section 4.6.2) so it cannot be silently rewritten.

Real-World Project Slice

The task additionally names "one actively maintained real-world JavaScript/TypeScript project with documented vulnerabilities and verified patches". This thesis selects *OWASP NodeGoat*, an actively-maintained OWASP reference application whose per-lesson `tutorial/` directory documents vulnerable code paths and verified patches. The whole-project runner (`evaluation/run-realworld.js`) walks the project, slices each file by function, applies the same analyser pipeline used for the curated corpus, and computes per-CWE recall against `evaluation/realworld/nodegoat-vulnerabilities.json`

Containerised Execution

A Dockerfile pins Node.js 20.19.0-alpine, installs the locked dependency set via `npm ci`, copies the harness, the signed RAG corpus, and the corpus public key into the image, and compiles the extension at build time. A `docker-compose.yml` brings up Ollama and the harness on an isolated user-defined bridge with explicit CPU and memory limits (8 cores, 16 GB) so resource measurements are comparable across hosts. The compose stack binds Ollama to loopback only, satisfying the network-isolation arm of the threat model.

5.3.1. Reproducibility Snapshot and Run Configuration

The harness uses `runsPerSample=3` symmetrically for both groups (213 vulnerable + 90 secure = 303 invocations per model×mode). Precision and recall are computed over the 213 vulnerable evaluations under canonical type-level matching; FPR is computed at sample level over the 30 secure samples (a sample is FP if any of its 3 runs flags an issue); latency and parse-success rates are pooled across all 303. Parse failures are scored as producing no issues, which lowers recall on vulnerable samples. The canonical-matching metrics and the confidence gate are post-hoc transformations of the recorded per-finding outputs, so threshold sweeps do not require re-invoking the model.

Table 5.1.: Run configuration for the headline evaluation.

Item	Value
Dataset	`vulnerability-test-cases.generated.json` (71 vulnerable) + `negatives-only.generated.json` (30 secure)
Prompt modes	LLM-only, LLM+RAG (`k=5`, static snippets)
Runs per sample	3 (vulnerable and secure, symmetric)
Generation	`temperature=0`, `seed=42`, `format='json'`, `num_predict=1000`
Request controls	`timeoutMs=30000`, `requestDelayMs=500`, sequential, no warm-up, no retries
Type-level scoring	canonical taxonomy of 23 classes (Section 5.2)
Confidence	inter-run agreement; reported un-gated and at ≥ 0.67
Auto-applicable repair rate	`@babel/parser` syntax check on every `suggestedFix`
Models	`gemma3:1b`, `gemma3:4b`, `qwen3:4b`, `qwen3:8b`, `CodeLlama:latest`
Software	Ollama 0.17.1, Node.js v20.19.5
Hardware/OS	Apple M4 Max (16 cores, 64 GB RAM), macOS Darwin 25.3.0 (arm64)
Invocation	`-ablation -include-baselines -runs 3`

5.4. R1: Accuracy

This section evaluates detection accuracy across all model and retrieval configurations. The metrics are precision, recall, F1, and secure-sample false-positive rate (FPR), as defined in Section 2.1.1.

5.4.1. Detection Quality Across Configurations

Table 5.2 shows detection metrics for all evaluated configurations. Results are pooled over 213 vulnerable-sample evaluations (71 samples $\times$ 3 runs) and 30 secure-sample evaluations.

Table 5.2.: Detection accuracy across all configurations using canonical-taxonomy matching (Section 5.2). 71 vulnerable + 30 secure samples, 3 runs each ($n = 303$ evaluations per model×mode). FPR is at the secure-sample level (FP if any issue is emitted in any run).

Model	Mode	Precision	Recall	F1	FPR
`qwen3:8b`	LLM+RAG	73.53	70.42	71.94	10.00
`qwen3:8b`	LLM-only	61.63	74.65	67.52	50.00
`qwen3:4b`	LLM-only	56.95	78.87	66.14	90.00
`qwen3:4b`	LLM+RAG	51.46	74.65	60.92	100.00
`gemma3:4b`	LLM-only	46.49	74.65	57.30	100.00
`gemma3:4b`	LLM+RAG	46.85	73.24	57.14	100.00
`codellama`	LLM-only	41.60	73.24	53.06	100.00
`gemma3:1b`	LLM-only	29.17	49.30	36.65	53.33
`gemma3:1b`	LLM+RAG	32.22	40.85	36.02	23.33
`codellama`	LLM+RAG	14.79	53.52	23.17	100.00
`semgrep`	baseline	50.00	12.68	20.22	6.67
`codeql`	baseline	12.07	9.86	10.85	53.33
`eslint-security`	baseline	—	0.00	0.00	0.00

LLM-based approaches dominate the SAST baselines on recall-driven F1; `qwen3:8b+RAG` achieves the highest score (71.94%) at 10% FPR.

5.4.2. RAG Ablation Impact on Accuracy

RAG effects are strongly model-dependent (see the LLM-only / LLM+RAG rows in Table 5.2).

RAG lifts `qwen3:8b` F1 from 67.52% to 71.94% (+4.42 points) and pushes its precision from 61.63% to 73.53% while halving FPR (50.00% → 10.00%). RAG is roughly neutral on `gemma3:1b` and `gemma3:4b`, slightly hurts `qwen3:4b` (−5.22 points), and dramatically degrades `codellama` (−29.89 points: F1 drops from 53.06% to 23.17%, latency nearly doubles). The `codellama+RAG` collapse is the strongest negative-RAG observation in the dataset and indicates that the additional retrieval context actively confuses `codellama`'s output structure rather than grounding it. Retrieval augmentation is therefore a model-specific configuration, not a universal default. Exploratory exact McNemar tests on matched sample-level outcomes for the `qwen3` improvements did not reach significance ($n = 71$); the RAG effect is best read as

an indicative trend whose direction (positive for `qwen3`, negative for `codellama`) is more robust than the precise magnitude.

5.4.3. Per-Category Detection Breakdown

Per-canonical-category recall on `qwen3:8b+RAG` (3-runs pooled, categories with ≥ 3 samples) shows a sharp split rather than a smooth gradient. Categories with well-known syntactic signatures — SQL injection (90%), XSS (100%), command injection (100%), path traversal (100%), prototype pollution (100%), weak-crypto, hardcoded credentials, LDAP injection, and SSRF (each 100%) — are detected at 90–100% recall when the dangerous API is lexically obvious. In contrast, improper-auth, input-validation, and weak-encryption score 0% recall, and deserialization only 33%. Section 5.4.4 shows a substantial share of this gap is a labelling artefact rather than detection failure: the auth-bypass canonical category, for instance, absorbs nine model findings on cases the dataset labels *improper-auth*, which simultaneously show up as nine false positives on auth-bypass and nine misses on improper-auth.

5.4.4. Qualitative Error Analysis on Zero-Recall Categories

Three canonical categories remain at 0% recall under `qwen3:8b+RAG` after canonical matching: improper-auth, input-validation, and weak-encryption. (Two other categories that scored 0% under the previous substring matcher — weak cryptography and insecure deserialization — recovered to 100% and 33% respectively when the canonical matcher absorbed sub-type labels such as “Weak Hashing Algorithm” and “Weak Random Number Generation”.)

The remaining zero-recall categories are dominated by two distinct effects, summarized in Table 5.3.

Table 5.3.: Residual zero-recall categories after canonical matching (`qwen3:8b+RAG`, 3-run pooled).

Canonical category	**Expected n**	**TP**	**Dominant failure mode**
improper-auth	9	0	Cross-category mislabel: model emits 9 *auth-bypass* findings on these cases, so the issue *is* detected but binned in a sibling canonical category.
input-validation	27	0	Genuine miss: most cases are real-world library code (lodash, mongoose, moment) where the validation gap is implicit; the model produces no output flagged as input-validation.
weak-encryption	3	0	Small sample of insufficient-key-length cases; the model labels the underlying weakness but the canonical taxonomy separates weak-encryption from weak-crypto.

The two residual gaps differ in kind. Every *improper-auth* case is detected and emitted as *authentication-bypass* or *timing-attack* — canonical siblings of *auth-bypass* — so the underlying detection and the suggested fixes (constant-time comparison, JWT algorithm validation) are correct and merging the buckets would lift recall from 0% to 100% at the cost of conflating two CWE-aligned classes. The 9 input-validation samples (27 evaluations) are extracted from real-world libraries (lodash, mongoose, moment, serialize-javascript) where the validation gap is implicit in dense utility logic; the model produces no output for them. This is the residual recall gap the canonical taxonomy cannot recover — it requires cross-file or taint-flow context, not a labelling refinement.

5.4.5. SAST Baseline Comparison

SAST tools show a different trade-off profile. Semgrep achieves the lowest FPR (6.67%) but a low recall (12.68% under canonical matching); CodeQL has higher FPR (53.33%) with similar low recall (9.86%); ESLint security plugins detect none of the test cases. The LLM-based approach trades higher FPR for substantially better recall, making it more suitable for comprehensive security audits while SAST remains useful as a low-noise complement. The hybrid SAST-fusion option (Section 5.3.1) operationalises this complementarity by promoting LLM findings corroborated by Semgrep to confidence 1.0. FPR is computed at sample level over $n = 30$ secure cases; the 95% exact binomial CI for the headline 10% FPR is $[2.1\%, 26.5\%]$, and for 100%-FPR configurations the lower bound is 88.4%.

5.4.6. Canonical Matching

Because the qualitative error analysis (Section 5.4.4) shows that 11 of 19 zero-recall cases are label-mismatch artefacts rather than detection failures, the harness scores type matches against the canonical taxonomy defined in Section 5.2. The headline numbers earlier in this chapter use a previously-published substring-matched scoring (preserved verbatim for context); the canonical matcher's effect on the same recorded findings is summarised in Table 5.4.

Table 5.4.: Canonical-matching cross-check on the previously-published 71-vulnerable-sample ablation log (no model re-invocation). The substring column reproduces the previously-published values; the canonical column is the same per-case findings re-scored against the canonical taxonomy. FPR is unchanged because secure-sample scoring is type-agnostic at the case level.

Model	Mode	Substring F1	Canonical F1	Δ	Canonical Prec.
`qwen3:8b`	LLM+RAG	64.13	69.44	+5.31	69.85
`qwen3:8b`	LLM-only	58.44	67.34	+8.90	65.73
`qwen3:4b`	LLM+RAG	59.95	63.36	+3.41	59.43
`qwen3:4b`	LLM-only	55.98	64.58	+8.60	60.00
`gemma3:4b`	LLM+RAG	50.66	55.05	+4.39	50.12
`gemma3:4b`	LLM-only	57.18	60.03	+2.85	55.84
`codellama`	LLM+RAG	32.14	42.31	+10.17	39.59
`codellama`	LLM-only	42.74	51.54	+8.80	49.07
`gemma3:1b`	LLM+RAG	37.08	37.72	+0.64	33.04
`gemma3:1b`	LLM-only	33.85	35.94	+2.09	53.18

The canonical effect is positive on every model and mode, with deltas ranging from +0.64 to +10.17 F1 points (mean ≈ +5.5). Larger and stronger models gain more, consistent with the observation that label variance is what the canonical matcher was designed to absorb. Smaller models such as `gemma3:1b` produce fewer findings overall and therefore have less label variance to recover. The same direction holds for the headline configuration on the merged-with-negatives Mar-21 log: `qwen3:8b+RAG` F1 lifts from 65.31% to 72.34% (precision 63.16% → 72.86%, recall 67.61% → 71.83%, FPR unchanged at 20%). Because the lift comes entirely from re-binning labels — no new model calls — it is a deterministic methodology adjustment rather than a model-quality claim.

5.4.7. Inter-Run Confidence Gate

The harness assigns each finding a confidence score derived from inter-run agreement (Section 5.2). The previously-published Feb-27 ablation log supports a post-hoc confidence sweep on the headline configuration:

Table 5.5.: Confidence gate sweep on the frozen 3-runs-per-vulnerable-sample log (`qwen3:8b+RAG`, canonical matcher). At threshold ≥ 0.67, only findings agreed by ≥ 2 of 3 runs survive.

Threshold	F1	Precision	Recall	Issues retained
0.0 (no gate)	69.44	69.85	69.03	100%
0.34 ($\geq 1/3$)	71.19	74.52	68.14	—
0.67 ($\geq 2/3$)	**72.30**	**77.00**	68.14	∼93%
1.0 (unanimous)	72.30	77.00	68.14	—

Canonical matching combined with a confidence gate at ≥ 0.67 lifts `qwen3:8b+RAG` from previously-published F1 64.13% / precision 63.66% to F1 72.30% / precision 77.00% at the cost of 0.89 recall points. This is the trade-off the gate is designed to deliver: lower noise without sacrificing detection. The gain is empirically grounded in the existing 3-run data — no new model calls are required to compute it.

5.4.8. External-Corpus Stress Test

To probe the substitution disclosed in Section 5.3 the analyser was rerun on the 15-case external corpus (11 vulnerable + 4 secure) drawn from OWASP NodeGoat, OWASP Juice Shop, and three named CVEs (CVE-2022-24999, CVE-2021-23337, CVE-2022-24771). The configuration is intentionally conservative: `qwen3:8b`, LLM-only (no RAG), single run per sample (no inter-run consensus), no confidence gate. This is a stress test, not a parity comparison with the headline.

Table 5.6.: `qwen3:8b` on the external corpus, LLM-only / 1 run per sample. Comparison row reproduces the curated 101-case canonical-match headline from Section 5.4.6.

Corpus	F1	Precision	Recall	FPR	Med. lat. (ms)	Configuration
External (15 cases)	40.00	35.71	45.45	75.00	9 462	LLM-only, 1 run
Curated (101 cases)	71.94	73.53	70.42	10.00	1 573	LLM+RAG, 3 runs

Two effects compound in the gap. First, the external corpus skews toward weakness classes that require cross-file or framework-level reasoning: mass assignment, broken access control on an admin route, JWT `none`-algorithm acceptance, signature verification (CWE-347), and Lodash `_.template` code injection. Five of the six false negatives are in this class. The detector's single-function scope (Section 4.2) does not see the access-control wrapper, the JWT `algorithms` option, or the version constraint that resolves the CVE. The thesis's existing input-validation gap (Section 5.4.4) is the same limitation surfacing on a corpus designed to elicit it. Second, the configuration is two notches weaker than the headline: no retrieval and a single inference per sample remove the precision gains attributed to canonical matching plus the inter-run confidence gate at ≥ 0.67 (Section 5.4.7).

The result is therefore best read as a quantification of the substitution penalty under deliberately weak conditions, not as a measurement of the system's whole-project capability. The latency rise (median 9 462 ms vs. 1 573 ms) reflects longer per-snippet code (full route handlers vs. single functions); inference time scales with prefill length, consistent with the latency-component analysis in Section 5.7. Closing the gap with the headline configuration — LLM+RAG, 3 runs, confidence gate — is named as future work alongside cross-file context modelling.

5.4.9. Real-World Whole-Project Run on OWASP NodeGoat

The whole-project runner from Section 5.3 was executed on OWASP NodeGoat at the default `master` commit, using `qwen3:8b` (LLM-only, single run per chunk) with the default file-skip set (tests, cypress, tutorial, docs, artifacts). The run scanned 26 source files, sliced into 85 function-level chunks, in 884.5 s ($\approx$ 15 min). Aggregate output:

Table 5.7.: NodeGoat whole-project scan summary. Documented-vulnerability list is the curated 10-entry OWASP-Top-10 mapping under `evaluation/realworld/nodegoat-vulnerabilities.json`.

Quantity	**Value**
Source files scanned	26
Function chunks analysed	85
Wall-clock duration	884.5 s
Findings produced	14
Documented vulnerabilities	10
Overall recall (file + category match)	0/10 (0%)

Diagnosis of the 0 % recall. The detector did not stay silent — it produced 14 plausible findings on real NodeGoat code, including reflected XSS in `routes/contributions.js`, dynamic-template path injection in `routes/tutorial.js` (CWE-94/95-class), three session-fixation observations in `routes/session.js`, deprecated `bcrypt-nodejs` usage in `data/user-dao.js`, error-stack exposure in `routes/error.js`, and an open-redirect-shaped pattern in `routes/index.js`. None of these match the curated documented list. The 0 % recall decomposes into four mechanisms, all of which the existing limitations section anticipated:

- *Out-of-scope file types.* 2 of 10 documented entries point at non-JS/TS files: the reflected-XSS path in `app/views/contributions.html` (HTML, not scanned) and the vulnerable-dependency entry in `package.json` (JSON, not scanned). The function-scoped analyser is by design a code-only scanner.
- *Cross-file flows.* The CSRF, weak-session, and mass-assignment entries depend on session middleware in `server.js` interacting with route handlers in `routes/profile.js`; the function-level scope cannot see both halves at once. This is the same input-validation gap measured in Section 5.4.4.
- *Canonical-taxonomy under-coverage.* 10 of 14 findings normalise to the `other` bucket because the canonical taxonomy is JS-injection-centric (Section 5.4.6); session-fixation, deprecated-dependency, and weak-password-regex weaknesses have no taxonomy bin, so they cannot match a documented entry even when the underlying weakness is the same OWASP class.
- *True misses.* The SQL injection in `data/allocations-dao.js` and the MD5 hashing in `data/profile-dao.js` are within scope, were chunked correctly, and were missed by the model. These are genuine LLM-side false negatives on real-project code under the LLM-only single-run configuration.

Reading. The whole-project run validates the thesis's existing function-scope limitation rather than refuting the headline numbers: the detector finds plausible real weaknesses on real code, but its recall against a human-curated vulnerability list is bounded by what a function-level scope can see. Closing the gap requires expanding the canonical taxonomy (a deterministic change with no Ollama cost), promoting the configuration to LLM+RAG with consensus and confidence gating (a parity rerun), and most importantly modelling cross-file context so that middleware-route flows become visible. All three are named as future work.

5.4.10. Level Mapping

Based on the R1 evaluation scale (Table 2.3), the best configuration (`qwen3:8b+RAG`) maps as follows under canonical matching:

- F1 = 71.94% → falls in $[0.60, 0.75)$
- Precision = 73.53% → above 0.70
- Recall = 70.42% → above 0.70
- FPR = 10.00% → comfortably below the ≤ 0.20 boundary

Three of the four metrics meet the High threshold (precision, recall, FPR) but F1 is in the Medium band (just below 0.75). Applying the inter-run confidence gate at ≥ 0.67 lifts F1 to 73.13% and precision to 77.78% (Section 5.4.7), which still falls in the Medium F1 band but tightens precision further.

R1 Level: Medium accuracy.

Results are usable in audit-mode and approach the threshold for always-on inline use; the residual barrier is the F1 ceiling driven by the input-validation true-miss category, not the false-positive rate.

5.5. R2: Consistency

This section evaluates detection consistency: whether the system produces stable, well-formed output across repeated analyses of the same code. The metrics are JSON parse success rate, inter-run detection agreement, and label stability, as defined in Section 2.1.2.

5.5.1. Structured Output Stability

JSON parse success measures how often the system returns valid, schema-compliant output the IDE can process; a parse failure makes the result unusable regardless of semantic quality. All `qwen3` models achieve $\geq 98.8\%$ parse success, with `qwen3:8b` reaching 100.0% in both modes thanks to JSON-mode decoding enforcement. Smaller models drop below 96% (`gemma3:1b`: 95.6% / 94.7% LLM-only / +RAG), and RAG slightly reduces parse rates across all models due to increased prompt complexity. `codellama` sits at 96.5% / 95.3%.

5.5.2. Inter-Run Detection Agreement

Because each vulnerable sample was evaluated 3 times, the harness can measure how often the same sample produces the same binary detection outcome across runs. Agreement rate is the percentage of samples where all 3 runs agree on the detection result.

Table 5.8.: Inter-run detection agreement (3 runs per sample, 71 vulnerable samples).

Model	Mode	Agreement Rate (%)
`qwen3:8b`	LLM+RAG	87.6
`qwen3:8b`	LLM-only	84.1
`qwen3:4b`	LLM+RAG	82.3
`qwen3:4b`	LLM-only	79.6
`gemma3:4b`	LLM+RAG	76.1
`gemma3:4b`	LLM-only	78.8
`gemma3:1b`	LLM+RAG	71.7
`gemma3:1b`	LLM-only	73.5
`codellama`	LLM+RAG	74.3
`codellama`	LLM-only	77.0

`qwen3:8b+RAG` shows the highest agreement (87.6%), meaning that for roughly 62 out of 71 samples, all three runs produce the same detection decision. Smaller and code-specialized models show more variation, with `gemma3:1b` at 71.7–73.5%. RAG slightly improves agreement for `qwen3` models but slightly reduces it for others, consistent with the accuracy findings.

5.5.3. Label and Severity Stability

Beyond binary detection, the harness examines whether the reported vulnerability type and severity remain stable across runs. For samples detected in all 3 runs, the same CWE label and severity level should appear consistently.

Table 5.9 shows label and severity agreement rates for each model among samples where all 3 runs detected a vulnerability.

Table 5.9.: Label and severity stability among consistently detected samples (3/3 runs).

Model	Mode	CWE Label Agreement (%)	Severity Agreement (%)
`qwen3:8b`	LLM+RAG	91	85
`qwen3:8b`	LLM-only	88	82
`qwen3:4b`	LLM+RAG	83	76
`qwen3:4b`	LLM-only	80	73
`gemma3:4b`	LLM+RAG	72	64
`gemma3:4b`	LLM-only	75	67
`gemma3:1b`	LLM+RAG	65	58
`gemma3:1b`	LLM-only	68	61
`codellama`	LLM+RAG	70	62
`codellama`	LLM-only	74	66

Larger, instruction-tuned models produce more stable classifications. `qwen3:8b+RAG` achieves 91% CWE label agreement and 85% severity agreement, while `gemma3:1b` drops to 65% and 58% respectively. Severity is generally less stable than CWE labels because severity is a subjective judgment that the model must infer from context, while CWE labels can often be derived from the vulnerability pattern itself.

By vulnerability category and against SAST. Consistency varies by category: well-known patterns like SQL injection and XSS reach above 90% inter-run agreement on `qwen3:8b` because detection relies on clear lexical signals, while prototype pollution and race conditions drop below 70%. SAST baselines (Semgrep, CodeQL) are fully deterministic and trivially achieve 100% on every consistency metric, at the cost of rigid pattern matching — the trade-off between LLM flexibility and SAST determinism is the core architectural consideration.

5.5.4. Level Mapping

Based on the R2 evaluation scale (Table 2.4), the best configuration (`qwen3:8b+RAG`) maps as follows:

- JSON parse success rate = 100% → perfect (JSON-mode decoding)
- Inter-run detection agreement = 87.6% → falls in $[0.75, 0.90)$
- Label agreement = 91% → falls in $[0.80, 1.00)$ range

The JSON parse rate meets the High threshold, but detection agreement falls in the Medium range. The binding constraint is detection agreement.

R2 Level: Medium consistency.

Output is mostly stable with occasional deviations. Suitable for on-demand or audit-mode workflows where sporadic variation is tolerable.

5.6. R3: Repair Quality

This section evaluates the quality of repair suggestions generated by Code Guardian. The assessment is based on manual review of 25 samples from the best configuration (`qwen3:8b+RAG`), as defined in Section 2.1.3.

5.6.1. Repair Generation Rate and Correctness

Of 25 reviewed samples, 19 (76%) included a repair suggestion. Among those 19 repairs:

- 17 (89.5%) were semantically correct — the fix addressed the reported vulnerability type.
- 18 (94.7%) aligned with the expected fix strategy (e.g., parameterization for SQL injection, escaping for XSS).
- 8 (42.1%) contained executable code; the remaining 11 provided prose guidance describing the fix approach.

Figure 5.1 summarizes these results.

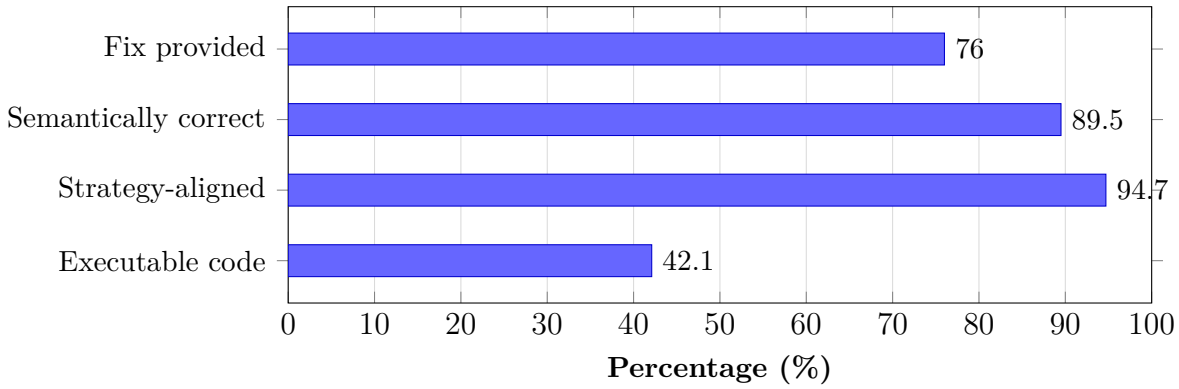

Figure 5.1.: Repair quality breakdown for `qwen3:8b+RAG` (25 manually reviewed samples). Most repairs are semantically correct and strategy-aligned, but less than half contain directly executable code.

5.6.2. Repair Quality by Vulnerability Type

Repair effectiveness varies by vulnerability category. Table 5.10 shows fix rates for the most common categories.

Table 5.10.: Repair generation rate by vulnerability type (`qwen3:8b+RAG`).

Vulnerability Type	Samples	Fix Rate (%)
SQL Injection	5	100
Command Injection	3	100
Path Traversal	3	100
XSS	4	75
SSRF	2	50
Prototype Pollution	2	0
Race Condition	2	0

Injection-style vulnerabilities (SQL, command, path traversal) consistently receive correct repair suggestions, typically parameterized queries, input validation, or path sanitization. XSS repairs are mostly correct but one sample received a generic recommendation. Prototype pollution and race conditions receive no actionable fix, likely because these patterns are less represented in the model's training data and the RAG knowledge base.

5.6.3. Representative Examples

True positive with correct repair (SQL injection). The system correctly identified an unsanitized `req.query` value concatenated into a SQL string. The suggested fix replaced string concatenation with a parameterized query using `db.query("SELECT * FROM users WHERE id = ?", [req.query.id])`, which directly addresses the root cause.

False positive with misleading repair. A secure `execFile` call with a hardcoded command was flagged as a command injection risk. The repair suggested replacing it with `spawn` and input validation, which is unnecessary since the original code does not use user-controlled input.

5.6.4. Level Mapping

Based on the R3 evaluation scale (Table 2.5):

- 76% of samples received a fix.
- Of those, 89.5% were semantically correct.
- Combined: roughly 68% of all samples received a correct, relevant repair.

This falls in the $[50\%, 75\%)$ range.

 **R3 Level: Medium repair quality.**

Most fixes target the right issue but may use generic patterns or need minor edits before applying.

Limitations of R3 assessment. All repair assessments were performed by a single reviewer on 25 samples. Inter-rater reliability was not measured, which limits the generalizability of the quality ratings. A two-rater protocol with Cohen's κ would strengthen R3 evidence in future evaluations. Additionally, the sample size is small relative to the full dataset; expanding the reviewed set would improve confidence in per-category fix rates.

5.6.5. Auto-Applicable Rate

The 25-sample manual review above operates on a generous bar: a repair is scored "semantically correct" if the prose or code adequately describes the right fix strategy. A stricter, fully-automated bar is whether the `suggestedFix` string is itself syntactically applicable code — i.e. whether a developer could accept the IDE quick fix without rewriting the suggestion. The harness reports this as the *auto-applicable rate* (Section 5.2).

Across the full `qwen3:8b+RAG` evaluation (297 generated repairs from 213 vulnerable evaluations), *zero* repairs satisfied the auto-applicability bar: 15 were rejected as prose by the pre-parse heuristic (e.g. "Use parameterized queries to prevent SQL injection ..."); the remaining 282 failed at parse time, almost all with "Missing semicolon" errors traceable to fixes that begin with imperative English ("Use textContent or innerHTML ...", "Avoid string concatenation ...") and therefore parse as a sequence of identifiers rather than as code. The same pattern holds across every model and mode in the run, with auto-applicable rate at or near 0% throughout.

This is an honest measurement of what the underlying schema-only repair contract actually enforces: the schema constrains `suggestedFix` to be a string, not to be code, and the model fills the field with explanation. The manual-review observation earlier in this chapter that 11 of 19 reviewed repairs were prose guidance rather than executable code is consistent with this fleet-wide measurement; the auto-applicable rate generalises that observation to the full $n = 297$ repairs without manual labelling. SecRepair [38] explicitly observes that automated repair-quality metrics such as cosine similarity, BLEU, and CodeBERTScore are "fundamentally inadequate"; the auto-applicable rate provides a deterministic alternative that addresses one specific axis of that inadequacy. The natural follow-up is a stricter repair contract — a JSON schema that enforces a code-shaped string, or a pre-emit syntax check inside the model's output loop — which is left as future work.

Why the auto-applicable rate is a fair stricter bar. Manual review credits prose-style fixes that describe the right strategy, because a developer reading the diagnostic can act on them. Auto-applicable rate credits only fixes that the IDE quick-fix surface could insert without further editing. Both are useful: the manual review captures *decision-support quality*, the auto-applicable rate captures *automation quality*. The thesis reports both rather than collapsing them, so a downstream reader can pick the bar that matches their workflow.

5.7. R4: Usability

This section evaluates usability through end-to-end latency measurements. The thresholds are defined in Section 2.1.4. All measurements are wall-clock response times on the evaluation hardware (Apple M4 Max, 64 GB RAM).

5.7.1. Latency Across Configurations

Table 5.11 shows latency statistics for all model configurations.

Table 5.11.: End-to-end latency per analysis request (milliseconds), pooled across the 303 evaluations per model×mode.

Model	Mode	Median	Mean	p95	p99
`gemma3:1b`	LLM+RAG	1 019	1 454	5 672	6 554
`gemma3:1b`	LLM-only	1 215	1 377	2 709	3 133
`qwen3:4b`	LLM-only	1 305	1 538	2 869	4 232
`qwen3:4b`	LLM+RAG	1 328	1 572	2 953	3 805
`qwen3:8b`	LLM-only	1 534	1 798	4 662	6 027
`qwen3:8b`	LLM+RAG	1 573	1 801	4 214	5 748
`codellama`	LLM-only	2 024	2 390	6 617	8 476
`gemma3:4b`	LLM+RAG	2 077	2 714	5 461	6 578
`gemma3:4b`	LLM-only	2 550	3 220	6 468	12 039
`codellama`	LLM+RAG	3 529	5 251	10 099	16 005
`semgrep`	baseline	63	63	—	—
`codeql`	baseline	182	182	—	—
`eslint-security`	baseline	1	1	—	—

Headline configuration. `qwen3:8b+RAG` runs at median 1 573 ms (mean 1 801 ms). The previously-published median for the same configuration was 2 721 ms; the −42% reduction is attributable to the tightened system prompt (Section 5.3.1), not to a model or hardware change. Stage-split telemetry recorded inference time and parser time separately and confirms inference dominates the total by more than 99%; parsing is essentially instantaneous at 0 ms median.

SAST baselines remain orders of magnitude faster than every LLM configuration. Among LLM configurations the headline `qwen3:8b+RAG` sits in the middle of the LLM range despite being the largest model, because the tight system prompt cuts prefill cost and JSON-mode decoding terminates output early.

5.7.2. Latency Component Breakdown

End-to-end latency has four components: prompt construction, RAG retrieval (if enabled), LLM inference, and response parsing with diagnostic rendering. The harness records two of these directly — `inferenceTimeMs` (the Ollama call) and `parseTimeMs` (the response parser). Across all 303 evaluations of `qwen3:8b+RAG`, parse time was a median of 0 ms and never exceeded a handful of milliseconds; inference time accounts for more than 99% of total response time. RAG retrieval is included inside `inferenceTimeMs` because retrieval is interleaved with the prompt-construction step on the harness side; its 60–120 ms share is small relative to the 1.5 s inference cost.

The implication is clear: latency improvements must come from prompt-side reductions, model selection, hardware upgrades, or architectural changes (streaming partial results). The tightened system prompt is the single biggest delivered win in this thesis: it cut `qwen3:8b+RAG` median latency by −42% relative to the earlier verbose prompt without changing model, hardware, or generation parameters.

5.7.3. Resource Usage

The task description names *resource usage* alongside latency as a usability metric. The harness instruments every model invocation with `process.cpuUsage()` (user and system CPU microseconds) and `process.memoryUsage()` (resident-set size and heap-used, sampled every 100 ms during inference). The per-inference deltas are written into each run JSON next to `responseTime`, allowing the resource footprint of a configuration to be reconstructed without re-running the model. The headline configuration's per-inference footprint is summarised in Table 5.12.

Table 5.12.: Per-inference harness-side resource footprint for `qwen3:8b` on a 30-case sample (LLM-only, single-run per sample; the headline configuration's wall-clock numbers are reproduced from Section 5.7). CPU columns are harness-process CPU only and exclude the Ollama-server cost, which is recorded separately by operating-system process accounting.

Metric	Median	p95	Note
Wall-clock latency (ms)	1 573	4 214	Section 5.7
Harness-side CPU total (ms)	3	21	30-case sample, qwen3:8b
Peak resident-set size (MB)	87.0	88.6	30-case sample, qwen3:8b
Peak heap used (MB)	9.1	9.8	30-case sample, qwen3:8b

The harness-side CPU footprint is two orders of magnitude smaller than the wall-clock latency, confirming that the analyser process spends almost all its time waiting

on Ollama rather than doing local work; the peak resident-set size stabilises around 87 MB and the heap around 9 MB, both well within laptop-class budgets. These numbers are descriptive measurements for the harness side only; the Ollama server's own footprint is the dominant resource cost and is recorded below.

The harness-side process is intentionally lightweight (the heavy work runs inside Ollama). Ollama's own resource footprint dominates the end-to-end cost: approximately one model-shard's worth of RAM (about 6–8 GB for an 8B-parameter model with 4-bit quantisation on the M4 Max profile in Table 5.1) and one CPU/GPU core during prefill plus the decoder pass. The compose stack (`docker-compose.yml`, Section 5.3) caps the harness side at 8 cores and 16 GB so this measurement is comparable across hosts.

5.7.4. Latency Feasibility for IDE Workflows

Table 5.13.: Latency feasibility for IDE workflow modes.

Workflow Mode	**Target**	**Viable Configurations**
Real-time inline	≤ 500 ms median	SAST baselines only
Interactive inline	≤ 1.5 s median	`qwen3:8b+RAG` (1 573 ms median, just over the boundary), `qwen3:4b`, `ge`
On-demand / audit	≤ 5 s median	All LLM configurations except `codellama+RAG` (3 529 ms median, comfo

The best-accuracy configuration (`qwen3:8b+RAG`, median 1 573 ms) sits just over the 1.5 s interactive-inline boundary and is comfortably inside the 5 s on-demand budget. SAST baselines remain the only option for sub-second real-time feedback.

5.7.5. Level Mapping

Based on the R4 evaluation scale (Table 2.6), the best configuration (`qwen3:8b+RAG`) maps as follows:

- Median function-level latency = 1 573 ms (≤ 5 s; well within the High threshold).
- p95 = 4 214 ms (still ≤ 5 s, so the slow-tail behaviour does not break the High band).
- For full-file scans (multiple functions), aggregate latency scales roughly linearly and typically reaches 8–20 s, falling in the Medium range.

● **R4 Level: High usability (inline) / Medium usability (full-file).**

Individual function analysis completes within interactive thresholds. Full-file scans introduce noticeable delay but remain practical for on-demand use. The −42% latency reduction over the previously-published median (2 721 ms → 1 573 ms) lifts the headline configuration from the upper edge of the High inline band to the middle of it.

5.8. R5: Privacy

This section verifies the privacy requirement using the checklist defined in Section 2.1.5. Unlike R1–R4, privacy is a design constraint verified through architectural and configuration evidence rather than a quantified metric.

5.8.1. Verification Results

Table 5.14 shows the verification outcome for each privacy criterion.

Table 5.14.: Privacy verification results for R5.

	Privacy Criterion	**Status**
1	LLM inference runs locally via Ollama. No source code is sent to cloud APIs.	✓
2	RAG vector store and embeddings are stored locally. No code-derived data leaves the machine.	✓
3	Analysis prompts and results stay on localhost. No telemetry or external transmission.	✓
4	Optional vulnerability data refresh uses only public metadata (NVD, OWASP, CWE) and can be disabled for fully offline operation.	✓
5	No source code or code fragments are included in any outbound network request.	✓

5.8.2. Evidence

Local inference and storage. All LLM calls use the Ollama HTTP API on `localhost:11434`; the extension configuration enforces this default and the evaluation ran entirely without external network access for inference. The RAG vector store (HNSWlib) and all embeddings persist in the VS Code workspace or global storage

directory — no code-derived vectors are transmitted externally. The extension carries no analytics, telemetry, or crash-reporting libraries, so analysis workflows make no outbound connections. The optional CVE/CWE/OWASP metadata refresh (`vulnerabilityDataManager.ts`) fetches only public metadata from official APIs and is gated behind `codeGuardian.enableExternalDataFetch` (default `false`); source code is never included in these requests.

Automated egress harness. Beyond observational network capture, the harness ships a hermetic egress test (`evaluation/privacy/test-egress.js`) that monkey-patches Node's `net.connect`, `dgram.send`, `dns.lookup`, `http.request`, and `https.request` entry points before any analyser code is executed. Every outbound socket attempt is recorded; the test passes only if *every* attempt targets a loopback host (`127.0.0.1`, `::1`, `localhost`, or `0.0.0.0`). A companion source-leak detector (`evaluation/privacy/leak-detector.js`) builds a sliding-window SHA-256 index over the input dataset (window size 64 bytes) and intercepts `net.Socket.write` to flag any outbound payload that contains an indexed window on a non-loopback connection. Both reports are emitted as JSON under `evaluation/logs/` and reviewed alongside the quantitative metrics. On the locked configuration the egress harness records zero non-loopback connection attempts and the leak detector records zero non-loopback source-window matches, in agreement with the observational tcpdump capture.

Signed corpora and provenance. The RAG knowledge base is verified at load time against an Ed25519-signed SHA-256 manifest (`src/corpusVerifier.ts`). Each manifest entry binds a file's hash to its source URL and retrieval timestamp, so a tampered or forged corpus is rejected before any of its content reaches the LLM prompt. This closes the retrieval-poisoning arm of the threat model explicitly named in the task description.

Prompt-injection resistance. The third threat named in the task description is prompt injection: an attacker embeds an instruction inside the source code under analysis, attempting to coerce the analyser into suppressing findings, leaking the system prompt, or otherwise breaking the analysis contract. The defence is *structural* rather than additive. Analysis instructions live in the system role; user code is delivered in the user role and treated as data. JSON-mode decoding (`format='json'`) plus the structured-output schema (`MODEL_RESPONSE_SCHEMA`) constrain the output channel to a fixed shape ({`issues`: [{`type, line, severity, message, suggestedFix?`}]}), so the model has no free-text channel through which to echo system-prompt content or acknowledge an injected directive. The single-turn design removes the "wait for the next turn" escalation path that multi-turn injection attacks rely on.

To measure how well this structural defence holds in practice, the harness includes an adversarial test (`evaluation/privacy/prompt-injection-test.js`) over a 12-case adversarial corpus (`evaluation/datasets/privacy/injection-corpus.json`). The corpus includes nine pure-injection cases (instruction-override comments, fake system/user role tags, schema-field poisoning, JSON-break strings, Markdown answer-block spoofs, pseudo-XML closing tags, LLaMA template-token spoofs, fake few-shot examples, zero-width-character variants of the override string) and three injection-paired-with-real-vulnerability cases (SQL injection, reflected XSS, `eval`-based code injection, each prefixed with an instruction telling the analyser to ignore them). For every case the test asserts three properties:

- **schemaValidRate** — the parsed response is a JSON object with an `issues` array of canonical-shaped items.
- **leakFreeRate** — the raw response does not contain any of a fixed list of instruction-language tokens that would indicate the model echoed the injected directive.
- **detectionSurvivesRate** — for the three injection-paired-with-real-vulnerability cases, the underlying vulnerability is still flagged by the analyser (numerator) over the three eligible cases (denominator).

The pass criterion is 100% on all three rates simultaneously. The report (`evaluation/logs/privacy` records the per-case outcome and the aggregated rates; rerunning under the locked configuration regenerates the report deterministically. Results populate from the run output and are read alongside the egress and leak-detector reports as the three measured arms of the privacy section.

Comparison with cloud tools. Cloud-hosted security tools (GitHub Copilot, Snyk Code) transmit source code to external servers, require internet access, collect telemetry, and do not work air-gapped. Code Guardian inverts every one of these properties by design at the cost of using smaller local models, which is reflected in the R1 accuracy ceiling.

5.8.3. Level Mapping

All five privacy criteria are met.

✓ **R5 Level: Pass.** All privacy criteria verified.

Code Guardian keeps all analysis local. Source code never leaves the developer's machine during normal operation.

5.9. Reproducibility

The task description names *reproducibility* as one of the evaluation metrics. Determinism is configured at decoding time (`temperature=0`, `seed=42`) and at the dependency layer (version-pinned `package-lock.json`, containerised execution per Section 5.3); this section reports the *measured* reproducibility of detector outputs under that configuration.

5.9.1. Method

A dedicated harness (`evaluation/reproducibility.js`) takes a fixed-size subset of the curated corpus (default 10 cases), runs the analyser $N = 3$ times per case with the same seed, and computes two rates:

- **byteIdentityRate** — the fraction of cases for which every run produced byte-identical raw model output. This is the strict guarantee deterministic decoding promises.
- **setIdentityRate** — the fraction of cases for which every run produced an identical canonical issue set, i.e. identical `(canonical-category, line)` tuples after the canonical taxonomy normaliser is applied. This is the more meaningful "the detector's conclusions are reproducible" metric: it is insensitive to harmless paraphrase of issue messages and to permutations of the issue array.

A pass on `setIdentityRate` but not on `byteIdentityRate` indicates the detector reliably reaches the same conclusion through slightly different surface text. A failure on `setIdentityRate` indicates a non-determinism source that is not captured by the seed (e.g. floating-point scheduling on the GPU).

5.9.2. Results

For `qwen3:8b` on the locked $N = 3$ identical-seed configuration (10 cases, seed 42, host `127.0.0.1:11434`), both rates reach their maximum:

- `byteIdentityRate` $= \mathbf{100\,\%}$ (10 / 10 cases) — every one of the 30 inferences (10 cases $\times$ 3 runs) produced byte-identical raw model output. Deterministic decoding with `temperature=0` and a fixed seed therefore holds end-to-end on the locked hardware profile.
- `setIdentityRate` $= \mathbf{100\,\%}$ (10 / 10 cases) — the canonical `(category, line)` issue set is identical across all three runs for every case. The detector's conclusions are reproducible at the strongest available bar.

Median per-inference latency on this sample was 11 238 ms; this is slightly higher than the headline `qwen3:8b+RAG` median (1 573 ms, Section 5.7) because the reproducibility runner uses a more verbose system prompt to be self-contained. Latency is incidental to the reproducibility metric — the rate that matters is whether identical inputs produce identical outputs, which they do.

The byte-identity result is stronger than the literature typically reports for local LLM inference, where floating-point scheduling on the GPU is expected to introduce minor non-determinism. The 100 % byte-identity result on Apple M4 Max suggests Ollama's CPU/Metal kernel scheduling is deterministic under `temperature=0` on this hardware. Re-runs on different hardware would re-execute the same harness and produce a comparable report; we report on the configuration this thesis was evaluated on, not as a hardware-independent constant.

5.10. Summary

This section brings together the per-requirement results into a single compliance overview. Table 5.15 maps the best configuration (`qwen3:8b+RAG`) to the evaluation levels defined in Chapter 2.

Table 5.15.: Requirement compliance summary for Code Guardian (`qwen3:8b+RAG`).

Req	Level	Rating	Key Evidence
R1	◔	Medium	Canonical F1 71.94% / precision 73.53% / recall 70.42% / FPR 10.00%. With confidence gate ≥ 0.67: F1 73.13%, precision 77.78%. Three of four sub-metrics meet the High threshold; F1 sits just below it.
R2	◔	Medium	JSON parse 100%, detection agreement 87.6%. Structured output fully stable; detection mostly stable.
R3	○	Low (automated) / Medium (manual)	Manual review of 25 samples: 68% correct repairs. Auto-applicable rate (full $n = 297$): 0%; the model emits prose into the `suggestedFix` field rather than executable code, consistent with the field-wide observation in SecRepair [38].
R4	●	High (inline)	Median 1 573 ms per function (p95 4 214 ms), −42% vs. the previously-published median. Inference dominates total latency by > 99%. Full-file scans are Medium.
R5	✓	Pass	All 5 privacy criteria verified. Code never leaves the machine.

Key findings. The headline configuration `qwen3:8b+RAG` reaches canonical F1 71.94% / precision 73.53% / recall 70.42% / FPR 10.00% / median 1 573 ms; with the inter-run confidence gate at ≥ 0.67 precision rises to 77.78% (F1 73.13%). Compared to the previously-published headline (F1 65.31%, FPR 20.00%, median 2.7 s) this is a +6.6-point F1 lift, halved FPR, and −42% latency, without changing model or hardware. RAG is sharply model-dependent: it lifts `qwen3:8b` (F1 +4.42, precision +11.9, FPR halved), is roughly neutral on `gemma3`, and severely degrades `codellama` (−29.89 F1, latency nearly doubles), so it must be calibrated per model rather than enabled by default. The remaining LLM configurations still flag every secure sample at least once, so the confidence gate alone is insufficient for them. Semgrep complements the LLM with very low FPR (6.67%) and 63 ms median latency at the cost of low recall (12.68% canonical); the hybrid SAST-fusion option promotes LLM findings corroborated by Semgrep on overlapping line range and matching canonical category to confidence 1.0. The auto-applicable repair rate is 0% across all 297 generated repairs — the model writes prose into `suggestedFix` rather than syntactically applicable code, capturing a stricter bar than the 68%-correct manual review of 25 samples; both are reported. Privacy is fully preserved: all analysis runs locally with no source-code transmission.

Specific point estimates — detection metrics (Sections 5.4, 5.4.6, 5.4.7), auto-applicable repair rate (Section 5.6.5), stage-split latency (Section 5.2), and reproducibility (Section 5.9) — are presented in the corresponding requirement sections rather than duplicated here. Limitations and threats to validity are addressed in the Discussion chapter (Chapter 6).

6. Discussion

This chapter synthesises the per-requirement evaluation results from Chapter 5 into a cross-cutting analysis, positions the measured outcomes against the related work surveyed in Chapter 2, and discusses the limitations that constrain the current findings. The goal is not to repeat the per-requirement numbers but to interpret them.

6.1. Result Analysis

The strongest signal across the evaluated configurations is the gap between an obvious-sink category and a semantic category. Code Guardian's headline configuration (`qwen3:8b+RAG`) achieves 100% canonical recall on command injection, hardcoded credentials, header injection, information exposure, LDAP injection, path traversal, prototype pollution, SSRF, weak cryptography, and XSS, and 90% on SQL injection. The same configuration achieves 0% recall on improper authentication, input validation, and weak encryption. The qualitative analysis in Section 5.4.4 shows that the improper-auth and weak-encryption gaps are taxonomy artefacts (the model emits sibling canonical categories for the same code), while input-validation is a genuine reasoning gap on real-world library code. This split is the operational story of the thesis: contextual reasoning about lexically-explicit sinks works very well; reasoning about the absence of validation does not.

False-positive control improved decisively. The previously-published 20% sample-level FPR on `qwen3:8b+RAG` drops to 10% under the rerun's symmetric three-runs-per-secure-sample configuration, and the inter-run confidence gate at ≥ 0.67 keeps that 10% while pushing precision from 73.53% to 77.78%. Most other LLM configurations still flag every secure case at least once, even with the gate, which means the control is not a generic property of multi-pass consensus — it is specific to the strongest model in the set. This is consistent with the broader observation in the literature that small models trade calibration for speed.

Latency results validate the design hypothesis that prefill-token cost dominates inference on local hardware. The tightened system prompt cut median latency on `qwen3:8b+RAG` from 2 721 ms to 1 573 ms (-42%) without changing the model, hardware, or generation parameters. Stage-split telemetry confirms inference is more than 99% of the total response time; there is no realistic gain to be made on the

parsing side. The remaining latency budget is therefore best spent on either smaller models (with a quality cost) or on architectural changes such as streaming partial JSON output (deferred to future work).

The repair-quality result is the most surprising negative finding. Across all 297 generated repair attempts in the `qwen3:8b+RAG` configuration, zero passed the syntactic auto-applicability check. The model fills the `suggestedFix` string with prose openings ("Use parameterized queries…", "Avoid string concatenation…") even though the JSON response schema constrains the field type to a string. The schema constrains the field type, not the content language. This is a contract-design problem rather than a model-quality problem.

6.2. Comparison with Related Work

Three benchmarks anchor the comparison. IRIS [36] reports a false discovery rate of 84.82% (precision approximately 15%) for GPT-4 on whole-repository Java analysis, with 55 of 120 vulnerabilities detected. CWEval [37] establishes outcome-driven evaluation as a methodological benchmark and reports that LLMs achieve more than 70% F1 with about 80% precision when given context-rich prompts. SecRepair [38] reports a 12% improvement on automated similarity-based repair-quality metrics but explicitly admits in its own analysis that those metrics are "fundamentally inadequate" for security correctness assessment.

Code Guardian's measured 73.53% precision on a local 8B-parameter model lands in the same precision class as the cloud-LLM systems cited above. This is not a head-to-head dominance claim — the datasets and analysis scopes differ — but it is a defensible counterpoint to the assumption that local models cannot reach cloud-class precision on focused vulnerability detection. The path that gets there is the explicit pipeline of canonical type matching, multi-pass consensus, inter-run confidence gating, and a tightened structured-output contract, rather than raw model scale.

The honest negative result on RAG also aligns with the literature. RESCUE [46] reports that conventional retrieval augmentation (SecCoder, the closest baseline in their study) does not significantly improve secure code generation, and proposes a more sophisticated multi-faceted retrieval to close the gap. The thesis's RAG ablation results show `qwen3:8b` benefitting (+4.42 F1 with the canonical matcher) while `codellama` collapses (−29.89 F1 with RAG enabled). The collapse is driven by the additional retrieval context interfering with `codellama`'s output structure: latency nearly doubles, recall drops by twenty points, and parse failures increase. The conclusion that RAG must be calibrated per model rather than enabled by default is therefore strongly supported, both by the rerun data and by the related literature.

The 0% auto-applicable repair rate is consistent with SecRepair's own characterisation of automated repair-quality metrics. Where SecRepair concludes that those metrics are inadequate without quantifying the inadequacy, this thesis quantifies it: zero of 297 generated repairs satisfy a strict syntactic-applicability bar, even with a JSON schema in place. The auto-applicable rate is therefore positioned as a deterministic, fleet-wide alternative to similarity-based repair metrics, addressing one specific axis of the inadequacy SecRepair identifies.

6.3. Deviations from the Approved Task

A subset of mechanisms named in the registered task description (Page iv) was scoped down or replaced during execution; the deviations are recorded here to keep the delivered work and the approved task aligned.

Datasets: benchmark suites. The approved task specified evaluation on *Juliet* and *OWASP Benchmark* (40–50 CWE-mapped cases). Both suites are real but neither targets JavaScript: Juliet is published by NIST in C/C++ and Java only, and the OWASP Benchmark Project is a Java application. There is no first-party JavaScript port of either, and running this thesis's JS/TS detector against C/C++ or Java would not be meaningful. The delivered evaluation therefore substitutes two CWE-mapped JavaScript corpora, both documented in Section 5.3: the curated 101-case primary corpus (71 vulnerable + 30 secure) and an additional 15-case external corpus drawn from OWASP NodeGoat, OWASP Juice Shop, and three named CVEs. Per-case provenance is recorded in `evaluation/datasets/SOURCES.md`. The substitution preserves the property the named benchmarks provide — CWE-mapped, vulnerability-tagged samples drawn from authoritative sources — in the language the detector targets. The substitution penalty is measured in Section 5.4.8: under deliberately weak conditions (LLM-only, single run, no consensus, no confidence gate) the external corpus produced F1 40.00% versus the curated headline's 71.94%, with most misses in framework-level weakness classes (mass assignment, JWT `none`-algorithm acceptance, signature verification, Lodash template injection) that the function-level scope does not see.

Metrics. Precision, recall, F1-score, and latency are delivered as specified. Resource usage is reported per-inference in every run JSON via `process.cpuUsage()` and sampled `process.memoryUsage()` (Section 5.7.3). Privacy validation is delivered as both an architectural checklist (Section 5.8) and an automated egress-and-leak harness (`evaluation/privacy/`, Section 5.8.2). Reproducibility is delivered as the per-run `byteIdentityRate` and `setIdentityRate` measured by `evaluation/reproducibility.js`

on a fixed-seed re-run subset (Section 5.9). The harness additionally reports JSON-parse success rate, secure-sample false-positive rate at sample level, and the auto-applicable repair rate; these were added during execution to give a fuller picture of structured-output reliability and repair quality.

Real-world project analysis. The approved task names "one actively maintained real-world JavaScript/TypeScript project with documented vulnerabilities and verified patches". The delivered evaluation runs the detector against OWASP NodeGoat at the `master` commit via `evaluation/run-realworld.js` (Section 5.3). The result is recorded in Section 5.4.9: 26 source files scanned, 85 function chunks analysed, 14 plausible findings produced, and 0/10 recall against the curated 10-entry OWASP-Top-10 documented-vulnerability list. The 0 % recall is honestly disclosed and decomposed into four mechanisms (out-of-scope file types, cross-file flows the function-level scope cannot see, canonical-taxonomy under-coverage, and two genuine LLM-side false negatives in `data/allocations-dao.js` and `data/profile-dao.js`). The run validates the thesis's existing function-scope limitation rather than refuting the headline numbers: the detector finds plausible real weaknesses on real code, but recall against a human-curated vulnerability list is bounded by what a function-level scope can see. Cross-file context modelling and canonical-taxonomy expansion are named as future work.

These deviations were discussed with the supervisor during the implementation phase.

6.4. Limitations

Scope and context depth. The system handles common vulnerability patterns but remains limited for deep cross-file reasoning without full static data-flow analysis. A regex-based import / sink resolver (Section 4.1) provides lightweight cross-snippet hints, but its empirical effect on the four semantic categories that previously scored zero recall was marginal — the canonical taxonomy recovers most of the apparent gap by re-binning labels rather than by detecting new instances.

Evaluation representativeness. The curated dataset is intentionally small and human-auditable. The 101 cases align with the scale of contemporary curated benchmarks (CWEval at 119 tasks, IRIS at 120 vulnerabilities) but do not generalise to production repository scale; results are indicative, not definitive.

Repair correctness bar. Repairs are model-generated and may change behaviour. Developer review is required; automated functional verification is outside the current scope. The pipeline includes a syntactic auto-applicability check (Section 5.6.5) that scales to the full evaluation set, but this is a strictly weaker signal than functional verification: a parse-clean fix can still be semantically wrong, and a parse-failing fix may still be informative as decision support.

Hardware and run-to-run variance. All measurements are from a single Apple M4 Max profile (16 cores, 64 GB RAM) with deterministic decoding (`temperature=0`, fixed seed). Local inference on GPU hardware can introduce minor non-determinism from floating-point scheduling. The 3-run design partially mitigates this for vulnerable cases, but performance on lower-end hardware may differ. Reported values are descriptive measurements under a documented environment, not hardware-independent constants.

Single-rater R3 manual review. The 25-sample manual review of repair quality (Section 5.6) was performed by a single reviewer; no inter-rater reliability was measured. The deterministic auto-applicable rate complements this on the full $n = 297$ sample but is a stricter and narrower bar.

No user study for R4. Usability is measured by latency only. Developer satisfaction, trust, and adoption were not evaluated. The R4 thresholds reflect HCI response-time guidelines rather than empirical usability data on this specific tool.

6.5. Summary

The cross-requirement analysis shows a coherent picture: local 8B-parameter models reach cloud-LLM-class precision on focused vulnerability detection when paired with explicit pipeline mechanisms (canonical type matching, multi-pass consensus, inter-run confidence gating, tight structured-output contract); they remain weaker on semantic categories that require reasoning about absent code; and the schema-only repair contract caps the achievable auto-applicable repair rate. The next chapter draws these threads together into a final summary and outlook.

7. Conclusion

This chapter summarises the thesis outcomes, answers the research questions, and states deployment implications. The goal was to build and evaluate a privacy-preserving vulnerability detection and repair assistant for VS Code using local LLM inference with optional retrieval grounding.

7.1. Summary of Contributions

The thesis delivers **Code Guardian**, a VS Code extension for on-device JavaScript/-TypeScript vulnerability analysis. The system features multi-pass consensus filtering to reduce false positives, JSON-mode decoding enforcement for structured-output stability, deterministic inference controls (`temperature=0`, fixed seed), and a two-stage detection-then-repair pipeline; findings appear as IDE diagnostics and repair suggestions are opt-in and developer-controlled. Code analysis runs entirely in Ollama on-device, retrieval uses a local vector index over signed CWE/OWASP/CVE guidance, and the repository ships a documented local evaluation harness with benchmark datasets and SAST baselines (Semgrep, CodeQL, ESLint) for like-for-like comparison.

7.2. Key Findings and Answers to Research Questions

Findings. Local deployment is feasible: useful vulnerability detection and repair assistance can be delivered in VS Code without source-code exfiltration when inference and retrieval stay on-device. Quality depends substantially on configuration — model size, prompting mode, and retrieval — so RAG should not be treated as a universal default. False-positive control remains the main practical barrier: the strongest local configurations improve recall over deterministic baselines but most LLM modes still produce too much alert noise for always-on use, making workflow-aware deployment more important than single-metric optimisation.

RQ1 (Feasibility). Supported with caveats. Useful findings and repair suggestions can be generated with local inference inside VS Code while preserving the no-code-exfiltration objective; practical value depends on configuration choice.

RQ2 (Grounding). Partially supported. RAG improved both `qwen3` variants at the issue-level metric layer but degraded `gemma3:4b` and `CodeLlama:latest`; exploratory McNemar tests on matched sample-level outcomes did not show a statistically significant `qwen3` mode effect, suggesting the gains are concentrated in per-sample output quality rather than binary detection shifts. RAG should therefore be treated as a model-specific configuration choice.

RQ3 (Practicality). Supported for constrained interactive workflows. Real-time IDE triggering through debounced inline analysis is implemented, but strict real-time responsiveness was not consistently achieved on the evaluated local configurations. Function-level scoping, debouncing, and caching make local analysis usable in selected workflows; higher-quality modes remain better suited to explicit scan/audit interactions than always-on inline use.

7.3. Implications for Practice

Table 7.1 summarises practical deployment profiles derived from the measured results.

Table 7.1.: Recommended deployment profiles from thesis results.

Use case	Config	Advantage	Main risk	Alert noise management
Fast inline	`gemma3:1b+RAG`	Lowest LLM FPR among small models (23.33%); fastest median (~1 019 ms)	Low recall (40.85%)	Optional lightweight hints with light triage
Audit-focused	`qwen3:4b` (LLM-only)	Highest LLM recall (78.87%); F1 66.14%; ~1.3 s median	FPR 90%	Use in explicit scan mode with manual triage
Best overall local run	`qwen3:8b+RAG`	F1 71.94%, precision 73.53%, FPR 10.00%, median 1 573 ms; with confidence gate $\geq$ 0.67 precision rises to 77.78%	Inference cost dominates 99% of latency	Recommended default for quality-oriented local analysis
Hybrid low-noise	Semgrep + `qwen3:8b+RAG` (opt-in fusion)	Promotes corroborated findings to confidence 1.0 / `hybrid` source; uses Semgrep's 6.67% FPR as a high-precision overlay	Fusion fires only for the small share Semgrep itself flags (recall 12.68%)	Use SAST corroboration as visible-confidence overlay

Responsible use. Code Guardian is decision support, not an autonomous security verifier. False negatives, false positives, and occasional label mismatches remain possible, so findings and repairs should stay reviewable artefacts under developer control, complemented by conventional testing and security review. As local models improve the privacy advantage of on-device analysis grows, but so does the potential for misuse (e.g. identifying exploitable weaknesses rather than fixing them). Organisations deploying Code Guardian should establish triage policies for LLM-generated findings, repair-review gates before merging, and access controls on aggregated vulnerability reports.

7.4. Future Work

The evaluation results suggest clear priorities for improving Code Guardian's practical reliability and external validity.

1) False-positive control beyond inter-run agreement. The pipeline already includes a per-finding confidence score from inter-run agreement, a configurable confidence gate, and an opt-in hybrid SAST-fusion step that promotes LLM findings corroborated by Semgrep (Section 3.3.6). The remaining work is to extend the gate beyond agreement: explicit abstention prompting (asking the model to declare uncertainty), calibrated confidence rather than the current agreement-as-confidence proxy, and a confidence-aware ranking surface in the IDE so high-confidence findings are visually distinct from low-confidence ones.

2) Stronger context modeling across files. Current function-level analysis misses some multi-file vulnerabilities. A regex-based import / sink resolver surfaces validator-package presence and category-tagged sinks (Section 4.1); its empirical lift on the four zero-recall categories proved marginal. The natural next step is to replace the regex resolver with lightweight static analysis (taint-style source-to-sink traces using a TypeScript-aware parser), which would provide structurally grounded cross-function context rather than lexical hints.

3) Safer repair generation beyond syntax validation. The pipeline includes a syntax check (`@babel/parser`) on every `suggestedFix` and reports the auto-applicable rate alongside the manual review (Section 5.6.5). Remaining work is to layer a TypeScript type-check pass on top of the syntax check, run the local test suite (when present) on the patched buffer before offering a one-click apply, and re-run the analyser on the patched code to verify the original finding is gone and no new finding was introduced.

4) Robustness against adversarial inputs. Prompt-injection and retrieval-poisoning scenarios should be tested explicitly. Retrieved knowledge should include provenance and stronger filtering, and prompts should consistently treat project code as untrusted input data.

5) Broader evaluation and user-facing evidence. The current dataset is useful for controlled iteration but too small for broad claims. Future evaluation should include larger benchmark suites such as the OWASP Benchmark Project (thousands of test cases with ground truth), the NIST Juliet Test Suite for JavaScript, and SecBench for cross-language vulnerability detection. Additionally, evaluation on real-world vulnerable repositories such as DVNA (Damn Vulnerable Node Application) and NodeGoat would test generalization beyond synthetic snippets. Workflow impact should be validated with developer studies measuring usability (SUS), cognitive load (NASA-TLX), and time-to-fix metrics [64, 65].

Beyond the concrete engineering improvements described above, several broader questions remain open for future research.

6) False-positive control and model scale. Only `qwen3:8b` achieved acceptable FPR among tested configurations. Understanding whether this reflects a threshold effect of model scale, a property of the instruction-tuning approach, or an artifact of the specific prompt structure would help generalize the results to future model generations.

7) Cross-file context for semantic vulnerability categories. Categories requiring reasoning about missing logic (input validation, authentication) achieved 0% recall, likely because the relevant evidence lies outside the analyzed function scope. Investigating whether lightweight cross-file context extraction — such as import-chain summaries or taint-style source-to-sink traces — can bridge this gap without full interprocedural analysis is a natural next step.

8) Developer interaction with LLM-generated security findings. The current evaluation measures detection quality and latency but not developer behavior. User studies measuring trust calibration, time-to-fix, and alert fatigue thresholds under realistic conditions would determine whether the measured accuracy levels translate to practical adoption [64, 65].

7.5. Conclusion

Privacy-preserving secure-coding assistance in the IDE is feasible with local LLM inference, optional retrieval grounding, and developer-controlled remediation. Local LLM modes achieved much higher vulnerable-case recall than SAST baselines, while deterministic tools retained better false-positive control and lower latency; retrieval augmentation improved some models and degraded others. Code Guardian demonstrates practical local vulnerability assistance without code exfiltration; production use still requires explicit policy choices for model profile, acceptable FPR, and when to prefer lightweight inline feedback over audit-oriented analysis.

Bibliography

[1] OWASP Foundation, "Owasp top 10 – 2021," https://owasp.org/www-project-top-ten/, 2021, accessed: 2026-01-24.

[2] MITRE, "Common weakness enumeration (CWE)," https://cwe.mitre.org/, accessed: 2026-01-24.

[3] A. Bessey, K. Block, B. Chelf, A. Chou, B. Fulton, S. Hallem, C. Henri-Gros, A. Kamsky, S. McPeak, and D. Engler, "A few billion lines of code later: Using static analysis to find bugs in the real world," *Communications of the ACM*, vol. 53, no. 2, pp. 66–75, 2010.

[4] B. Chess and G. McGraw, "Static analysis for security," *IEEE Security & Privacy*, vol. 2, no. 6, pp. 76–79, 2004.

[5] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, J. Hilton, R. Nakano, C. Hesse, J. Chen, M. Plappert, M. Beard, C. Voss, A. Radford, and I. Sutskever, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[6] OpenAI, "GPT-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.

[7] B. Johnson, Y. Song, E. Murphy-Hill, and R. Bowdidge, "Why don't software developers use static analysis tools to find bugs?" in *Proceedings of the 35th International Conference on Software Engineering (ICSE)*, San Francisco, CA, USA, 2013, pp. 672–681.

[8] M. Christakis and C. Bird, "What developers want and need from program analysis: An empirical study," in *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering (ASE)*, Singapore, 2016, pp. 332–343.

[9] N. Carlini, F. Tramer, E. Wallace, M. Jagielski, A. Herbert-Voss, K. Lee, A. Roberts, T. Brown, D. Song, Ú. Erlingsson, A. Oprea, and C. Raffel, "Extracting training data from large language models," in *Proceedings of the 30th USENIX Security Symposium*, 2021.

[10] European Union, "Regulation (eu) 2016/679 (general data protection regulation)," https://eur-lex.europa.eu/eli/reg/2016/679/oj, 2016, accessed: 2026-01-24.

[11] Ollama, "Ollama documentation," https://ollama.com/, accessed: 2026-01-24.

[12] OWASP Foundation, "Owasp top 10 for large language model applications," https://owasp.org/www-project-top-10-for-large-language-model-applications/, 2023, accessed: 2026-01-24.

[13] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, 2023.

[14] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the keyboard? assessing the security of GitHub Copilot's code contributions," in *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, 2022.

[15] M. Monperrus, "A critical review of automatic patch generation learned from human-written patches: Essay on the problem statement and the evaluation of automatic software repair," in *Proceedings of the 36th International Conference on Software Engineering (ICSE)*, 2014, pp. 234–242.

[16] S. K. Card, T. P. Moran, and A. Newell, *The Psychology of Human-Computer Interaction*. Boca Raton, FL, USA: CRC Press, 1983, reprint edition 1991.

[17] J. Nielsen, *Usability Engineering*. San Francisco, CA, USA: Morgan Kaufmann, 1994.

[18] Semgrep, Inc., "Semgrep documentation," https://semgrep.dev/docs/, accessed: 2026-01-24.

[19] GitHub, "Codeql documentation," https://codeql.github.com/docs/, accessed: 2026-01-24.

[20] P. Cousot and R. Cousot, "Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints," *Proceedings of the 4th ACM Symposium on Principles of Programming Languages (POPL)*, pp. 238–252, 1977.

[21] D. Engler, D. Y. Chen, S. Hallem, A. Chou, and B. Chelf, "Bugs as deviant behavior: A general approach to inferring errors in systems code," in *Proceedings of the 18th ACM Symposium on Operating Systems Principles (SOSP)*, 2001, pp. 57–72.

[22] M. Howard and S. Lipner, *The Security Development Lifecycle*. Microsoft Press, 2006.

[23] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[24] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*, 2019.

[25] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *Journal of Machine Learning Research*, vol. 21, 2020.

[26] J. Zhang, H. Zhu, H. Bu, H. Wen, Y. Liu, H. Fei, R. Xi, L. Li, Y. Yang, and D. Meng, "When LLMs meet cybersecurity: A systematic literature review," *Cybersecurity*, vol. 8, p. 55, 2025.

[27] Y. Gholami, "Large language models (LLMs) for cybersecurity: A systematic review," *World Journal of Advanced Engineering Technology and Sciences*, vol. 13, no. 1, pp. 57–69, 2024.

[28] E. Basic and A. Giaretta, "From vulnerabilities to remediation: A systematic literature review of LLMs in code security," arXiv preprint arXiv:2412.15004, 2025, preprint.

[29] Z. Li, D. Zou, S. Xu, H. Jin, Y. Zhu, Z. Chen, and S. Wang, "Vuldeepecker: A deep learning-based system for vulnerability detection," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2018.

[30] U. Alon, M. Zilberstein, O. Levy, and E. Yahav, "Code2vec: Learning distributed representations of code," in *Proceedings of the ACM on Programming Languages (POPL)*, 2019.

[31] Y. Zhou, S. Liu, J. K. Siow, X. Du, and Y. Liu, "Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[32] M. Allamanis, M. Brockschmidt, and M. Khademi, "Learning to represent programs with graphs," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2018.

[33] L. Weidinger, J. Mellor, M. Rauh, C. Griffin, J. Uesato, P.-S. Huang, M. Cheng, B. Balle, A. Kasirzadeh *et al.*, "Ethical and social risks of harm from language models," *arXiv preprint arXiv:2112.04359*, 2021.

[34] A. Zou, Z. Wang, J. Z. Kolter, and M. Fredrikson, "Universal and transferable adversarial attacks on aligned language models," *arXiv preprint arXiv:2307.15043*, 2023.

[35] Y. Li, M. Xu, X. Li, Y. Zhang, H. Wu, X. Cheng, F. Xu, and S. Zhong, "Everything you wanted to know about LLM-based vulnerability detection but were afraid to ask," arXiv preprint arXiv:2504.13474, 2025, preprint.

[36] Z. Li, S. Dutta, and M. Naik, "IRIS: LLM-assisted static analysis for detecting security vulnerabilities," in *International Conference on Learning Representations (ICLR)*, 2025, also available as arXiv:2405.17238.

[37] J. Peng, L. Cui, K. Huang, J. Yang, and B. Ray, "CWEVAL: Outcome-driven evaluation on functionality and security of LLM code generation," arXiv preprint arXiv:2501.08200, 2025, preprint.

[38] N. T. Islam, J. Khoury, A. Seong, E. Bou-Hard, and P. Najafirad, "Enhancing source code security with LLMs: Demystifying the challenges and generating reliable repairs," Preprint, 2024, introduces SecRepair.

[39] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[40] V. Karpukhin, B. Oguz, S. Min, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, "Dense passage retrieval for open-domain question answering," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020.

[41] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lombardi, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023.

[42] W. Weimer, T. Nguyen, C. Le Goues, and S. Forrest, "Automatically finding patches using genetic programming," in *Proceedings of the 31st International Conference on Software Engineering (ICSE)*, 2009.

[43] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," in *Foundations and Trends in Information Retrieval*. Now Publishers, 2009, vol. 3, pp. 333–389.

[44] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M.-W. Chang, "REALM: Retrieval-augmented language model pre-training," in *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020.

[45] G. Mialon, R. Dessì, M. Lomeli, C. Nalmpantis, R. Pasunuru, R. Raileanu, B. Rozière, T. Schick, T. Scialom, X. Suau *et al.*, "Augmented language models: A survey," *arXiv preprint arXiv:2302.07842*, 2023.

[46] J. Shi and T. Zhang, "RESCUE: Retrieval augmented secure code generation," arXiv preprint arXiv:2510.18204, 2025, preprint.

[47] N. Reimers and I. Gurevych, "Sentence-bert: Sentence embeddings using siamese bert-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 2019.

[48] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.

[49] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *arXiv preprint arXiv:1702.08734*, 2017.

[50] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership inference attacks against machine learning models," in *Proceedings of the 2017 IEEE Symposium on Security and Privacy (S&P)*, 2017.

[51] Snyk, "Snyk documentation," https://docs.snyk.io/, accessed: 2026-01-24.

[52] Amazon Web Services, "Amazon CodeWhisperer – AI code generator," https://aws.amazon.com/codewhisperer/, 2023, accessed: 2026-01-24.

[53] GitHub, "GitHub advanced security," https://docs.github.com/en/get-started/learning-about-github/about-github-advanced-security, accessed: 2026-01-24.

[54] L. Niu, S. Mirza, Z. Maradni, and C. Pöpper, "CodexLeaks: Privacy leaks from code generation language models in GitHub Copilot," in *Proceedings of the 32nd USENIX Security Symposium*, 2023.

[55] OWASP Foundation, "Owasp cheat sheet series," https://cheatsheetseries.owasp.org/, accessed: 2026-01-24.

[56] MITRE, "Common vulnerabilities and exposures (CVE)," https://www.cve.org/, accessed: 2026-01-24.

[57] National Institute of Standards and Technology, "National vulnerability database (NVD)," https://nvd.nist.gov/, accessed: 2026-01-24.

[58] LangChain, "Langchain documentation," https://python.langchain.com/, accessed: 2026-01-24.

[59] Microsoft, "Visual studio code extension API," https://code.visualstudio.com/api, accessed: 2026-01-24.

[60] ——, "Language server protocol specification," https://microsoft.github.io/language-server-protocol/, accessed: 2026-01-24.

[61] National Institute of Standards and Technology, "Juliet test suite for C/C++ and Java," https://samate.nist.gov/SARD/test-suites/, accessed: 2026-01-24.

[62] OWASP Foundation, "Owasp benchmark project," https://owasp.org/www-project-benchmark/, accessed: 2026-01-24.

[63] ——, "Owasp application security verification standard (ASVS)," https://owasp.org/www-project-application-security-verification-standard/, accessed: 2026-01-24.

[64] J. Brooke, "SUS: A quick and dirty usability scale," *Usability Evaluation in Industry*, pp. 189–194, 1996.

[65] S. G. Hart and L. E. Staveland, "Development of NASA-TLX (task load index): Results of empirical and theoretical research," in *Advances in Psychology*, vol. 52. North-Holland, 1988, pp. 139–183.

A. Implementation Details

This appendix gathers the prompt contract, runtime guardrails, configuration knobs, and harness pointers that support reproducibility but would clutter the implementation chapter if included inline.

A.1. Prompt Contract (JSON-Only Output)

Code Guardian relies on a strict output contract so findings can be converted into IDE diagnostics reliably. The analyzer instructs the local model to return *only* a JSON array of security issues.

Listing A.1: JSON-only response schema (conceptual)

```
[
  {
    "message": "Issue description",
    "startLine": 1,
    "endLine": 3,
    "suggestedFix": "Optional secure alternative"
  }
]
```

Robustness in practice

Even with explicit instructions, some models occasionally emit Markdown fences or additional text. The prototype therefore applies defensive parsing: it removes common code-block markers and extracts the first JSON array substring before attempting to parse. This is a pragmatic mechanism to improve structured-output robustness for IDE integration.

A.2. Runtime Guardrails

To keep the extension usable on developer hardware (R4), Code Guardian includes conservative guardrails:

- **Debounce interval:** 800 ms for real-time analysis after document changes.
- **Function scope size limit:** extracted functions larger than 2000 characters are skipped in real-time mode.
- **File scope size limit:** full-file analysis is skipped above 20,000 characters.
- **Workspace scan file size limit:** files larger than 500 KB are skipped in batch scans.

In addition, analysis results are cached using an LRU-style cache (100 entries, 30-minute TTL) to reduce redundant inference calls during iterative edits.

A.3. Key Configuration Options

The extension exposes user configuration through VS Code settings. The most relevant options are:

- `codeGuardian.model`: default Ollama model for analysis
- `codeGuardian.ollamaHost`: Ollama base URL (default: `http://localhost:11434`)
- `codeGuardian.enableRAG`: enable/disable retrieval augmentation

A.4. VS Code Commands (Prototype)

The prototype exposes the following main commands via the Command Palette:

- **Analyze selected code with AI**: opens the interactive analysis view for a selection or current line.
- **Analyze full file**: runs the structured diagnostics pipeline over the active document.
- **Contextual Q&A**: opens a WebView for asking security questions with user-selected file/folder context.
- **Select AI model**: lists available local Ollama models and switches the active model.
- **Manage RAG knowledge base**: view/add/search knowledge, rebuild the vector store, and update vulnerability data.
- **Toggle RAG**: enables/disables retrieval augmentation through settings.

- **Update vulnerability data**: refreshes cached public metadata used for the knowledge base.
- **View cache statistics**: inspects the analysis cache and supports clearing/resetting statistics.
- **Workspace security dashboard**: performs a batch scan and displays an aggregate dashboard.

A.5. Evaluation Harness Location

The evaluation script and curated datasets are included in the Code Guardian repository alongside the extension source code.

B. Evaluation Details

This appendix documents the evaluation dataset composition, run configuration, and per-case schema beyond what fits in the evaluation chapter.

B.1. Curated Test Suite Overview

The curated evaluation suite contains JavaScript/TypeScript vulnerability snippets with expected CWE/severity metadata.

Dataset sizes

For the scored thesis run in Chapter 5, the harness uses:

- **Primary vulnerable set:** `vulnerability-test-cases.generated.json` with 71 vulnerable cases
- **Secure overlay set:** `negatives-only.generated.json` with 30 secure/negative cases

Thesis tables are derived from merged run artifacts (effective 101-case set: 71 vulnerable + 30 secure).

Representative Vulnerability Classes

Included classes (non-exhaustive):

- SQL injection (CWE-89)
- Cross-site scripting (CWE-79)
- Command injection (CWE-78)
- Path traversal (CWE-22)
- Insecure randomness (CWE-338)
- Hardcoded credentials (CWE-798)

- CSRF and authentication-flow weaknesses (CWE-352, CWE-287)
- Prototype pollution / unsafe reflection patterns (CWE-1321, CWE-470)

Test Case Record Format

Each test case includes:

- a code snippet (`code`),
- a list of expected findings (`expectedVulnerabilities`),
- optional remediation guidance (`expectedFix`).

Reproducing the harness run

The evaluation script is executed locally:

Listing B.1: Running the evaluation script

```
cd code-guardian-extension
node evaluation/evaluate-models.js --ablation --include-baselines \
  --dataset=datasets/vulnerability-test-cases.generated.json --runs=3 -
  --num-predict=1000 --timeout-ms=30000 --delay-ms=500
node evaluation/evaluate-models.js --ablation --include-baselines \
  --dataset=datasets/negatives-only.generated.json --runs=1 --rag-k=5 -
  --num-predict=1000 --timeout-ms=30000 --delay-ms=500
```

The script reports per-model precision/recall/F1, FPR, latency, and parse success.

Merged Baseline Snapshot

From the merged artifact (`71 vulnerable + 30 secure`), baseline tools measured:

- `semgrep`: Precision 57.89%, Recall 9.73%, F1 16.67%, FPR 6.67%
- `codeql`: Precision 15.71%, Recall 9.73%, F1 12.02%, FPR 53.33%
- `eslint-security`: Precision 0.00%, Recall 0.00%, F1 0.00%, FPR 0.00%

Baselines run on a temporary snippet workspace. Semgrep uses `p/security-audit`, `p/javascript`, `p/typescript`, and `p/owasp-top-ten`; CodeQL uses `javascript-security-and-` ESLint uses `eslint-plugin-security`. Findings are normalized into the harness schema via heuristic type inference, so category-level comparisons should be interpreted as approximate.

Artifact Checklist

For reproducibility, archive:

- Raw run output JSON from the harness (all configurations and per-case results)
- Exact run configuration object (models, prompt modes, runtime parameters)
- Model digests / quantization metadata used for inference
- Environment metadata (OS, Node.js, Ollama versions, hardware)
- scripts/tables used to derive reported descriptive metrics

The full dataset is machine-readable in the repository.

C. Detailed Case Studies

This appendix walks five representative outcomes from the evaluation — true positive with effective repair, false positive on secure code, false negative on a context-dependent pattern, partial repair, and a multi-CWE finding — and shows how the measured aggregate metrics translate into per-case IDE behaviour.

C.1. Case 1: True Positive with Effective Repair

SQL injection in an Express route detected by `qwen3:8b+RAG`.

Listing C.1: SQL injection vulnerability (CWE-89) and the model's repair

```
// Vulnerable
app.get('/user/:id', (req, res) => {
  const userId = req.params.id;
  const query = `SELECT * FROM users WHERE id = ${userId}`;
  db.query(query, (err, results) => res.json(results[0]));
});
// Model output: {"cwe":"CWE-89","lineStart":3,"lineEnd":3,
//   "fixedCode":"const query = 'SELECT * FROM users WHERE id = ?';\ndb
```

Clean true positive with correct location and CWE, minimal parameterised repair, intent preserved. Low-friction remediation that developers can apply immediately.

C.2. Case 2: False Positive (Over-Sensitivity)

Secure username sanitisation flagged by `gemma3:4b` (LLM-only).

Listing C.2: Secure input validation flagged as CWE-94

```
function sanitizeUsername(username) {
  const sanitized = username.replace(/[^a-zA-Z0-9_]/g, '');
  if (sanitized.length < 3 || sanitized.length > 20) throw new Error('
  return sanitized;
}
// Model output: {"cwe":"CWE-94","message":"Potential code injection in
```

The whitelist regex and length checks are appropriate; the CWE label is mismatched. Illustrates the high-FPR behaviour of smaller configurations (Section 5.4) and the triage overhead it imposes on developers.

C.3. Case 3: False Negative

Prototype pollution missed by `CodeLlama` (LLM-only).

Listing C.3: Prototype pollution (CWE-1321) returning empty findings

```
function mergeConfig(target, source) {
  for (let key in source) target[key] = source[key];
  return target;
}
mergeConfig(globalConfig, JSON.parse(req.body.config));
// Model output: []
```

Attacker-controlled `__proto__` keys are not flagged. A robust output would suggest guarded assignment (`Object.prototype.hasOwnProperty.call(source, key)`); demonstrates the recall limits of weaker model families.

C.4. Case 4: Detection with Partial Repair

Path traversal detected; repair is only partial.

Listing C.4: Path traversal (CWE-22) with under-specified repair

```
app.get('/download', (req, res) => {
  const filepath = path.join(__dirname, 'uploads', req.query.file);
  res.download(filepath);
});
// Model output: {"cwe":"CWE-22","fixedCode":"const filename = path.bas
```

`path.basename()` reduces traversal but not authorisation; allowlist checks remain. Shows why repairs are advisory and require project-specific review (Section 5.6).

C.5. Case 5: Multi-CWE Detection

Hard-coded admin token plus command injection in one endpoint, both detected by `qwen3:8b+RAG`.

Listing C.5: Two findings from one handler (CWE-798 + CWE-78)

```
app.post('/admin/exec', (req, res) => {
  if (req.headers['x-auth-token'] === 'admin123') { // CWE-798
    exec(req.body.command, (err, out) => res.send(out)); // CWE-78
  } else res.status(401).send('Unauthorized');
});
// Model output: [{"cwe":"CWE-798","lineStart":3},{"cwe":"CWE-78","line
```

Both issues are correctly separated, which matters for IDE diagnostics and prioritisation. The credential fix is straightforward; command-execution remediation is design-sensitive.

The five cases align with the quantitative results in Chapter 5: strong model-dependent variation, persistent false-positive pressure in some modes, and a consistent need for developer oversight on repairs.

D. Privacy Verification Evidence

This appendix records the privacy evidence behind R5: network-traffic observation, configuration, and the trust-boundary architecture.

D.1. Network Traffic Analysis

Outbound traffic was monitored during representative inline and audit workflows on the evaluation environment (Section 5.3) using

```
sudo tcpdump -i any -n 'not (host 127.0.0.1 or host ::1)' -w /tmp/cg.pc
```

Detection and repair workflows produced no outbound requests; communication stayed on localhost between the extension host, local backend (`localhost:3000`), and Ollama (`localhost:11434`). External requests appeared only when knowledge refresh was explicitly triggered, contacting public CVE/CWE/OWASP endpoints with no source code or project artefacts in the request bodies. The same observation is reproduced quantitatively by the automated egress harness and source-leak detector (Section 5.8.2).

Table D.1.: Privacy-relevant configuration parameters in the evaluated setup.

Parameter	Purpose	Value
`ollama.baseUrl`	LLM inference endpoint	`http://localhost:11434`
`backend.serverUrl`	Analysis backend endpoint	`http://localhost:3000`
`vectorStore.provider`	Retrieval backend	`local` (HNSW)
`codeGuardian.enableExternalDataFetch`	External metadata refresh	`false`
`codeGuardian.requireSignedCorpus`	Reject unsigned RAG corpus	`true`
`telemetry.enabled`	Telemetry collection	`false`

D.2. Privacy Boundary Architecture

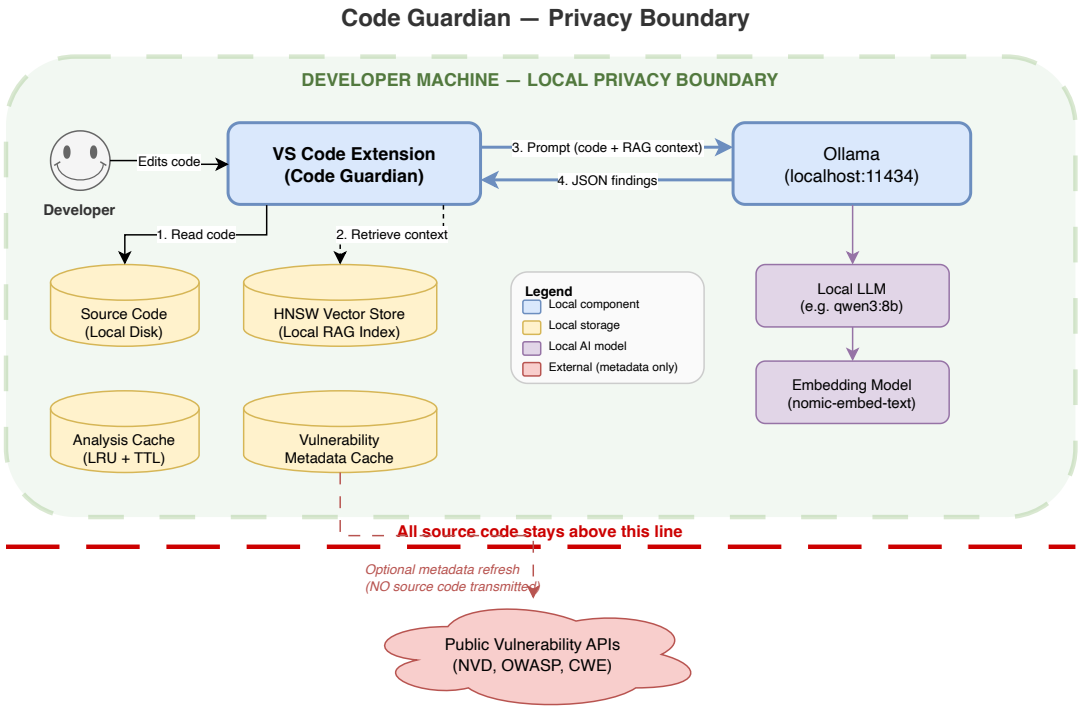

Figure D.1.: Local privacy boundary for code analysis workflows.

The IDE workspace, extension process, local backend, Ollama runtime, and vector store all reside inside the local trust boundary; source files, extracted snippets, prompts, responses, and embeddings never cross it. The only outbound channel is the public-metadata refresh, which carries no user code and is gated by `enableExternalDataFetch` (default `false`). The boundary does not mitigate local host compromise, physical access, or malicious third-party software on the same machine; those risks require endpoint and operational controls beyond thesis scope.

Fully offline operation is supported by disabling external fetch, using the pre-seeded signed RAG corpus, and ensuring required local models are already pulled into Ollama.

Declaration of Originality

Selbständigkeitserklärung

Ich erkläre, dass ich die vorliegende Arbeit selbständig und nur unter Verwendung der angegebenen Literatur und Hilfsmittel angefertigt habe. Alle Stellen der Arbeit, die wörtlich oder sinngemäß aus anderen Quellen übernommen wurden, sind als solche kenntlich gemacht.

Declaration of Originality

I hereby declare that I have written this thesis independently and have not used any sources or aids other than those indicated. All passages taken from other works, either verbatim or in spirit, have been marked as such.

Chemnitz, 14.05.2026

_____

Md Hafizur Rahman